

챕터 1. MSA란 무엇인가?

이번 챕터가 끝나면

- 모놀리식 아키텍처의 한계를 이해할 수 있습니다.
- 마이크로서비스 아키텍처가 어떤 방식으로 문제를 해결하는지 이해할 수 있습니다.
- 이 책에서 만들 쇼핑몰 주문 시스템의 전체 구조를 파악할 수 있습니다.
- MSA의 핵심 과제인 분산 트랜잭션과 그 해법(보상 트랜잭션·비동기 메시지)을 이해할 수 있습니다.
- 챕터 2~5의 학습 흐름을 파악할 수 있습니다.

준비하기. 챕터 2부터 시작될 실습을 위해 미리 준비

이 챕터는 개념만 다루므로 직접 코드를 작성하지 않습니다. 챕터 2부터 실습이 시작되니, 그전에 도구 설치와 레포 위치를 미리 확인해 두세요.

1. 실습 환경

도구	사용 챕터	설치 주소
Docker Desktop	챕터 2~	https://www.docker.com/products/docker-desktop/
Minikube	챕터 3~	https://minikube.sigs.k8s.io/
Hoppscotch (브라우저 확장)	챕터 2~	https://hoppscotch.io/

Docker Desktop을 설치하고 실행한 뒤 "Engine running" 상태인지 확인합니다.

2. 챕터별 소스 코드

이 책의 실습은 챕터마다 GitHub 레포가 하나씩 대응합니다. 챕터 2부터 해당 레포를 클론하여 진행합니다.

챕터	레포	주제
챕터 2	github.com/metacoding-12-msa/ex01	동기 REST + 보상 트랜잭션
챕터 3	github.com/metacoding-12-msa/ex02	DDD + 클린 아키텍처 + Kubernetes
챕터 4	github.com/metacoding-12-msa/ex03	Kafka + Orchestration Saga
챕터 5	github.com/metacoding-12-msa/ex04	WebSocket 실시간 알림

3. 진행 순서

챕터 2부터 챕터 5까지 한 시스템이 단계적으로 진화합니다. **챕터 2부터 순서대로 진행하세요.**

새벽 세 시. 휴대폰 알람이 울렸다. 가까운 데서 한 번, 멀리서 한 번. 침대 옆 협탁의 알람과 거실 노트북의 알람이 동시에 울려대고 있었다.

화면을 열었다. 모니터링 대시보드가 빨갭게 차 있었다. 에러 그래프가 가파른 절벽처럼 솟아 있었다. 주문 API가 죽었고, 그 옆에 로그인 API도 같이 죽어 있었다.

주문이랑 로그인이 무슨 상관이야?

서버 한 대를 재시작했다. 그래프가 잠시 떨어지더니 다시 솟았다. 또 한 대. 또. 마지막 한 대를 재시작하자 알람이 조용해졌다. 시계를 보니 새벽 네 시 반이었다.

월요일 아침, 팀장이 자리로 왔습니다.

팀장 : "왜 같이 죽었어요? 주문이 몰린 건데 로그인이 왜 같이 멈췄요? 방법 좀 찾아 봐요."

오픈이가 답을 못 했습니다. 서버를 다시 살린 건 자기지만, 왜 같이 죽었는지는 모릅니다. 사무실 끝에 앉은 선배 자리로 걸어갔습니다.

오픈이 : "선배님, 금요일에 주문이 몰렸는데 로그인까지 같이 죽었어요. 주문이랑 로그인이 무슨 상관인데 같이 멈추는 건지 모르겠어요."

선배 : "한 건물에 가게를 다 넣어두면 그래요. 한 층에서 불나면 전체가 대피해야 하잖아요."

1.1 모놀리식 - 쇼핑몰을 하나의 서버로 만들면 어떻게 될까?

1.1.1 처음에는 아무 문제가 없었다

백화점을 떠올려 보세요. 수십 개의 매장이 한 건물 안에 모여 있습니다. 고객은 한 곳에서 모든 것을 해결할 수 있습니다. 운영사 입장에서는 전기·냉방·보안·고객 데이터를 한 곳에서 통합 관리합니다. 이 구조는 단순하고 효율적입니다.



그림 1-1. 백화점 - 모든 매장이 한 건물에 모여 있는 구조

소프트웨어도 같은 이야기입니다. 처음에는 하나의 서버에 모든 기능을 넣는 **모놀리식** 구조가 단순하고 빠릅니다.

1.1.2 성장하면서 균열이 생긴다

백화점이 잘 돼서 방문객이 열 배로 늘었습니다. 이제 문제가 보이기 시작합니다. 한 매장에서 불이 나도 전 층이 영업을 멈춥니다. 한 매장에 손님이 몰려도 그 매장만 따로 늘릴 수 없으니 건물 전체를 새로 지어야 합니다. 전기 배선 한 번 교체하려면 공사 기간 내내 전 층이 닫힙니다.

소프트웨어 세계에서도 똑같은 일이 벌어집니다. 회원 10만 명, 하루 주문 1만 건이 되었을 때, 모놀리식 구조의 균열이 드러납니다.

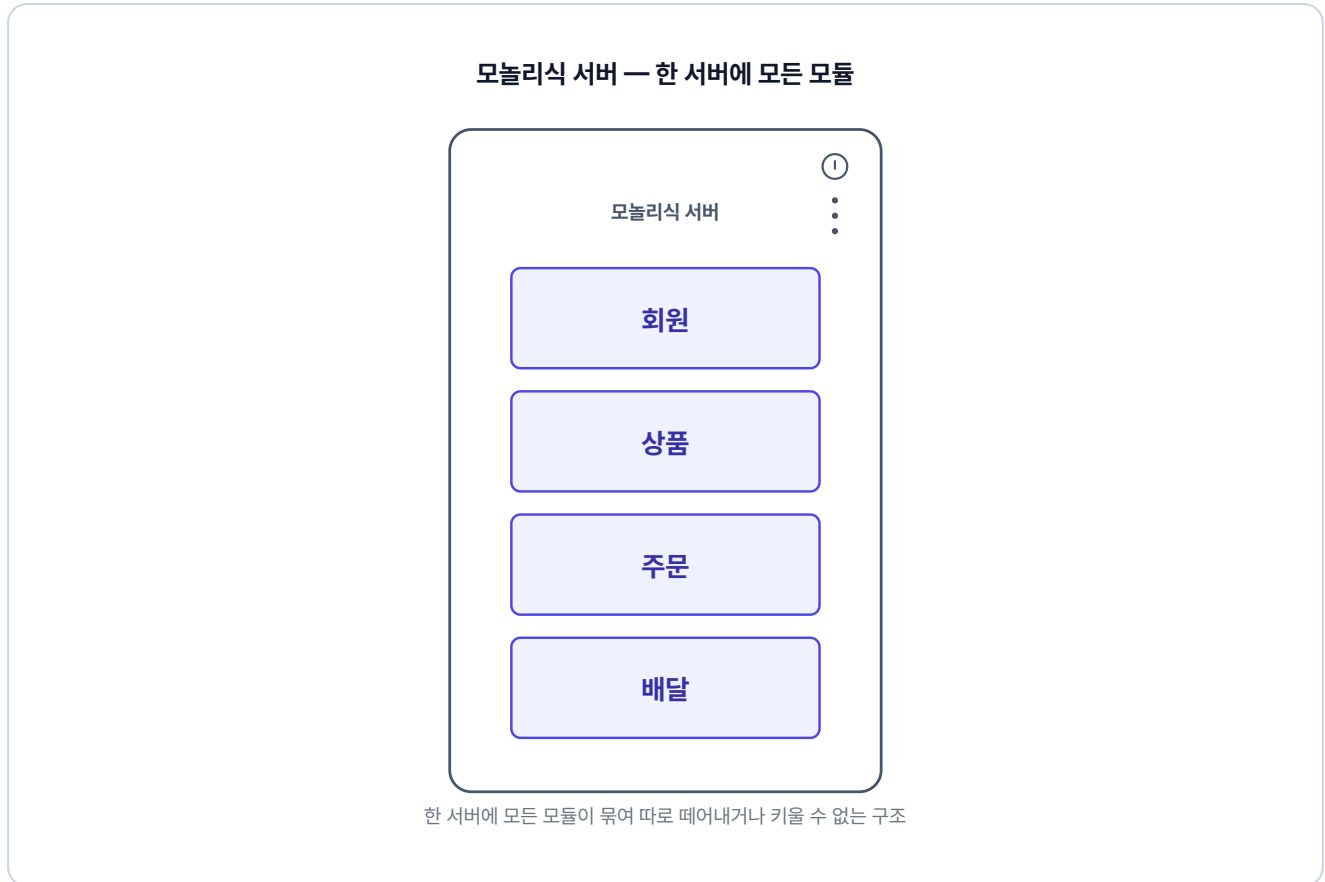


그림 1-2. 모놀리식 쇼핑몰 - 모든 기능이 하나의 서버에

배포가 두렵습니다. 주문 기능 하나를 수정해도 배포 시 회원·상품·배달까지 전부 재시작해야 합니다.

장애가 전파됩니다. 배달 기능 버그로 서버가 느려지면 배달과 무관한 회원 로그인도 함께 느려집니다.

확장이 어렵습니다. 블랙프라이데이에 주문 기능만 서버를 늘리고 싶어도 모놀리식에서는 전체 서버를 복제해야 합니다.

팀이 커지면 충돌이 잦아집니다. 10명이 하나의 코드베이스를 동시에 수정하면 하루에도 수십 번 충돌이 발생합니다.

이래서 서버가 전부 같이 죽었구나. 한 건물에 다 넣어뒀으니까.

오픈이 : "그럼 어떻게 해야 하나요?"

선배 : "백화점 대신 개별 상점을 열면 돼요. 각자 건물을 가지면 한 곳에서 불이 나도 나머지는 멀쩡하잖아요."

1.2 마이크로서비스 - 역할을 나눈다

1.2.1 백화점 vs 개별 상점

개별 상점 방식은 백화점과 구조 자체가 다릅니다. 각 매장이 독립된 건물로 운영되어, 자신만의 전기·냉방·입구를 가집니다.



그림 1-3. 개별 상점 - 각 매장이 독립된 건물로 운영되는 구조

백화점 구조와 비교하면 차이가 분명합니다.

	백화점	개별 상점
한 매장 화재 발생	스프링클러·대피령으로 전 층 영업 중단	해당 매장만 소방 처리, 나머지 정상 영업
전자제품 수요 폭증	건물 전체를 확장해야 함	전자제품점만 확장
건물 전기 배선 교체	공사 기간 동안 전 층 영업 중단	해당 매장만 임시 폐쇄, 나머지 정상 영업

마이크로서비스 아키텍처가 바로 이 방식입니다. 하나의 큰 서버 대신, 기능별로 작은 서비스들을 분리합니다.



그림 1-4. 마이크로서비스 쇼핑몰 - 기능별로 서비스를 분리

이제 각 서비스는 독립적으로 배포하고, 독립적으로 확장할 수 있습니다. 주문 서비스에 버그가 생겨도 회원 서비스는 영향을 받지 않습니다.

1.2.2 MSA vs 모놀리식

특성	모놀리식	마이크로서비스
배포	전체 재배포	해당 서비스만 배포
장애 격리	전체 영향	해당 서비스만 영향
확장	전체 복제	필요한 서비스만 확장
팀 분리	하나의 코드베이스	서비스별 독립 개발

오픈이 : "우리 쇼핑몰로 치면 어떻게 나누면 되나요?"

선배 : "회원, 상품, 주문, 배달. 이 네 개부터 떼어내 봐요."

1.3 시스템과 핵심 과제

문제를 이해했으니, 이제 직접 만들어볼 시스템을 설계해 보겠습니다.

1.3.1 만들 시스템

이 책의 쇼핑몰 주문 시스템은 4개의 마이크로서비스로 구성됩니다.

서비스	포트	역할
회원 서비스	8083	로그인, JWT 발급, 사용자 조회
상품 서비스	8082	상품 목록, 재고 조회 및 증감
주문 서비스	8081	주문 생성·조회·취소 (핵심)
배달 서비스	8084	배달 생성·조회·취소

주문 서비스가 중심에서 다른 서비스를 엮습니다. 사용자가 주문하면 상품 서비스의 재고를 차감하고, 배달 서비스에 배달을 만듭니다.

1.3.2 분산 트랜잭션 — 이 책의 핵심 과제

서비스를 분리하면 곧장 새로운 문제가 생깁니다. 모놀리식에서는 `@Transactional` 한 줄로 주문·재고·배달을 자동 롤백할 수 있었습니다. 그런데 MSA에서는 각 서비스가 독립된 데이터베이스를 가집니다. 한쪽에서 일어난 일을 다른 쪽이 모르므로, 자동 롤백이 불가능합니다. 이렇게 여러 DB에 걸친 작업을 하나로 묶어야 하는 상황을 **분산 트랜잭션**이라고 합니다.

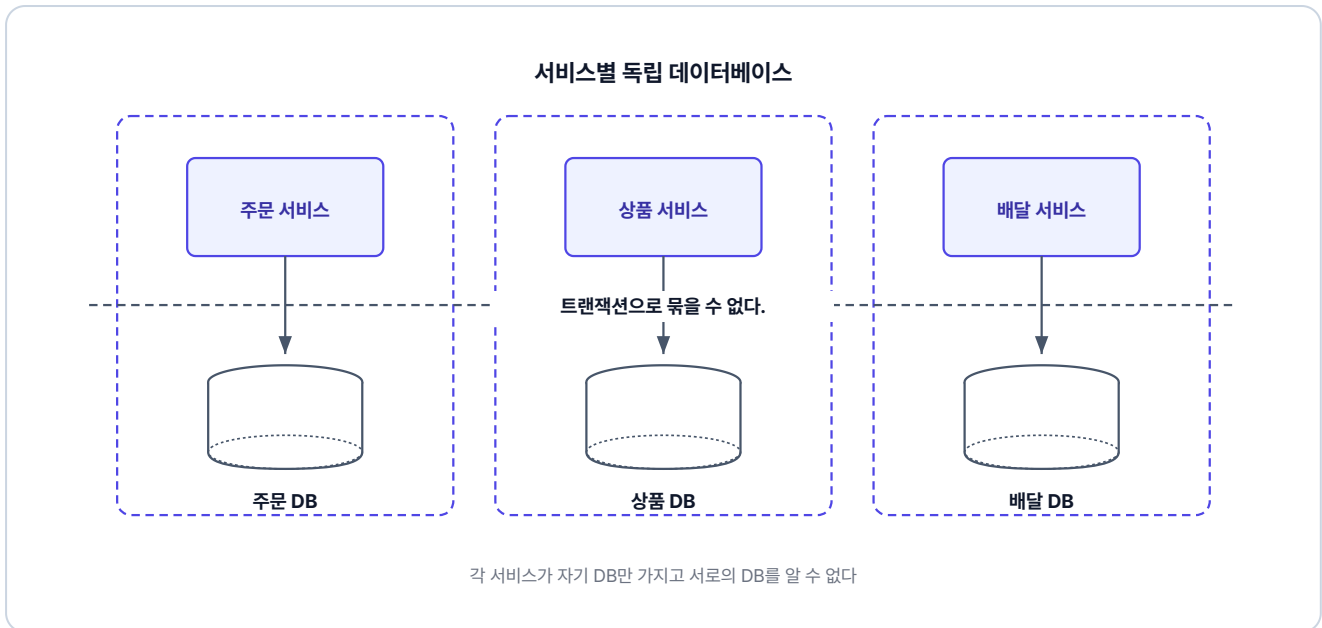


그림 1-5. 서비스별 독립 데이터베이스

분산 트랜잭션(Distributed Transaction)이란? 여러 독립된 데이터베이스에 걸친 작업을 하나의 논리적 단위로 처리해야 하는 상황입니다. MSA에서는 서비스마다 DB가 분리되어 있으므로 단일 트랜잭션이 불가능하고, 별도의 전략이 필요합니다.

재고는 줄였는데 배달이 실패하면... 자동으로 안 돌아간다고?

오픈이 : "선배님, 이거 되돌리는 방법이 없나요?"

선배 : "자동으로 안 되니까 직접 짜는 거죠. 재고를 줄였으면 다시 늘리고, 배달을 만들었으면 취소하고. 한 단계씩 거꾸로 되돌리는 거예요. 보상 트랜잭션이라고 해요."

보상 트랜잭션(Compensating Transaction)이란? 중간에 실패가 나면 이미 끝낸 작업을 역순으로 돌려 원래 상태로 되돌리는 방법입니다.

1.3.3 분산 트랜잭션을 푸는 두 가지 방식

이 책에서는 분산 트랜잭션을 두 가지 방식으로 풀어 갑니다.

방식 1. 직접 호출과 보상 (챕터 2, 챕터 3)

각 서비스가 다음 서비스를 **직접** 부릅니다. 주문 서비스가 상품 서비스에 "재고 줄여줘"를 요청해 처리하다가 뒤에서 실패하면, 다시 직접 "재고 돌려줘"를 요청해 되돌립니다. 호출자가 응답까지 멈춰 있는 동기 호출 방식이라, 한 호출이 1초 걸리면 사용자도 그만큼 기다립니다.



그림 1-6. 동기 REST - 서비스 간 직접 호출과 보상

단순한 구현이지만, 서비스 수가 늘어날수록 보상 코드가 깊어집니다.

방식 2. 메시지로 비동기 전환 (챕터 4, 챕터 5)

서비스끼리 직접 부르지 않고 메시지로 주고받습니다. 보내는 쪽은 결과를 기다리지 않으므로(비동기), 한 서비스가 느려도 다른 서비스가 멈추지 않습니다. 전체 흐름을 조율하는 별도 서비스는 뒤에서 자세히 다룹니다.



그림 1-7. 비동기 메시지 - 서비스를 분리하는 구조

지휘자가 전체 흐름을 알고 있어 실패 시 자동으로 롤백 명령을 내립니다. 두 방식을 모두 직접 구현해 보면, 각각의 장단점이 자연스럽게 체감됩니다.

1.4 이 책의 학습 흐름

이제 전체 그림이 보입니다. 이 책은 하나의 시스템이 단계별로 진화하는 여정입니다. 각 챕터는 이전 챕터의 한계를 느끼는 것에서 시작합니다.

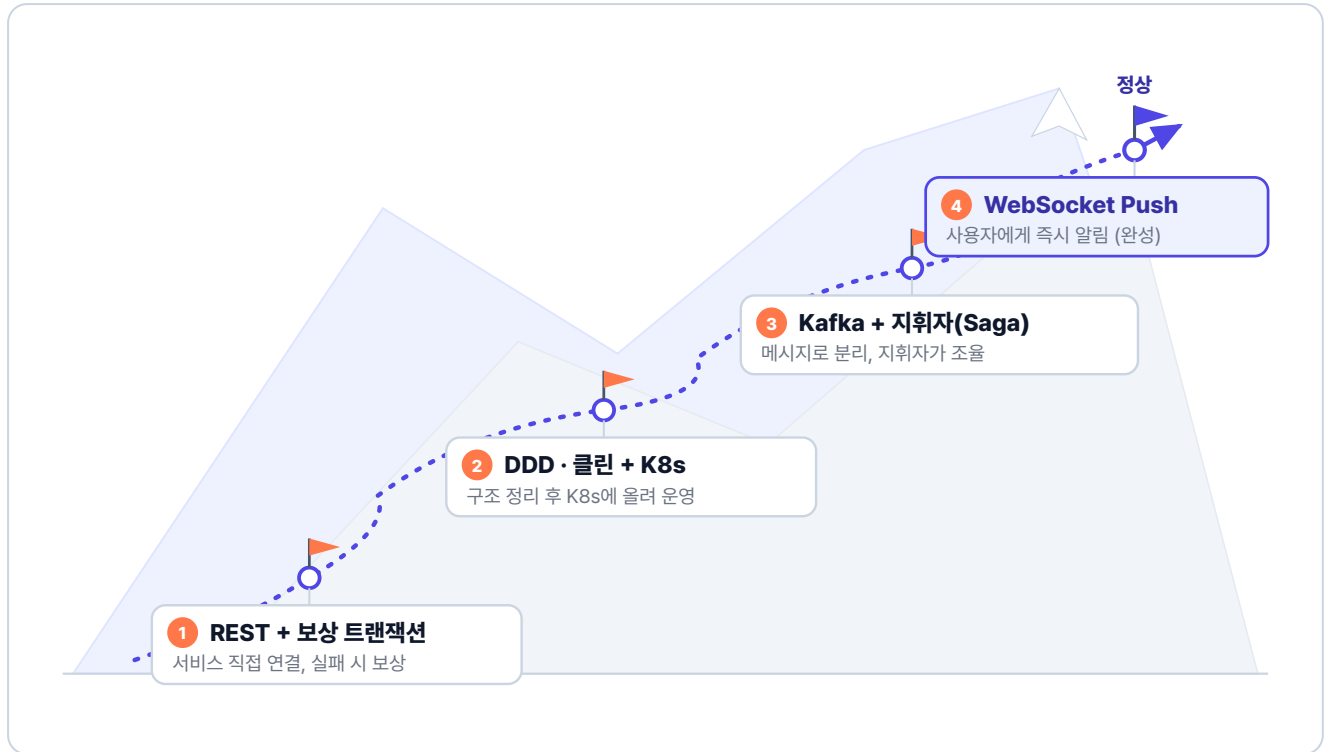


그림 1-8. 이 책의 학습 흐름

가장 단순한 방법, REST로 네 서비스를 직접 잇는 것에서 출발합니다. 뼈대가 서고 나면 호출이 뒤엉켜 손대기 어려워지므로, 도메인을 중심으로 구조를 정리해 쿠버네티스에 올립니다. 구조가 깔끔해져도 동기 호출은 서로를 기다리다 함께 멈추기에, 메시지로 서비스를 분리해 비동기로 바꿉니다. 끝으로 처리가 끝난 순간, 사용자에게 실시간으로 알립니다.

선배 : "개념은 이 정도면 됐어요. 이제 직접 만들어 봐요. 제일 단순한 방법부터."

손가락이 움직이지 않았던 게 엇그제인데, 이제 뭘 만들어야 할지는 보인다.

이것만은 기억하자

- **모놀리식**은 처음에는 단순하지만, 서비스가 커지면 배포·장애·확장 문제가 생깁니다.
- **마이크로서비스**는 기능별로 서비스를 분리하여 각자 독립적으로 배포하고 확장할 수 있게 합니다.
- MSA의 핵심 과제는 **분산 트랜잭션**입니다. 각 서비스의 DB가 분리되어 있어 단일 트랜잭션으로 묶을 수 없습니다.
- 분산 트랜잭션을 두 가지 방식으로 풀니다. 챕터 2~3에서는 **보상 트랜잭션**으로 서비스끼리 직접 호출·보상하고, 챕터 4~5에서는 메시지로 비동기 통신해 서비스를 분리합니다.
- 이 책은 하나의 쇼핑몰 주문 시스템이 챕터 2부터 챕터 5까지 단계적으로 진화하는 이야기입니다.

이제 직접 코드를 작성할 시간입니다. 챕터 2에서는 4개의 서비스를 REST로 연결하고, 보상 트랜잭션을 구현해 봅니다. 처음에는 단순하게 시작합니다. 그 단순함이 나중에 어떤 문제를 만드는지, 챕터 2에서 직접 느껴 봅니다.

챕터 2. 동기식 MSA 구현 - 서비스를 연결하다

이 챕터의 전체 소스코드는 <https://github.com/metacoding-12-msa/ex01> 에서 확인할 수 있습니다.

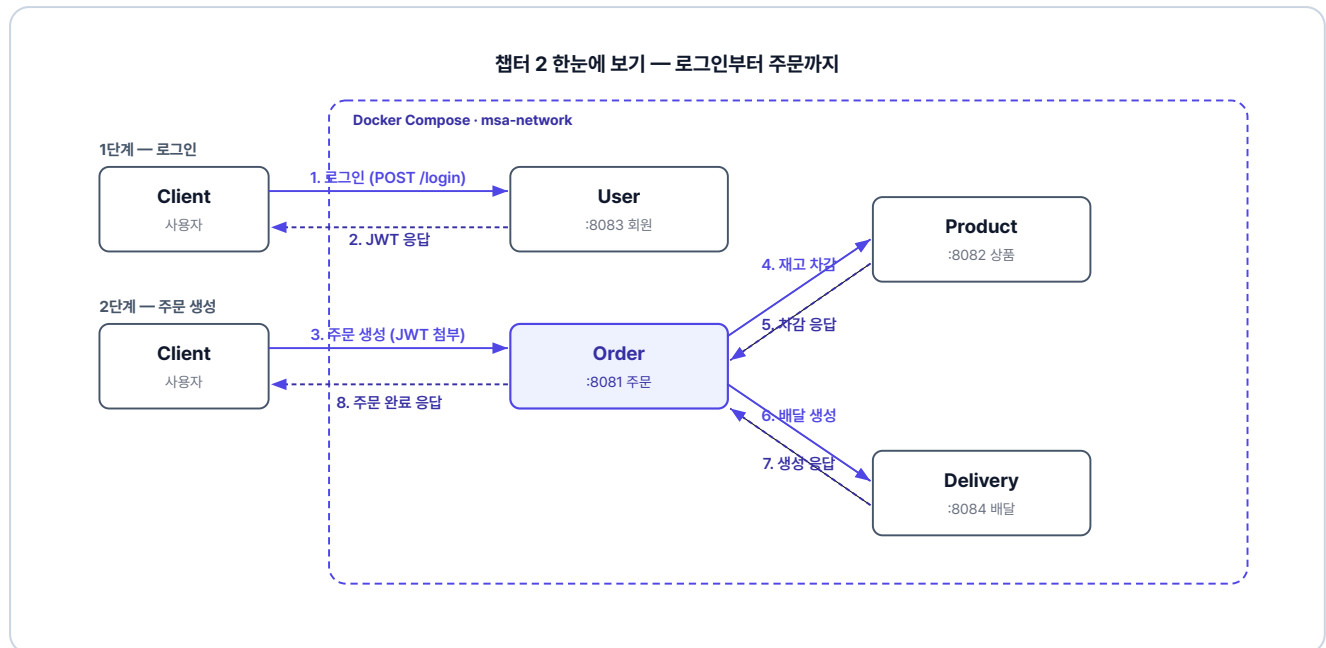


그림 2-1. 챕터 2 한눈에 보기 - 로그인부터 주문까지

이번 챕터가 끝나면

- Spring Boot로 4개 서비스를 독립적으로 실행하고 REST로 연결할 수 있습니다.
- JWT 인증 필터를 구현하고 서비스 간 Authorization 헤더를 전달할 수 있습니다.
- 주문 생성 시 재고 감소 → 배달 생성 흐름을 동기적으로 구현할 수 있습니다.
- 중간에 실패가 발생했을 때 이전 작업을 되돌리는 보상 트랜잭션을 작성할 수 있습니다.

준비하기. 실습 시작 전 한 번만 설정

1. 소스 코드 클론

터미널 레포 클론

```
git clone https://github.com/metacoding-12-msa/ex01.git  
cd ex01
```

2. 파일 구조

ex01 디렉토리

```
ex01/  
├─ user/           # 포트 8083  
├─ product/        # 포트 8082  
├─ order/          # 포트 8081  
├─ delivery/       # 포트 8084  
└─ docker-compose.yml # 전체 서비스 실행
```

각 서비스 내부는 동일한 구조입니다. 주문 서비스 기준으로 보여드리며, 회원/상품/배달 서비스도 같은 구조입니다.

주문 서비스 패키지 구조

```
src/main/java/com/metacoding/order/
├── OrderApplication.java           # [참고]
├── core/
│   ├── config/
│   │   ├── WebConfig.java         # [참고] JWT 필터 등록
│   │   └── RestClientConfig.java   # [참고] JWT 헤더 전달 인터셉터
│   ├── filter/
│   │   └── JwtAuthenticationFilter.java # [참고] JWT 인가 필터
│   ├── handler/
│   │   ├── GlobalExceptionHandler.java # [참고] 전역 예외 처리
│   │   └── ex/                     # 커스텀 예외 (Exception400~500)
│   └── util/
│       ├── JwtProvider.java        # [참고] JWT 파싱/검증
│       ├── JwtUtil.java            # [참고] JWT 생성
│       └── Resp.java               # [참고] 표준 응답 래퍼
├── orders/
│   ├── Order.java                 # [참고] JPA 엔티티
│   ├── OrderStatus.java           # [참고] 주문 상태 enum
│   ├── OrderController.java        # [참고] REST 컨트롤러
│   ├── OrderService.java           # [작성] 비즈니스 로직
│   ├── OrderRepository.java        # [참고] Spring Data JPA
│   └── OrderRequest.java / OrderResponse.java # [참고]
├── adapter/                       # 주문 서비스에만 존재
│   ├── ProductClient.java          # [참고] 상품 서비스 호출
│   ├── DeliveryClient.java         # [참고] 배달 서비스 호출
│   └── dto/                        # 어댑터용 DTO (ProductRequest, DeliveryRequest)
└── Dockerfile                     # [참고] Docker 이미지 빌드
```

회원/상품/배달 서비스는 `adapter/` 패키지와 `RestClientConfig` 가 없고, 나머지 구조는 동일합니다.

3. 실습 환경

도구	용도	비고
Docker Desktop	4개 서비스를 컨테이너로 실행	https://www.docker.com/products/docker-desktop/
Hoppscotch	API 호출 결과 확인	https://hoppscotch.io/ (설치 불필요, 브라우저 확장만 추가)

4. 실습 순서

1. 공통 설정(JWT·표준 응답·예외 처리)을 `core/` 패키지에 두기
2. 회원·상품·배달 서비스의 핵심 코드 살펴보기
3. 주문 서비스에 RestClient + 보상 트랜잭션 작성하기
4. Docker Compose로 4개 서비스를 한 번에 띄우고 시나리오 3개 검증하기

오픈이 : "선배님, 서비스 네 개로 나누는 건 알겠는데, 이걸 어떻게 코드로 옮겨요?"

선배 : "일단 각자 따로 띄워서 REST로 연결해 봐요. 서로 전화를 거는 것처럼 직접 호출하는 거예요. 제일 단순한 방법이죠."

방향이 보였습니다. 이제 만들어 봅시다.

주문 서비스가 이번 챕터의 핵심입니다. 주문 하나를 만들려면 상품 서비스의 재고를 줄이고, 배달 서비스에 배달을 만들어야 합니다. 머릿속에 흐름을 그려 봤습니다.

주문 저장 → 상품 재고 차감 → 배달 생성 → 주문 완료

재고를 줄였는데, 그 다음 배달 생성이 실패하면... 어떻게 되돌리지?

모놀리식이라면 `@Transactional` 한 줄이 다 롤백해 줍니다. 그런데 HTTP로 보낸 재고 차감 요청은 되돌릴 수 없습니다. 상품 DB의 재고는 줄어든 채로 남고, 자동으로 재고가 복구되지 않습니다.

챕터 1에서 머리로만 알았던 분산 트랜잭션을 그제야 실감했습니다. 선배 자리로 갔습니다.

오픈이 : "선배님, 재고는 줄였는데 그 다음 배달 생성이 실패하면요? 자동으로 안 돌아가요?"

선배 : "기억나죠? 보상 트랜잭션. 자동으로 안 되니 직접 짜는 거예요. 재고를 줄였으면 다시 늘리고, 배달을 만들었으면 취소하고. 어디까지 진행됐는지 플래그로 기록하고. 그게 **보상 트랜잭션**이에요."

앞에서 머리로만 알던 보상 트랜잭션을, 손으로 직접 짜는 거였구나.

2.1 이야기의 시작 - 네 개의 서비스가 만나다

챕터 1에서 설계한 네 개의 서비스(회원, 상품, 주문, 배달)를 이제 직접 만들어 보겠습니다.

주문 서비스가 두 서비스를 엮는 자리입니다. 주문을 생성하려면 상품 서비스에서 재고를 줄이고, 배달 서비스에서 배달을 만들어야 합니다. 즉 주문 서비스가 두 서비스를 직접 호출해야 합니다.

각 서비스는 독립된 스프링 프로젝트입니다. 하나의 서비스를 배포할 때 다른 서비스를 건드릴 필요가 없습니다. 디렉토리 구조와 패키지 구조는 준비하기에서 미리 살펴보았습니다.

2.2 공통 설정 - 모든 서비스가 공유하는 뼈대

MSA에서는 서비스마다 서버가 다르므로 세션을 공유할 수 없습니다. 대신 JWT 토큰을 발급하고, 각 서비스가 토큰만 검증하는 방식을 사용합니다.

4개 서비스 모두 JWT 인증, 표준 응답 형식, 예외 처리가 필요합니다. 이를 `core/` 패키지에 모아 두면, 각 서비스는 비즈니스 로직에 집중할 수 있습니다.

각 컴포넌트의 역할은 다음과 같습니다.

컴포넌트	역할
<code>JwtAuthenticationFilter</code>	매 요청마다 JWT를 검증하고 사용자 정보를 컨트롤러로 전달합니다.
<code>JwtUtil / JwtProvider</code>	<code>JwtUtil</code> 은 토큰을 발급하고, <code>JwtProvider</code> 는 토큰을 검증합니다.
<code>Resp</code>	모든 API 응답을 동일한 형태로 통일하는 래퍼입니다.
<code>GlobalExceptionHandler</code>	전역에서 발생한 예외를 잡아 일관된 에러 응답으로 변환합니다.
<code>WebConfig</code>	JWT 필터를 인증이 필요한 경로에 등록합니다.

공통 설정이 준비되었습니다. 주문 서비스의 보상 트랜잭션을 직접 작성하기 전에, 나머지 세 서비스의 핵심 코드를 먼저 살펴보겠습니다. 아래 2.3~2.5는 참고 코드라 클래스 단위로 요약만 짚습니다. 자세한 구현은 완성 레포(GitHub)를 참고하세요.

2.3 회원 서비스 - JWT로 로그인하다

회원 서비스는 로그인과 사용자 조회를 담당합니다. 사용자가 `POST /login` 으로 아이디와 비밀번호를 보내면, 회원 서비스가 DB에서 조회하고 비밀번호를 검증합니다. 검증에 성공하면 JWT 토큰을 응답 바디에 담아 돌려줍니다. 이 토큰이 이후 모든 서비스 요청의 인증 수단이 됩니다.

HTTP 메서드	경로	기능
POST	/login	로그인 (JWT 발급)
GET	/api/users/{userId}	사용자 조회

2.3.1 클래스 구성

회원 서비스의 로그인과 사용자 조회를 다음 네 클래스가 처리합니다.

클래스	역할
User	사용자 정보를 담는 엔티티입니다.
UserRepository	사용자 데이터를 DB에서 조회·저장합니다.
UserService	로그인 시 비밀번호를 검증하고 JWT를 발급하며, 사용자 조회 요청을 처리합니다.
UserController	로그인과 사용자 조회 API를 제공합니다.

2.4 상품 서비스 - 재고를 관리하다

상품 서비스는 상품 목록 조회와 재고 증감을 담당합니다. 주문 서비스가 주문을 생성할 때 이 서비스의 재고 감소 API를 호출하고, 주문이 취소되거나 실패하면 재고 증가 API로 되돌립니다.

HTTP 메서드	경로	기능
GET	/api/products/{productId}	상품 조회
PUT	/api/products/{productId}/decrease	재고 감소
PUT	/api/products/{productId}/increase	재고 증가

2.4.1 클래스 구성

상품 조회와 재고 증감을 다음 네 클래스가 처리합니다.

클래스	역할
Product	상품 정보를 담는 엔티티이며, 재고 증감 로직을 자기 안에 둡니다.
ProductRepository	상품 데이터를 DB에서 조회·저장합니다.
ProductService	재고 감소 전 상품 존재·재고·가격을 검증한 뒤 재고를 줄이거나 늘립니다.
ProductController	상품 조회와 재고 증감 API를 제공합니다.

2.5 배달 서비스 - 배달을 생성하고 취소하다

배달 서비스는 배달 생성과 취소를 담당합니다. 이번 챕터에서는 배달 생성과 동시에 완료 처리합니다. PENDING 상태로 생성한 뒤 즉시 COMPLETED로 전이되며, 배달 취소 요청이 들어오면 CANCELLED로 바뀝니다.

HTTP 메서드	경로	기능
POST	/api/deliveries	배달 생성
GET	/api/deliveries/{deliveryId}	배달 조회
PUT	/api/deliveries/{orderId}	배달 취소

2.5.1 클래스 구성

배달 생성과 취소를 다음 네 클래스가 처리합니다.

클래스	역할
Delivery	배달 정보와 상태 전이(PENDING → COMPLETED / CANCELLED)를 담는 엔티티입니다.
DeliveryRepository	배달 데이터를 DB에서 조회·저장합니다.
DeliveryService	배달을 생성하고 취소 요청을 처리합니다.
DeliveryController	배달 생성·조회·취소 API를 제공합니다.

나머지는 제 일만 하면 되는데, 주문 하나는 왜 이렇게 걸리는 게 많지?

선배 : "이제 주문 서비스예요. 어디까지 진행됐는지 기록해 뒀다가, 실패하면 역순으로 취소해요."

2.6 주문 서비스 - 보상 트랜잭션의 현장

배달 서비스까지 살펴봤으니, 이제 주문 서비스로 넘어갑니다.

HTTP 메서드	경로	기능
POST	/api/orders	주문 생성
GET	/api/orders/{orderId}	주문 조회
PUT	/api/orders/{orderId}	주문 취소

2.6.1 클래스 구성

주문 생성과 보상 트랜잭션을 다음 여섯 클래스가 처리합니다.

클래스	역할
Order	주문 정보와 상태 전이(PENDING → COMPLETED / CANCELLED)를 담는 엔티티입니다.
OrderStatus	주문 상태(대기·완료·취소)를 정의하는 열거형입니다.
OrderRepository	주문 데이터를 DB에서 조회·저장합니다.
OrderRequest / OrderResponse	주문 API의 요청과 응답 형식을 정의합니다.
OrderController	주문 생성·조회·취소 API를 제공합니다.
OrderService	[작성] 주문 생성 시 보상 트랜잭션을 수행하며, 조회와 취소도 처리합니다.

주문 한 건에 상품 한 개를 담는 단순한 모델로 시작하여, 분산 트랜잭션 보상 흐름에 집중합니다.

2.6.2 보상 트랜잭션 흐름

엔티티가 준비되었으니, 이제 주문 서비스의 진짜 역할을 살펴봅니다. 주문 서비스는 상품 서비스와 배달 서비스를 **직접 호출**합니다. 직접 호출했으니 실패 시 되돌리는 보상도 직접 처리합니다. 이미 발송한 택배를 취소하려면 받는 사람에게 직접 "되돌려달라"고 요청해야 하는 것과 같습니다.

어느 단계에서 실패하면 무엇을 되돌려야 하는지 먼저 그려 보겠습니다.

단계	동작	실패 시 보상
1	재고 감소 (상품 서비스)	없음 (아직 진행 안 함)
2	배달 생성 (배달 서비스)	재고 복구 (1단계 되돌리기)
3	주문 완료	배달 취소 + 재고 복구 (2, 1단계 되돌리기)
4	성공 응답 반환	—

예를 들어 2단계 배달 생성에서 실패하면, 이미 줄어든 재고 한 단계만 되돌리면 됩니다. 3단계에서 실패하면 배달 취소와 재고 복구를 모두 되돌립니다. 진행한 만큼만 거꾸로 거슬러 올라가려면, 어디까지 갔는지를 코드에서 기록해 두어야 합니다.

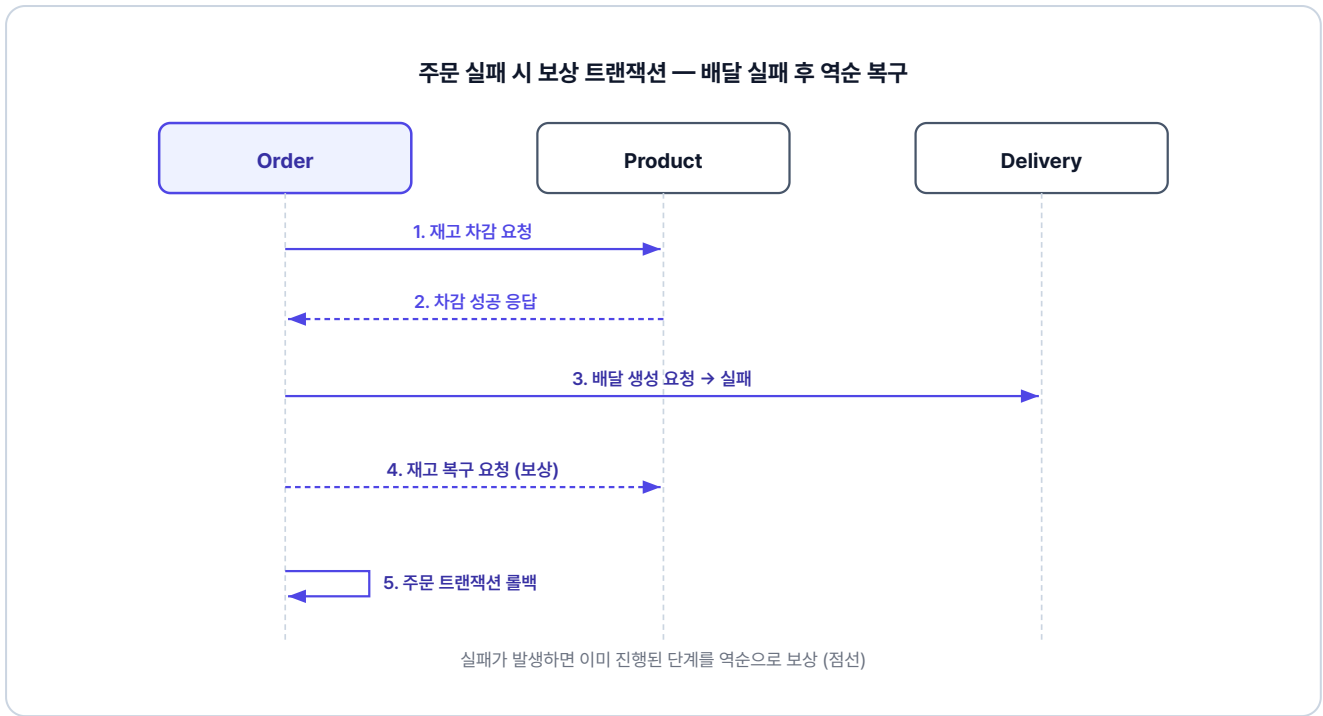


그림 2-2. 주문 실패 시 보상 트랜잭션 흐름

2.6.3 adapter - 외부 서비스 호출 설정

`adapter/` 폴더에는 외부 서비스를 호출하는 **클라이언트** 두 개가 있습니다. OrderService는 클라이언트의 메서드만 부르면 HTTP 디테일은 알 필요가 없습니다. 두 클라이언트 앞에 둔 RestClient 설정이 모든 호출에 JWT를 자동으로 실어 줍니다.

클래스	역할
RestClientConfig	RestClient에 인터셉터를 등록하여 외부 호출 시 JWT를 자동 전달합니다.
ProductClient	상품 서비스의 <code>decreaseQuantity</code> (재고 감소)와 <code>increaseQuantity</code> (재고 복구)를 호출합니다.
DeliveryClient	배달 서비스의 <code>createDelivery</code> (배달 생성)와 <code>cancelDelivery</code> (배달 취소)를 호출합니다.

세 클래스의 전체 코드는 완성 레포(GitHub)에서 확인할 수 있습니다. 챕터 2.6.4의 OrderService에서 이 메서드들이 어떻게 조합되는지가 핵심입니다.

재고를 줄였으면 다시 늘리고, 배달을 만들었으면 취소하고... 그 말이 이거였구나.

2.6.4 OrderService - 보상 트랜잭션의 핵심

이제 이번 챕터의 하이라이트입니다. 보상 트랜잭션 패턴을 직접 구현합니다. 코드를 읽을 때

`productDecreased` 와 `deliveryCreated` 두 플래그를 추적하면서 읽어보세요. 단계가 성공할 때마다 플래그를 `true` 로 올려두고, catch 블록에서는 켜져 있는 플래그만 골라 역순으로 되돌립니다.

`orders/OrderService.java` 를 열고 아래 메서드를 작성합니다.

실습 1 orders/OrderService.java. createOrder - 보상 트랜잭션 핵심

```
@Transactional
public OrderResponse createOrder(int userId, int productId, int quantity, Long price,
String address) {
    // 보상트랜잭션을 위한 변수 선언
    boolean productDecreased = false;
    boolean deliveryCreated = false;

    // 보상트랜잭션에서 id를 전달해야해서 상위로 빼둬
    Order createdOrder = null;

    try {
        // 1. 주문 생성
        createdOrder = orderRepository.save(Order.create(userId, productId, quantity,
price));

        // 2. 상품 재고 차감
        productClient.decreaseQuantity(new ProductRequest(productId, quantity, price));
        productDecreased = true;

        // 3. 배달 생성 (어댑터)
        deliveryClient.createDelivery(new DeliveryRequest(createdOrder.getId(), address));
        deliveryCreated = true;

        // 4. 주문 완료
        createdOrder.complete();
        return OrderResponse.from(createdOrder);
    } catch (Exception e) {
        // 배달 취소
        if (deliveryCreated) {
            deliveryClient.cancelDelivery(createdOrder.getId());
        }

        // 재고 복구
        if (productDecreased) {
            productClient.increaseQuantity(new ProductRequest(productId, quantity, price));
        }
        throw new Exception500("주문 생성 중 오류가 발생했습니다: " + e.getMessage());
    }
}
```

참고로 catch 마지막의 `throw` 로 `@Transactional` 이 주문 트랜잭션까지 함께 롤백합니다. 외부 서비스는 보상 호출, 주문 데이터는 트랜잭션 롤백으로 정리됩니다.

2.7 Docker Compose - 네 개의 서비스를 한 번에 실행하다

코드가 완성되었습니다. 이제 직접 실행하여 보상 트랜잭션이 작동하는 것을 눈으로 확인해 봅니다.

시나리오를 따라가기 전에, 각 서비스에 어떤 데이터가 미리 등록되어 있는지 살펴봅니다. 컨테이너를 띄우면 H2 in-memory DB에 다음 데이터가 자동으로 삽입되어 별도 설정 없이 곧장 사용할 수 있고, 컨테이너를 재시작하면 매번 같은 상태로 초기화됩니다. 데이터 정의는 각 서비스의 `db/data.sql`에 있습니다.

서비스	더미 데이터
회원	계정 3개: <code>ssar</code> , <code>cos</code> , <code>love</code> (비밀번호는 모두 <code>1234</code>)
상품	MacBook Pro(재고 10), iPhone 15(재고 0 품절), AirPods(재고 10)
배달	주문 3건에 대응하는 배달 데이터, 모두 <code>COMPLETED</code> 상태
주문	사용자별 주문 3건(완료·취소·대기)

2.7.1 서비스 실행

각 서비스의 Dockerfile은 동일한 구조로, 스프링 프로젝트를 컨테이너 안에서 빌드하고 실행하는 4단계 흐름을 따릅니다.

단계	무엇을	왜
베이스	JDK 21 이미지	Spring Boot 실행 환경을 마련하기 위함입니다.
소스 복사	프로젝트 → /app	컨테이너 안에서 빌드하기 위해 소스를 가져옵니다.
JAR 빌드	<code>gradlew bootJar</code>	실행 가능한 단일 JAR 파일을 생성합니다.
실행	<code>java -jar</code>	컨테이너가 시작될 때 JAR를 자동 실행합니다.

ex01 디렉토리의 `docker-compose.yml` 이 4개 서비스를 한 번에 묶어 실행합니다. 네 서비스 모두 같은 패턴이고, 빌드 컨텍스트와 포트만 다릅니다. H2 접속 정보는 네 서비스에 환경변수로 동일하게 주입됩니다.

서비스	build.context	호스트:컨테이너 포트	네트워크
order	<code>./order</code>	<code>8081:8081</code>	msa-network
user	<code>./user</code>	<code>8083:8083</code>	msa-network
product	<code>./product</code>	<code>8082:8082</code>	msa-network
delivery	<code>./delivery</code>	<code>8084:8084</code>	msa-network

`msa-network` 로 묶여 있기 때문에, 컨테이너끼리는 서비스 이름(예: `http://product-service:8082`)으로 통신합니다.

프로젝트가 위치한 폴더로 이동 후, 터미널에서 Docker Compose로 4개 서비스를 한 번에 빌드하고 실행합니다.

터미널 Docker Compose 실행

```
cd ex01
docker compose up
```

실행이 완료되면 각 서비스에 접근할 수 있습니다.

서비스	주소
주문 서비스	<code>http://localhost:8081</code>
상품 서비스	<code>http://localhost:8082</code>
회원 서비스	<code>http://localhost:8083</code>
배달 서비스	<code>http://localhost:8084</code>

처음 실행 시 이미지 빌드에 5~10분이 소요될 수 있습니다. 터미널이 멈춘 것처럼 보여도 정상이니 기다려주세요. 빌드 진행 상황은 `docker compose logs -f [서비스명]` 으로 확인할 수 있습니다.

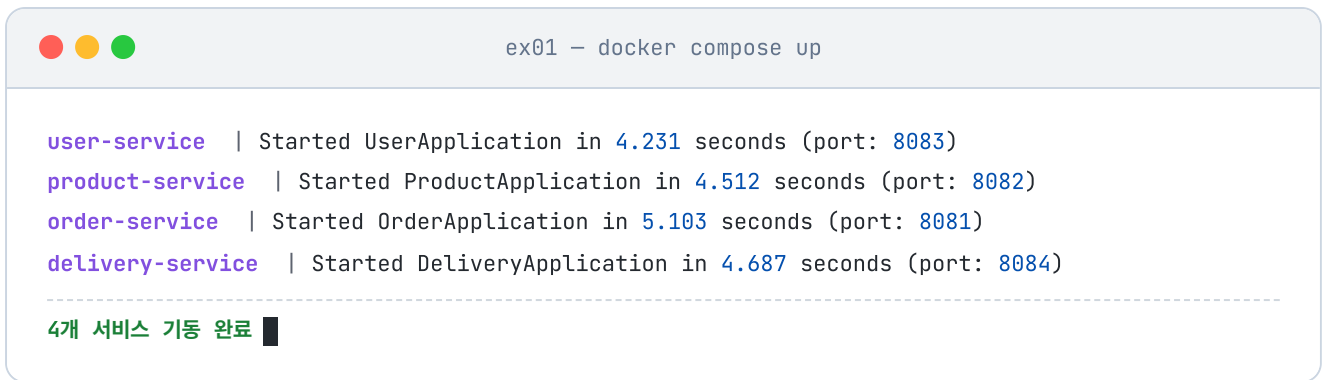


그림 2-3. Docker Compose 실행 결과

2.7.2 Hoppscotch와 인터셉터 설정

서비스가 실행되면 API를 호출하여 결과를 확인해야 합니다. 이 책에서는 Hoppscotch(<https://hoppscotch.io/>)를 사용합니다. localhost로 요청을 보내려면 Chrome 웹 스토어에서 **Hoppscotch Browser Extension**을 설치합니다. 설치 후 Hoppscotch 화면 하단의 인터셉터 설정에서 **"Browser Extension"**을 선택하세요.

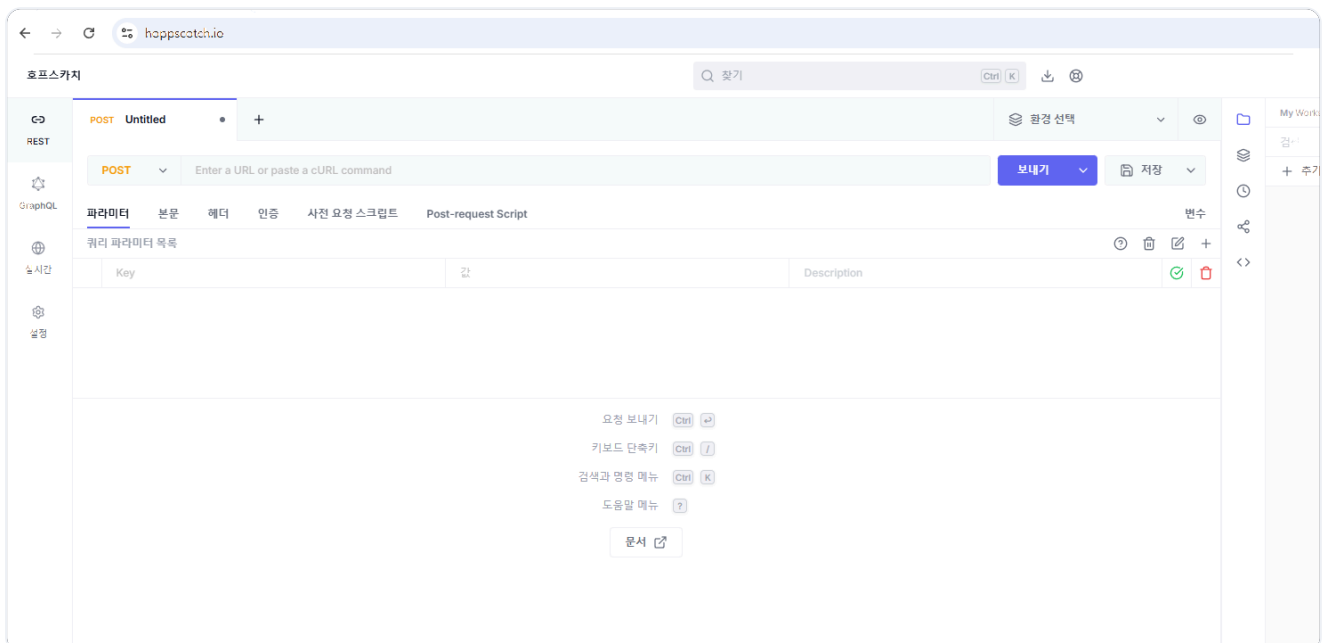


그림 2-4. Hoppscotch 화면

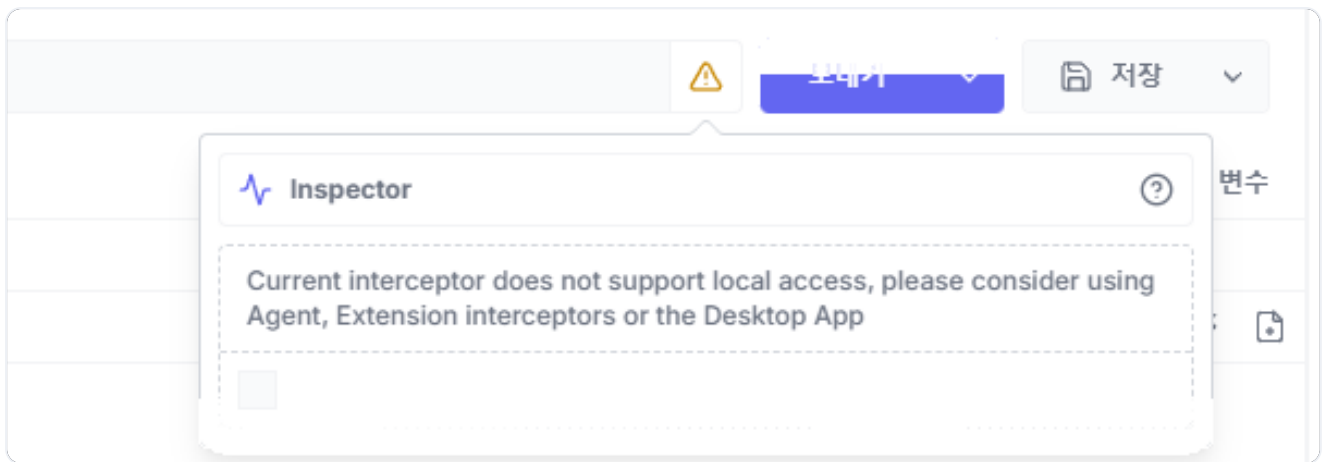


그림 2-5. Browser Extension 인터셉터 설정

2.7.3 시나리오 1: 정상 주문

먼저 로그인하여 JWT 토큰을 받습니다. 이때 콘텐츠 종류(Content-Type)를 `application/json`으로 설정해야 합니다. 이 헤더는 서버에게 "내가 보내는 데이터는 JSON 형식이다"라고 알리는 역할을 합니다.

POST http://localhost:8083/login

```
{
  "username": "ssar",
  "password": "1234"
}
```

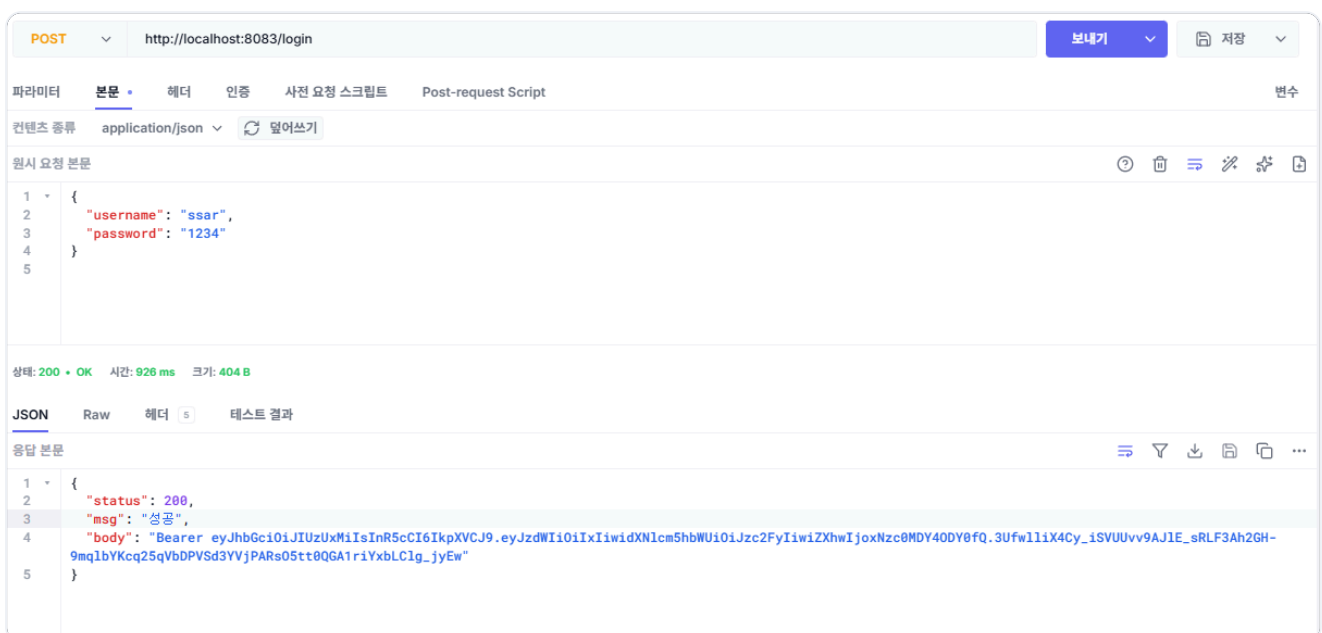


그림 2-6. 로그인 API 호출 결과

응답 바디 데이터에 포함된 JWT 토큰을 확인할 수 있습니다.

받은 토큰을 Hoppscotch의 인증 > 인증 유형(Bearer) 항목의 토큰 필드에 넣습니다.

POST

Untitled

GET

Untitled

+

GET

▼

Enter a URL or paste a cURL command

파라미터

본문

헤더

인증

사전 요청 스크립트

Post-request Script

인증 유형

Bearer ▼

토큰

Bearer eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxliwidXNlcm5hbWUiOiJzc2FyYiwZxhwjoxNzc0MDY4ODV9

요청을 보낼 때 인증
사용법 알아보기

그림 2-7. Bearer 토큰 설정

MacBook Pro(상품 ID 1, 재고 10개)를 1개 주문합니다. 요청 바디는 상품 정보(`productId`, `quantity`, `price`)와 배달 주소(`address`)를 한 번에 담습니다.

POST http://localhost:8081/api/orders

```
{
  "productId": 1,
  "quantity": 1,
  "price": 2500000,
  "address": "Addr 4"
}
```

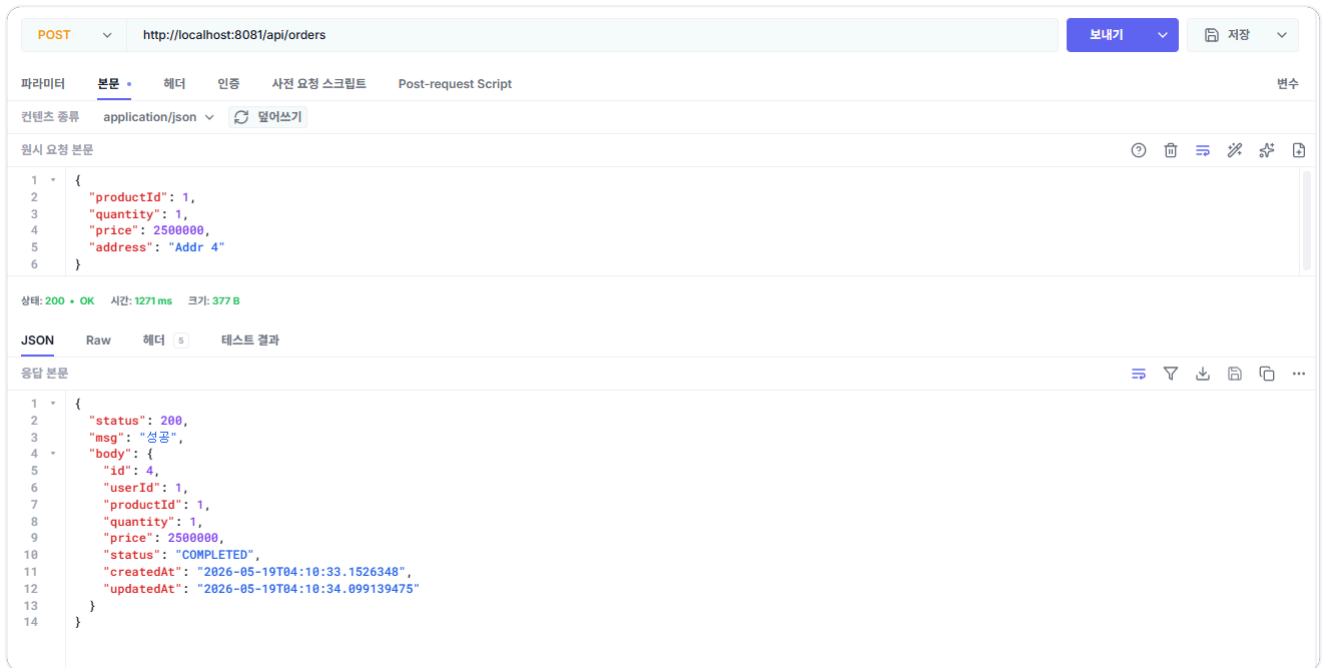


그림 2-8. 주문 생성 API 호출 결과

주문이 성공하면 상품 서비스에서 재고가 10 → 9로 줄어들고, 배달 서비스에 배달이 생성됩니다.

GET http://localhost:8082/api/products/1

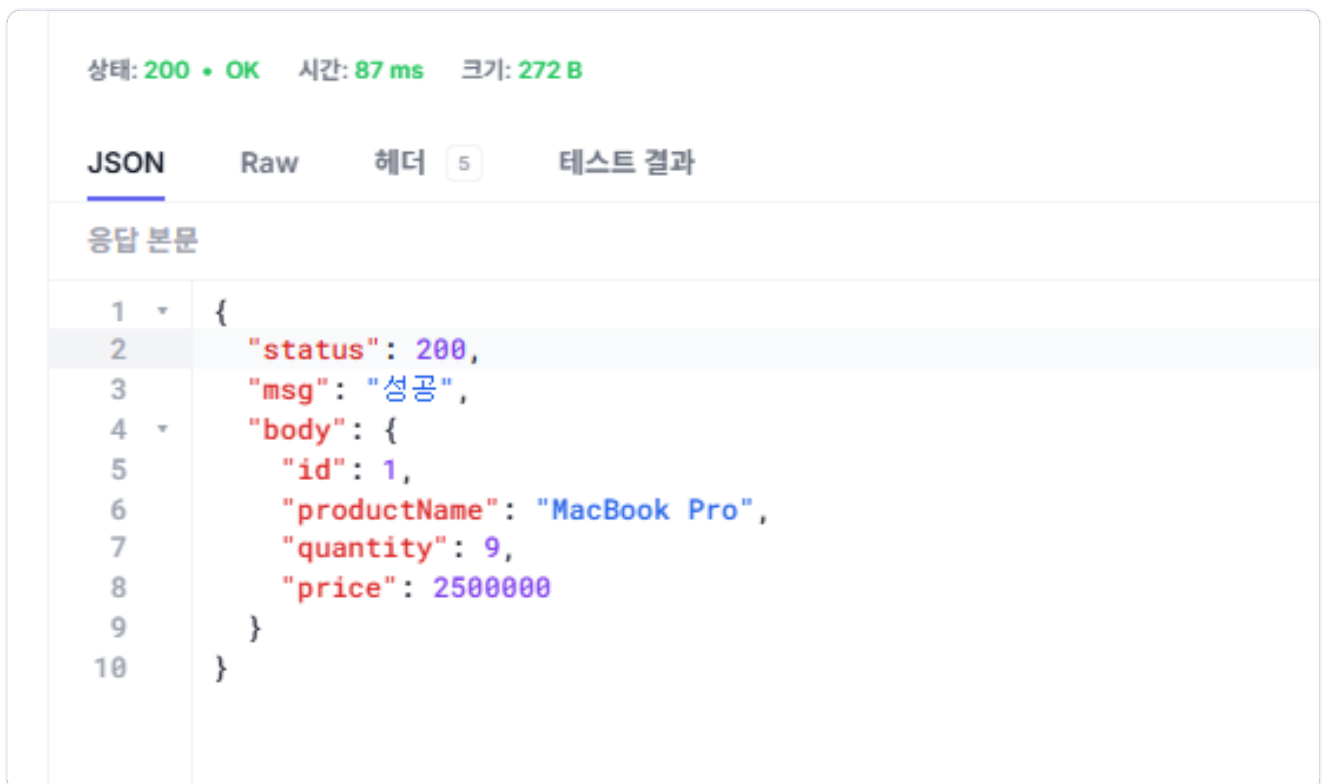


그림 2-9. 재고 감소 확인

GET <http://localhost:8084/api/deliveries/4> # 더미 배달 3건 다음이라 새 배달은 4번

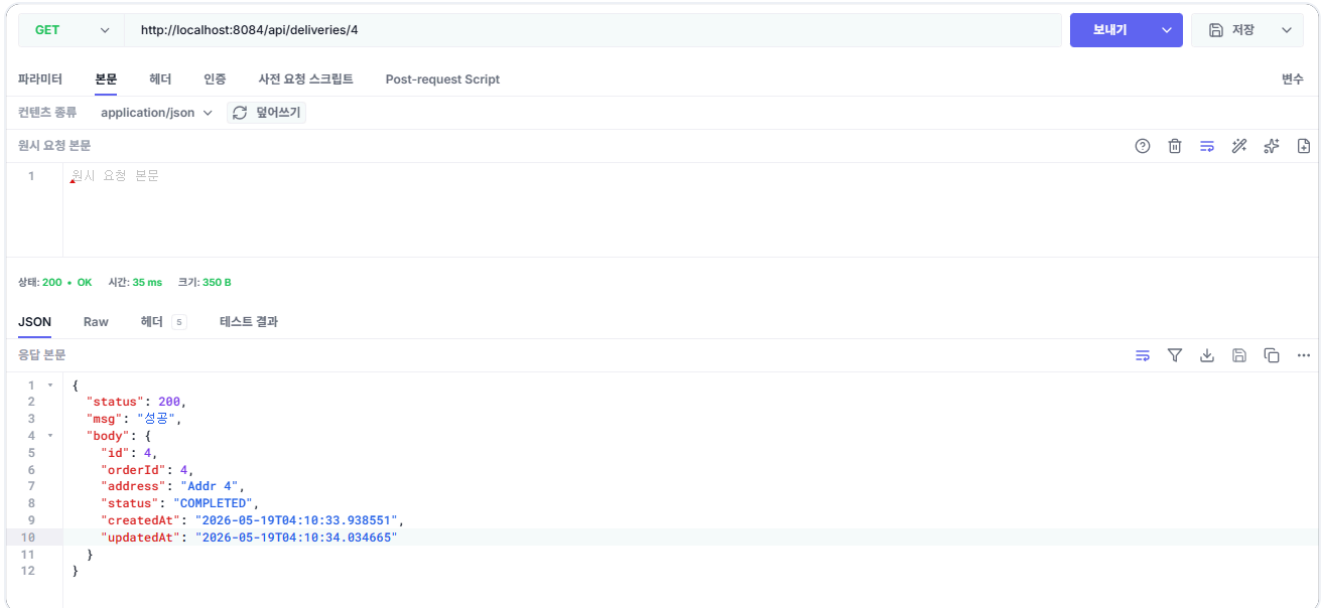


그림 2-10. 배달 생성 확인

2.7.4 시나리오 2: 재고 부족

품질 상품인 iPhone 15(상품 ID 2, 재고 0)를 주문해 보겠습니다. 첫 번째 단계(재고 차감)에서 바로 실패하므로 보상할 작업이 없어 즉시 에러가 반환됩니다.

POST <http://localhost:8081/api/orders>

```

{
  "productId": 2,
  "quantity": 1,
  "price": 1300000,
  "address": "Addr 4"
}

```



그림 2-11. 재고 부족 시 에러 응답

오픈이 : "품질은 첫 단계에서 막혀서 보상이 안 도네요. 그러면 보상이 진짜 도는 건 언제죠?"

선배 : "주소를 비워서 보내 봐요. 재고는 줄어든 다음 배달에서 실패할 거예요."

2.7.5 시나리오 3: 주소 누락

이번에는 주소를 빈 문자열로 보내 보겠습니다. 주문 자체는 재고 차감까지 진행되지만, 배달 서비스에서 주소가 없으므로 실패합니다. 이때 보상 트랜잭션이 작동하여 차감된 재고가 복구되고, 주문 데이터는 트랜잭션 롤백으로 처음부터 없던 일처럼 DB에서 사라집니다.

POST `http://localhost:8081/api/orders`

```

{
  "productId": 1,
  "quantity": 1,
  "price": 2500000,
  "address": ""
}
    
```

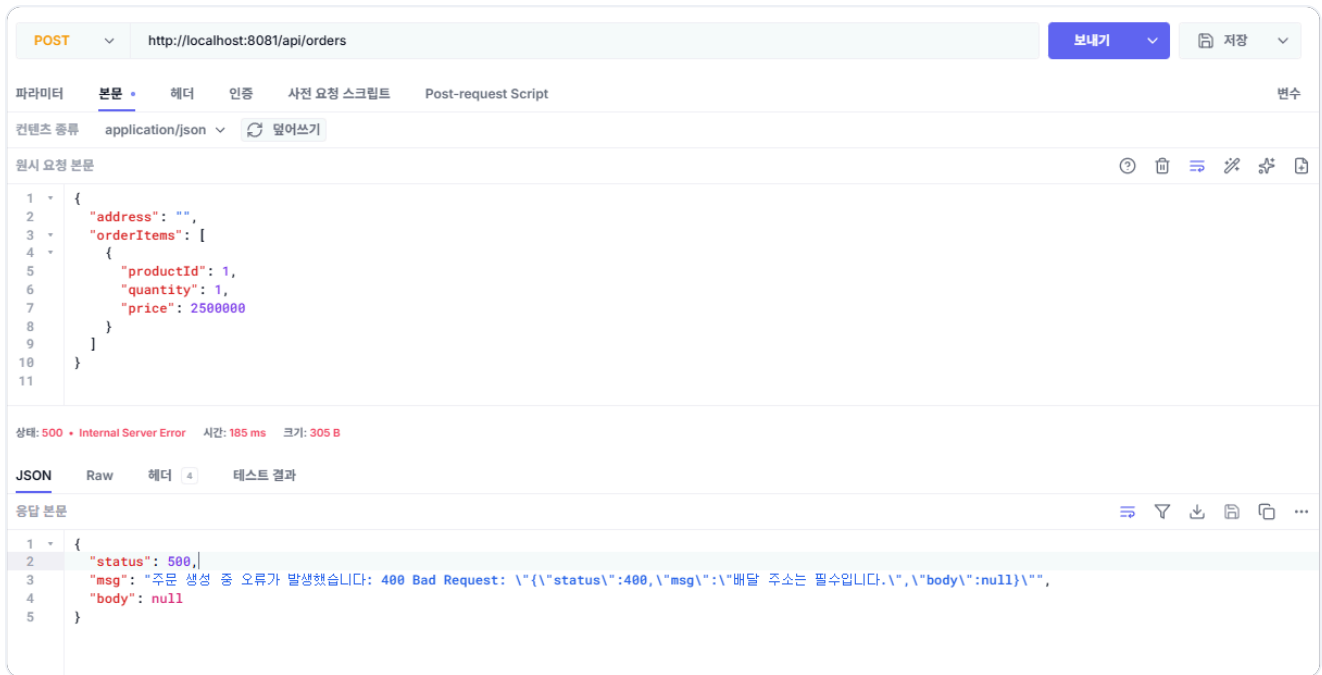


그림 2-12. 주소 누락 시 에러 응답

그리고 재고가 원복되었는지 확인합니다.

GET http://localhost:8082/api/products/1

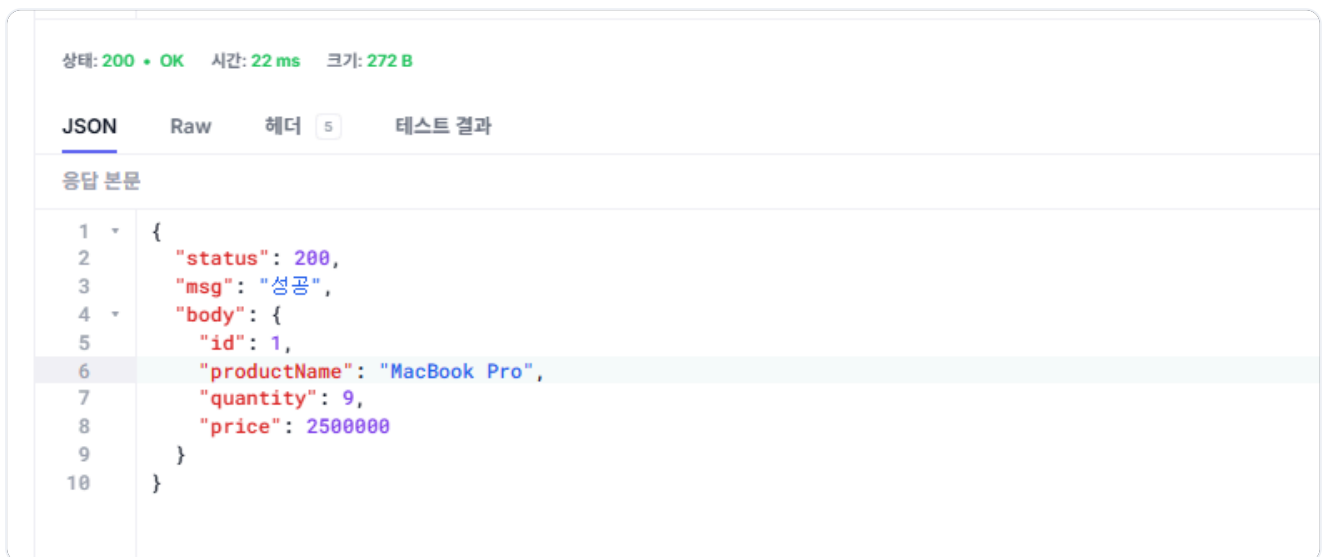


그림 2-13. 재고 원복 확인

테스트가 끝났으면 실행 중인 컨테이너를 정리합니다.

터미널 컨테이너 정리

```
docker compose down
```

이 명령어를 실행하면 docker compose up으로 띄운 모든 컨테이너가 종료되고 제거됩니다.

보상 트랜잭션이 진짜 도는구나. 재고가 자동으로 돌아왔다.

선배 : "잘 됐네요. 근데 한 가지 더 생각해 봐요. 상품 서비스가 느려지면 주문 서비스는 어떻게 될까요?"

...아. 전화를 걸고 기다리니까, 상대가 느리면 나도 느려지는 거잖아.

보상 트랜잭션 덕분에 데이터 일관성을 유지할 수 있었습니다. 하지만 이 구조에는 눈에 잘 보이지 않는 문제가 몇 가지 있습니다. 모든 서비스 호출이 동기적이라 상품 서비스가 1초 걸리면 주문 서비스도 1초를 기다립니다. 보상 트랜잭션 코드도 비즈니스 로직과 섞이면서 try-catch 중첩이 깊어집니다.

실패한 주문은 트랜잭션 롤백으로 데이터 자체가 사라지므로 "어떤 주문이 왜 실패했는지" 이력도 남지 않습니다. 게다가 비즈니스 로직이 서비스 코드에 섞여 있어, 손볼 때마다 어디를 고쳐야 할지 막막한 코드 구조도 같이 따라옵니다.

이것만은 기억하자

- MSA에서 분산 트랜잭션은 자동으로 풀리지 않습니다. 실패를 감지한 쪽이 직접 보상 호출을 보내야 합니다.
- 4개 서비스를 REST로 연결하고, JWT 인증 헤더를 RestClient 인터셉터로 자동 전달했습니다.
- 플러그 기반 보상 트랜잭션으로 외부 서비스(재고와 배달)를 원상복구했습니다.
- 단 주문 데이터 자체는 트랜잭션 롤백으로 사라지므로 실패 이력은 남지 않습니다.
- 이 구조의 한계는 다음 챕터 운영 환경에서, 그다음 챕터 비동기 전환에서 차례로 해소됩니다.

다음 챕터에서는 비즈니스 로직이 서비스 코드에 흩어져 있는 문제를 마주칩니다. 도메인이 자기 일을 모르고 Service가 다 답하고 있다는 통증을 인식하고, **DDD + 클린 아키텍처**로 코드 구조를 정리한 뒤 운영 환경(K8s)에 올립니다.

챕터 3. 도메인을 중심으로 - DDD + 클린 아키텍처와 Kubernetes

이 챕터의 전체 소스코드는 <https://github.com/metacoding-12-msa/ex02> 에서 확인할 수 있습니다.

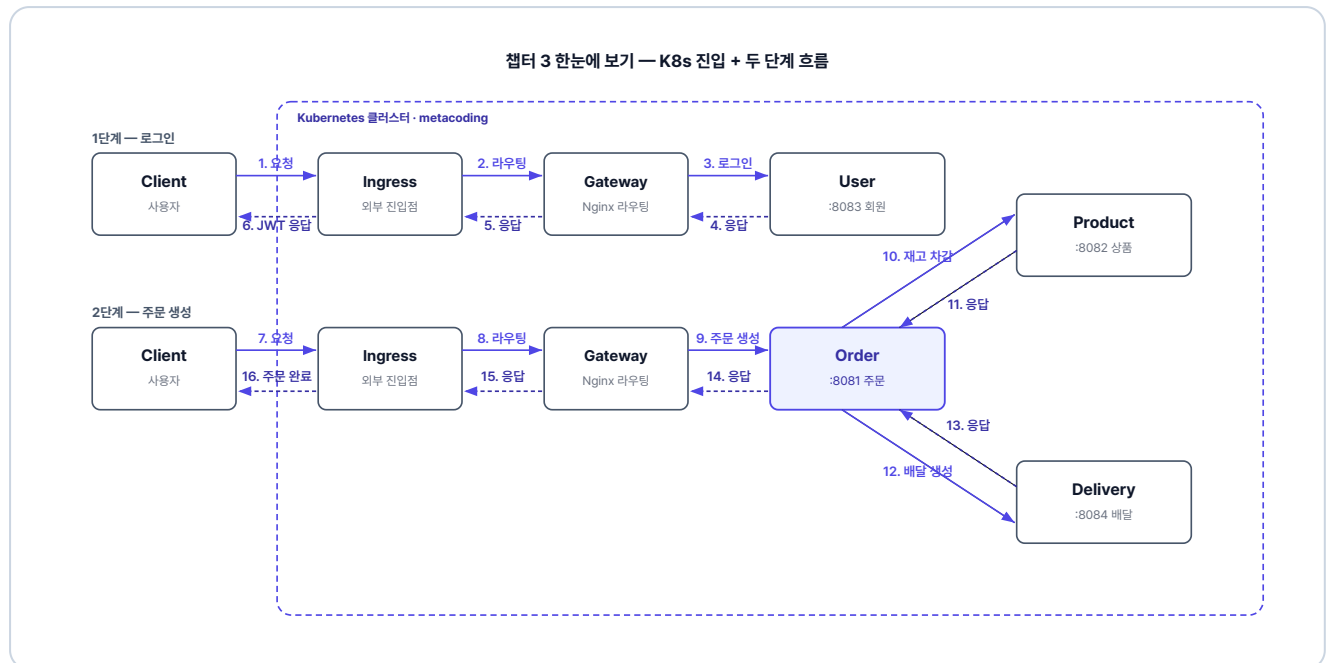


그림 3-1. 챕터 3 한눈에 보기 - K8s 진입과 두 단계 흐름

이번 챕터가 끝나면

- 비즈니스 규칙을 도메인 객체에 캡슐화하는 **DDD(도메인 주도 개발)** 를 적용할 수 있습니다.
- 컨트롤러가 `OrderService` 코드 대신 UseCase 인터페이스에 의존하도록 **클린 아키텍처**로 구조를 정리할 수 있습니다.
- Nginx API Gateway로 4개 서비스 진입점을 하나로 통합할 수 있습니다.
- Kubernetes에 매니페스트로 배포하고 ConfigMap·Secret으로 환경 변수를 주입할 수 있습니다.

준비하기. 실습 시작 전 한 번만 설정

1. 소스 코드 클론

터미널 레포 클론

```
git clone https://github.com/metacoding-12-msa/ex02.git
cd ex02
```

2. 파일 구조

ex02 디렉토리

```
ex02/
├─ order/           # 포트 8081
├─ product/         # 포트 8082
├─ user/            # 포트 8083
├─ delivery/        # 포트 8084
├─ gateway/         # Nginx API Gateway
├─ db/              # MySQL Dockerfile
└─ k8s/             # Kubernetes 매니페스트
```

주문 서비스 패키지 구조 (챕터 3에서 재구성)

```
src/main/java/com/metacoding/order/
├─ domain/          # 엔티티 + 비즈니스 규칙
├─ repository/      # Spring Data JPA
├─ usecase/         # UseCase 인터페이스 + 서비스 코드
├─ web/             # 컨트롤러 + DTO
├─ adapter/         # 외부 서비스 클라이언트 (order 전용)
└─ core/            # JWT, 예외처리 (챕터 2와 동일)
src/main/resources/
└─ application.properties # DB·JWT 설정 (값은 환경변수로 주입)
```

user/product/delivery도 동일한 구조이며, adapter/ 패키지만 order 전용입니다.

3. 실습 환경

도구	용도	비고
Docker Desktop	컨테이너 런타임	챕터 2에서 설치한 그대로. 실행 중이어야 함
Minikube	로컬 Kubernetes 클러스터	https://minikube.sigs.k8s.io/

Docker Desktop이 "Engine running" 상태인지 확인하고, Minikube가 설치되어 있지 않다면 위 주소에서 설치합니다.

4. 실습 순서

1. 챗터 2 코드를 DDD + 클린 아키텍처로 재구성한 결과 살펴보기
2. Nginx API Gateway 살펴보기
3. K8s 매니페스트(ConfigMap·Secret·Deployment·Service·Ingress) 5종 살펴보기
4. Minikube에서 빌드·배포·실행

이번 챗터는 직접 코드를 작성하지 않습니다. 챗터 2 프로젝트를 DDD + 클린 아키텍처로 재구성한 결과를 살펴보고, Kubernetes 배포를 실습합니다.

팀장이 코드 정리를 부탁했습니다.

팀장 : "취소되는지 검사하는 코드가 서비스마다 흩어져 있어요. 손볼 때마다 어디를 고쳐야 할지 헷갈리니까, 한곳으로 모아 주세요."

오픈이가 `OrderService`의 주문 취소 메서드를 열었습니다. "이미 취소된 주문인가?"를 막는 `if`가 거기 있었습니다. 그런데 코드를 더 훑어보니 같은 종류의 검증이 서비스마다 박혀 있습니다. 재고가 충분한지는 `ProductService`의 `if`가, 배달 주소가 비어 있는지는 `DeliveryService`의 `if`가 따로 판단합니다. `Order`는 데이터만 담고 있을 뿐, 취소 가능 여부 판단은 모두 바깥 서비스에 흩어져 있습니다.

함께 수정 작업을 하던 동료와 와서 코드를 들여다봤습니다.

동료 : "근데 이상하지 않아요? `Order` 안에 데이터는 다 있는데, 취소 여부는 `OrderService`가 결정하잖아요. 서비스는 그냥 흐름만 처리하고, 검증은 도메인에서 하는 게 낫지 않을까요?"

그 말을 들은 오픈이는 선배 자리로 가 물었습니다.

선배 : "맞아요. 비즈니스 로직은 도메인에 두고, 서비스는 흐름만 처리하는 게 좋아요. 그 방식이 **DDD(도메인 주도 개발)**예요. 이렇게 하면 흩어져 있던 검증이 도메인 한곳에 모여서, 다음에 손볼 때도 도메인만 보면 됩니다."

DDD(도메인 주도 개발)란? 비즈니스 규칙(검증·상태 변경 같은 핵심 로직)을 도메인 객체 안에 두는 설계 방식입니다. 데이터와 규칙을 같은 객체에 모아, 서비스 계층은 흐름 조율에만 집중하게 합니다.

3.1 도메인 주도 개발(Domain-Driven Design) - 비즈니스 로직을 도메인으로

가게 운영 규칙이 사장 한 명의 머릿속에만 있다고 해보겠습니다. 환불 가능 시간, 결제 방식, 재고 처리까지 전부 사장이 외우고 있습니다. 그래서 누가 손님을 응대하든 사장이 대답을 해야 합니다.

이 문제는 가게 운영 매뉴얼을 만들면 해결됩니다. 규칙은 매뉴얼에 정리해 두고, 사장은 손님 응대 흐름만 진행하면서 매뉴얼에 적힌 대로 따릅니다. 누가 응대하든 매뉴얼만 보면 같은 판단을 할 수 있습니다. 새 규칙이 들어와도 매뉴얼 한 곳만 업데이트하면 끝입니다.



그림 3-2. 머릿속 규칙에서 운영 매뉴얼로

`OrderService`가 비즈니스를 수행하는 '사장님'이라면, `Order` 도메인 객체는 그 안에 담긴 핵심 규칙인 '운영 매뉴얼'입니다. 비즈니스 로직을 서비스에서 분리해 도메인에 응집시키면 기술적 환경이 변해도 비즈니스의 본질은 흔들리지 않으며, 복잡한 요구사항 속에서도 코드의 가독성과 유지보수성을 지킬 수 있습니다.

3.2 UseCase 인터페이스(Use Case Interface) - USB 허브처럼 갈아끼우기

`Order`가 규칙을 갖게 됐어도, 컨트롤러가 `OrderService` 코드를 직접 참조하면 그 코드가 바뀔 때마다 컨트롤러도 바뀌어야 하고, 테스트 때 가짜 `OrderService`로 갈아끼우기도 어렵습니다.

USB 허브를 떠올려 보세요. USB 규격만 같으면 마우스든 키보드든 외장하드든 같은 허브에 꽂아 쓸 수 있고, 필요에 따라 갈아끼웁니다. 컴퓨터는 어떤 장치가 꽂혔는지 모르고, USB 규격대로 통신만 합니다.

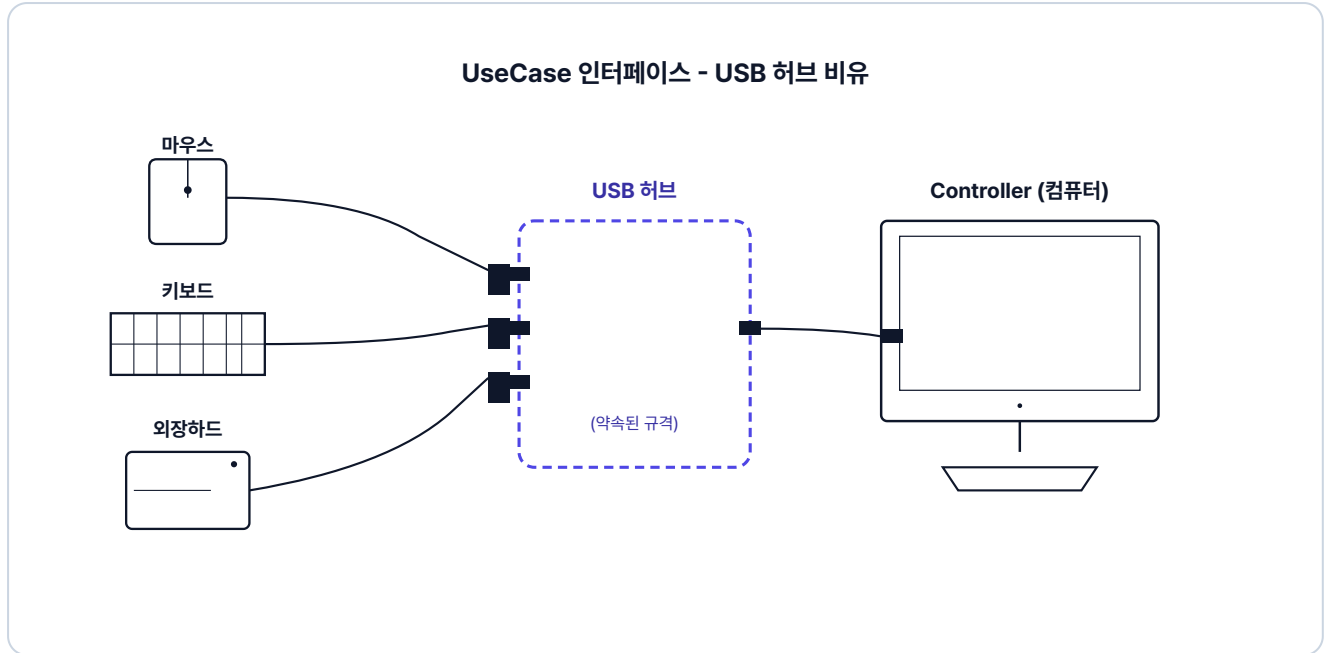


그림 3-3. UseCase 인터페이스 - USB 허브 비유

UseCase 인터페이스가 컨트롤러와 약속한 'USB 규격'이라면, `OrderService`는 그 규격에 꽂히는 'USB 장치'입니다. 규격만 지키면 장치를 자유롭게 갈아끼울 수 있고, 컴퓨터(컨트롤러)는 어떤 장치가 꽂혔는지 알 필요가 없습니다.

UseCase 인터페이스란? 시스템이 수행할 비즈니스 행위(주문 생성·조회·취소 같은)를 메서드로 약속한 인터페이스입니다. 컨트롤러는 이 약속만 보고 호출하므로, 뒤에 어떤 코드가 꽂혀도 동작합니다.

컨트롤러도 **UseCase 인터페이스**(USB 규격)만 보고, 뒤에 어떤 `OrderService` 코드가 꽂혔는지는 모르게 만듭니다. 약속에만 의존하는 이 방식이 **클린 아키텍처(Clean Architecture)**의 의존성 역전입니다.

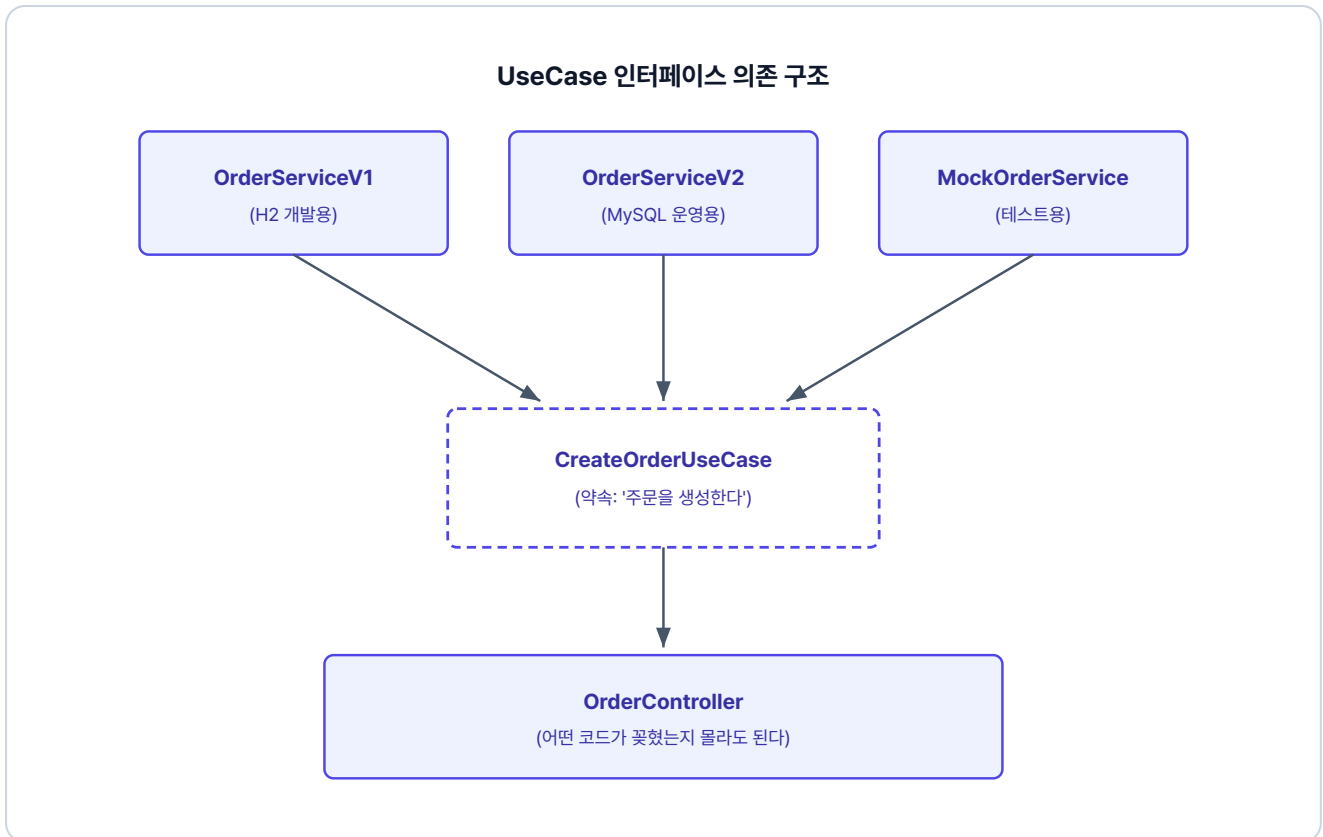


그림 3-4. UseCase 인터페이스 의존 구조

"무엇을 할 것인가"(UseCase 인터페이스)와 "어떻게 할 것인가"(Service 코드)를 분리하는 것이 핵심입니다.

이 책에서는 클린 아키텍처의 핵심인 UseCase 인터페이스를 통한 의존성 역전에 집중합니다. 완전한 아키텍처보다는 실습에 필요한 개념만 적용합니다.

3.3 UseCase 인터페이스 + 도메인 캡슐화 도입

UseCase 인터페이스가 왜 필요한지 이해했으니, 이제 실제 코드에 적용해보겠습니다. 패키지 구조부터 손보겠습니다. 챕터 2의 단순 레이어드 구조에서 `domain` 패키지를 안쪽으로 옮깁니다.

3.3.1 UseCase 인터페이스 정의

주문·상품·회원·배달 서비스의 각 기능을 인터페이스 형태로 UseCase로 정의합니다. 인터페이스 하나가 하나의 행위(Use Case)를 담습니다.

usecase/CreateOrderUseCase.java

```
public interface CreateOrderUseCase {
    OrderResponse createOrder(int userId, int productId, int quantity, Long price, String address);
}
```

3.3.2 엔티티의 비즈니스 로직 — DDD의 핵심

"주문이 취소 가능한가?" 같은 비즈니스 규칙은 서비스가 아닌 엔티티에 둡니다. 엔티티 메서드로 캡슐화하면 어디서 호출하던 동일한 규칙이 적용되고, 새 규칙이 들어와도 도메인 메서드만 손대면 됩니다.

domain/Order.java. validateCancelable 추가

```
public class Order {
    // 챗터 2 Order.java 참조 - 필드.create().complete() 동일, cancel()은 validateCancelable() 호출 추가

    // 비즈니스 규칙을 엔티티에 위임 (챗터 3에서 추가)
    public void validateCancelable() {
        if (this.status == OrderStatus.CANCELLED) {
            throw new Exception400("주문이 이미 취소되었습니다.");
        }
    }
}
```

3.3.3 OrderService - 인터페이스 구현

OrderService는 세 UseCase 인터페이스를 구현하고, 내부에서 도메인 객체의 비즈니스 메서드를 호출합니다. 이 구조 덕분에 책임이 분리됩니다. **서비스는 흐름 조율에만 집중하고, 실제 비즈니스 규칙은 도메인이 담당합니다.**

챗터 2와 비교해서 달라진 점은 다음과 같습니다.

1. **UseCase 인터페이스 구현.** 서비스가 직접 메서드를 노출하지 않고, `CreateOrderUseCase` 등 인터페이스를 구현합니다.
2. **비즈니스 규칙을 엔티티에 위임.** 챗터 2에서 서비스의 `if` 문으로 처리하던 검증을 도메인 객체의 메서드로 이동합니다.

usecase/OrderService.java. UseCase 인터페이스 구현

```
@RequiredArgsConstructor
@Service
@Transactional(readOnly = true) // 1. 클래스 레벨 읽기 전용 트랜잭션
public class OrderService implements CreateOrderUseCase, GetOrderUseCase, CancelOrderUseCase {
    // 2. UseCase 인터페이스를 구현

    @Override
    @Transactional // 쓰기 메서드만 오버라이드
    public OrderResponse cancelOrder(int orderId) {
        // ...
        findOrder.cancel(); // 3. cancel() 내부에서 검증 후 취소
        // ...
    }
}
```

3.3.4 OrderController 수정

`OrderService` 코드가 아닌 인터페이스를 주입받도록 컨트롤러를 수정합니다. 앞으로 `OrderService`를 다른 코드로 바꿔도 이 컨트롤러는 전혀 수정하지 않아도 됩니다.

web/OrderController.java. UseCase 인터페이스 주입

```
@RestController
@RequestMapping("/api/orders")
@RequiredArgsConstructor
public class OrderController {
    private final CreateOrderUseCase createOrderUseCase; // 구현체가 아닌 인터페이스 주입
    private final GetOrderUseCase getOrderUseCase;
    private final CancelOrderUseCase cancelOrderUseCase;

    @PostMapping
    public ResponseEntity<?> createOrder(...) {
        return Resp.ok(createOrderUseCase.createOrder(...)); // 인터페이스 메서드 호출
    }

    // GET /{orderId} - 주문 조회
    // PUT /{orderId} - 주문 취소
}
```

order-service와 동일한 패턴으로 product-user-delivery 서비스도 UseCase 인터페이스를 도입하고, 검증 로직은 각 엔티티 메서드로 모읍니다.

코드 정리는 마쳤지만, 막상 운영에 올리려니 새 문제가 보입니다.

오픈이 : "이제 운영에 올려야겠죠? 그런데 네 서비스 포트가 흩어져 있어요. :8081 · :8082 · :8083 · :8084 . 사용자가 다 알아야 하나요?"

선배 : "내부를 수정했으니, 외부에서 들어오는 단일 진입점도 만들면 좋겠네요."

3.4 Docker - Gateway와 MySQL 인프라 이미지

3.4.1 Nginx - API Gateway 라우팅

서비스가 늘어날수록 클라이언트가 모든 포트를 알아야 하고, 서비스 주소가 바뀌면 클라이언트도 따라 바뀌어야 합니다. **API Gateway**를 앞에 두면 클라이언트는 하나의 진입점으로만 요청하고, 게이트웨이가 URL 경로를 보고 알맞은 서비스로 넘깁니다.

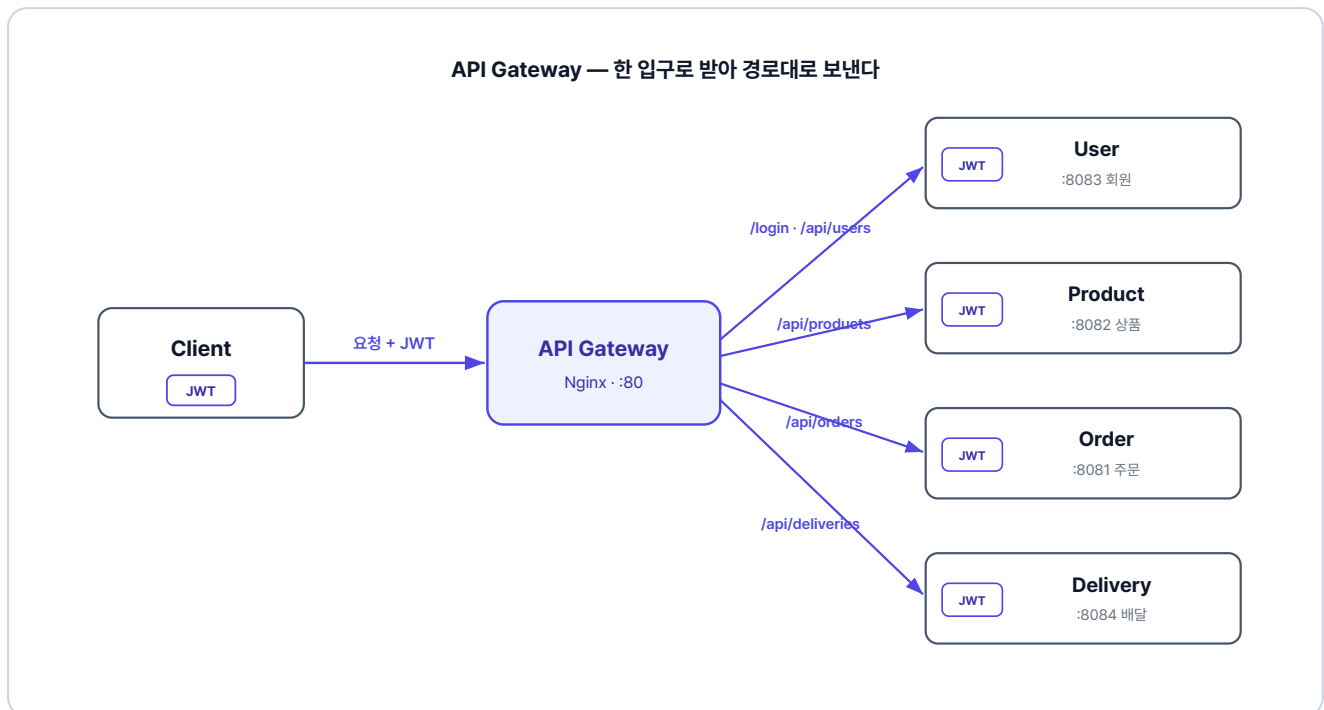


그림 3-5. 한 입구로 받아 경로대로 전달하고, 토큰 검증은 각 서비스가 합니다

`gateway/` 디렉토리에는 두 파일이 있습니다.

파일	역할
Dockerfile	Nginx 베이스 이미지에 설정 파일을 넣어 게이트웨이 컨테이너를 만듭니다.
nginx.conf	URL 경로별로 어느 서비스에 요청을 보낼지 정의합니다.

`nginx.conf`의 핵심은 네 서비스를 이름으로 등록하고, 들어온 URL을 해당 서비스로 보내는 것입니다. Kubernetes가 서비스 이름을 내부 DNS로 자동 해석하므로 IP 주소를 직접 지정하지 않아도 됩니다. 전체 설정 파일은 레포의 `gateway/nginx.conf`를 참고하세요.

게이트웨이에서 토큰을 검증하는 방식

지금 이 책에서는 게이트웨이(Nginx)가 요청을 경로대로 넘기지만 하고, JWT는 각 서비스가 직접 검증합니다. 다른 방법으로, 게이트웨이가 입구에서 토큰을 먼저 검증하고 통과한 요청만 서비스로 보내는 구조도 널리 쓰입니다. 이때 게이트웨이는 토큰에서 꺼낸 사용자 정보를 헤더에 실어 전달하고, 뒤 서비스는 게이트웨이를 지나온 요청을 이미 인증된 것으로 신뢰합니다.

이렇게 하면 인증 검사가 한 곳에 모여 각 서비스는 비즈니스 로직에만 집중하고, 인증 정책이 바뀌어도 게이트웨이만 고치면 됩니다. 대신 인증 책임이 게이트웨이에 몰리고, 게이트웨이를 우회한 내부 호출은 막지 못합니다. Nginx만으로는 토큰 검증이 안 되어 Spring Cloud Gateway 같은 도구를 쓰며, 이 책은 단순함을 위해 서비스별 검증을 씁니다.

3.4.2 MySQL - 데이터베이스 인프라

모든 서비스가 동일한 MySQL 인스턴스(`db-service:3306`)의 `metadb` 데이터베이스를 공유합니다. 서비스별로 테이블이 분리되어 있으나, 물리적으로는 단일 DB 인스턴스입니다.

실제 MSA에서는 서비스마다 독립된 DB를 둡니다. 이 책에서는 학습 편의를 위해 하나의 MySQL을 공유합니다. 학습 흐름을 익히는 데는 차이가 없으니 DB 구성보다 흐름에 집중해 주세요.

DB 컨테이너는 `db/` 디렉토리의 두 파일로 구성됩니다.

파일	역할
Dockerfile	MySQL 공식 이미지를 베이스로 초기화 SQL을 컨테이너에 넣어 띄웁니다.
init.sql	네 서비스에 필요한 테이블을 만들고 더미 데이터를 채웁니다.

3.5 Kubernetes - YAML로 선언하는 배포

3.5.1 매니페스트 구조 설계

Kubernetes는 YAML 파일로 원하는 상태를 선언합니다. "이 서비스는 이렇게 실행되어야 한다" 고 파일에 적어두면, K8s가 그 상태를 유지합니다.

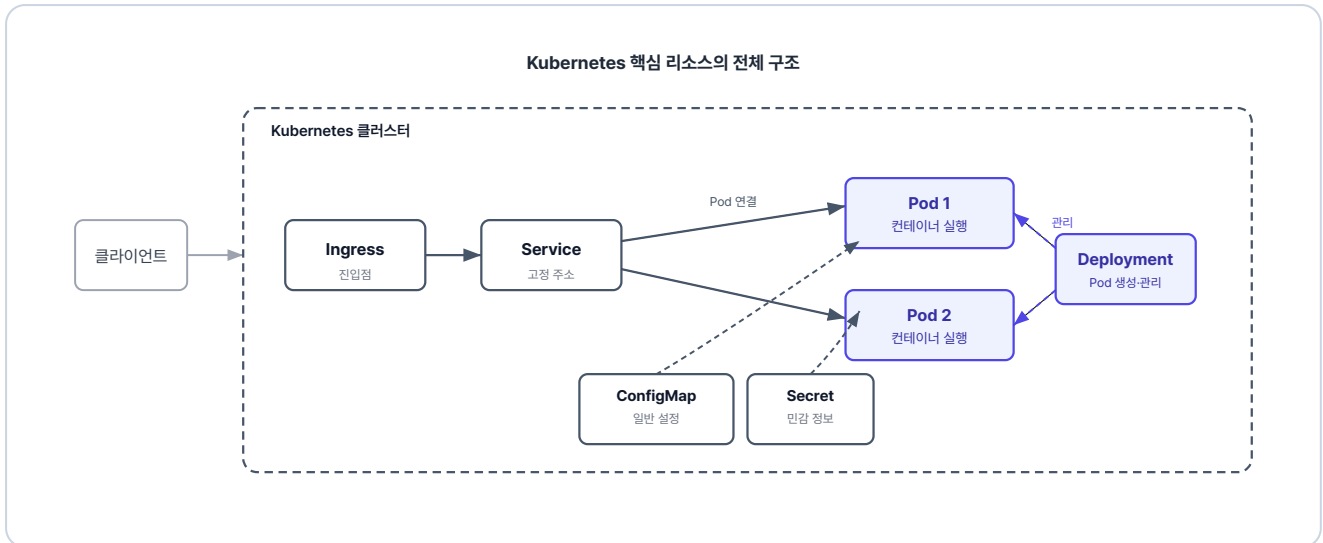


그림 3-6. Kubernetes 리소스 관계

각 K8s 리소스의 역할을 정리하면 다음과 같습니다.

리소스	역할
ConfigMap	일반 환경변수(DB 주소 등)를 외부에서 주입합니다.
Secret	민감 정보(DB 계정·비밀번호)를 분리해 관리합니다.
Secret (DB)	MySQL 컨테이너가 시작될 때 데이터베이스와 계정을 자동으로 만들 수 있도록 환경변수를 줍니다.
Deployment	Pod를 어떻게 실행할지 정의하고, ConfigMap과 Secret을 한꺼번에 주입합니다.
Service	Pod에 고정 주소를 부여해 클러스터 안에서 안정적으로 통신할 수 있게 합니다.
Ingress	클러스터 외부 요청을 클러스터 안으로 들여보냅니다.

나머지 서비스(product, user, delivery)도 동일한 패턴입니다. 전체 매니페스트는 레포의 [k8s/](#) 디렉토리를 참고하세요.

Gateway API로 두면 라우팅 규칙을 재사용하기 쉽다

이 책은 minikube에서 Ingress로 외부 진입을 구성하지만, Ingress의 후속 표준인 Gateway API를 쓰는 선택지도 있습니다. 쿠버네티스에 기본 내장된 Ingress와 달리 이 표준은 별도로 설치해 쓰며, 진입점을 정의하는 Gateway와 경로 규칙을 정의하는 HTTPRoute로 역할이 나뉩니다. 경로 규칙은 환경이 바뀌어도 그대로 두고, 진입점 정의만 환경에 맞춰 바꿉니다.

AWS EKS 같은 환경에서는 컨트롤러가 이 선언을 읽어 관리형 로드 밸런서를 붙여 줍니다. 로컬에서는 진입점을 직접 띄워야 하지만, 클라우드에서는 이 과정이 자동으로 처리됩니다. 라우팅 규칙을 다시 짤 필요가 없어, 로컬에서 쓰는 경로 설정을 운영 환경에서도 그대로 씁니다.

3.6 Minikube - 실행 및 결과 확인

3.6.1 Minikube 시작

Minikube는 로컬 PC에 가벼운 Kubernetes 클러스터를 만들어주는 도구입니다. 설치되어 있지 않다면 OS에 맞게 먼저 설치합니다.

터미널 Minikube 설치 - macOS

```
brew install minikube
```

터미널 Minikube 설치 - Windows

```
winget install Kubernetes.minikube
```

설치한 뒤 새 터미널을 열고, Docker Desktop이 실행 중인 상태에서 아래 명령을 입력하면 클러스터가 생성됩니다.

터미널 Minikube 시작

```
minikube start
```

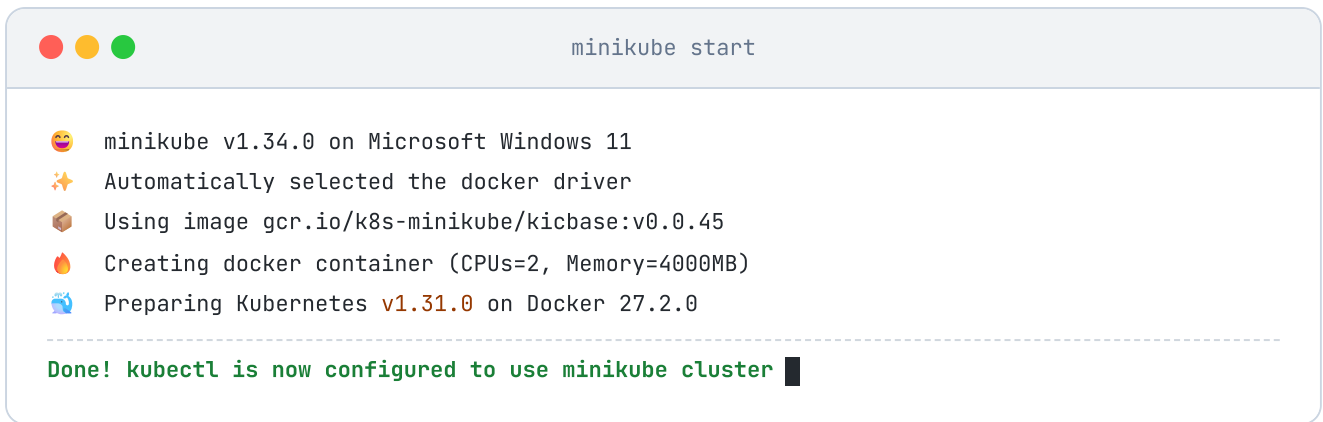


그림 3-7. Minikube 시작

처음 실행하면 필요한 이미지를 다운로드하므로 몇 분 정도 걸릴 수 있습니다.

3.6.2 이미지 빌드

`minikube image build` 는 Minikube 내부에 직접 이미지를 빌드합니다.

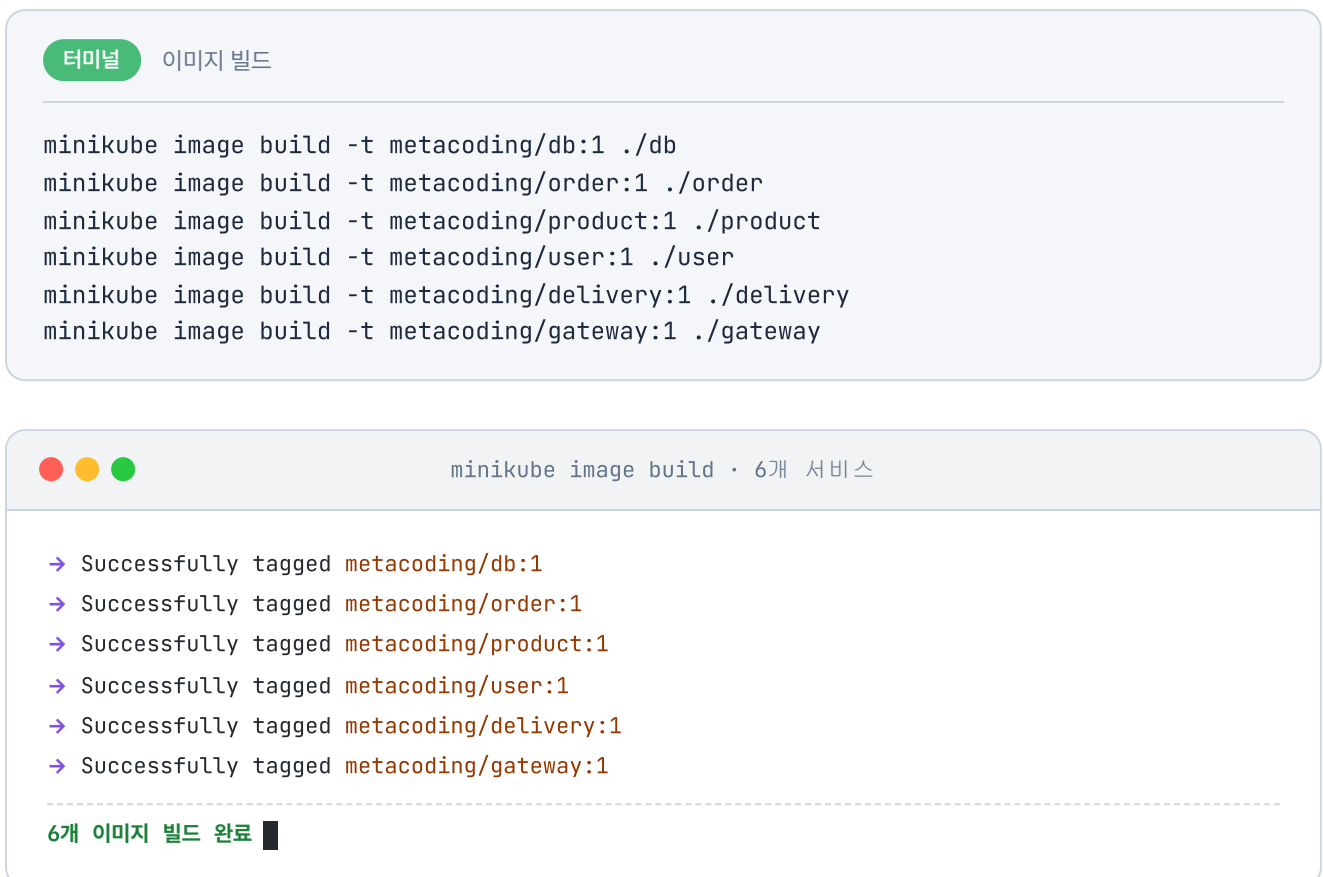


그림 3-8. 이미지 빌드 결과

3.6.3 배포 순서

네임스페이스를 먼저 생성하고, DB가 준비된 뒤에 나머지 서비스를 배포합니다.

터미널 배포 순서

```
# 1. 네임스페이스 생성
kubectl create namespace metacoding

# 2. DB 관련 리소스 먼저 배포
kubectl apply -f k8s/db

# 3. 각 서비스 배포
kubectl apply -f k8s/order
kubectl apply -f k8s/product
kubectl apply -f k8s/user
kubectl apply -f k8s/delivery
kubectl apply -f k8s/gateway

# 4. Ingress 활성화 (Minikube에서는 애드온 활성화 필요)
minikube addons enable ingress
```

kubectl apply · 네임스페이스 + 6개 서비스

```
namespace/metacoding created
secret/db-secret created
deployment.apps/db-deploy created
service/db-service created
configmap/order-configmap created
secret/order-secret created
deployment.apps/order-deploy created
service/order-service created
... product · user · delivery · gateway 동일 패턴 ...
ingress.networking.k8s.io/gateway-ingress created

-----
전체 리소스 배포 완료 ■
```

그림 3-9. 네임스페이스 생성 및 배포

3.6.4 배포 상태 확인

배포가 끝나면 모든 Pod가 제대로 실행되고 있는지 확인합니다.

터미널 Pod 상태 확인

```
kubectl get pods -n metacoding
```



kubectl get pods -n metacoding

NAME	READY	STATUS	AGE
db-deploy-6f9b7c4d8-m4t2q	1/1	Running	42s
gateway-deploy-5c8d6f7b9-h7w3r	1/1	Running	38s
order-deploy-8b7f6c9d4-q2k8m	1/1	Running	36s
product-deploy-7c9d8b6f5-x4r2t	1/1	Running	35s
user-deploy-6d8c7b9f4-p3m9k	1/1	Running	33s
delivery-deploy-9f7c8b6d5-t6w2x	1/1	Running	30s

모든 Pod Running ■

그림 3-10. Pod 상태 확인

모든 Pod가 **Running** 상태가 되면 배포 완료입니다.

3.6.5 서비스 접근

Ingress를 통해 외부에서 접속하려면 **minikube tunnel** 을 실행합니다.

터미널 외부 접근 터널

```
minikube tunnel
```

minikube tunnel 은 터미널을 점유합니다.

터널이 실행되면 **http://127.0.0.1:80** 로 gateway-service에 접속할 수 있습니다. 회원 서비스가 발급한 토큰은 하루 동안 유효하므로, 챕터 2에서 받은 토큰을 그대로 써서 아래 주문을 생성합니다. 만료됐다면 같은 방법으로 다시 발급받습니다.

POST `http://127.0.0.1:80/api/orders`

```
{
  "productId": 1,
  "quantity": 1,
  "price": 2500000,
  "address": "Addr 4"
}
```

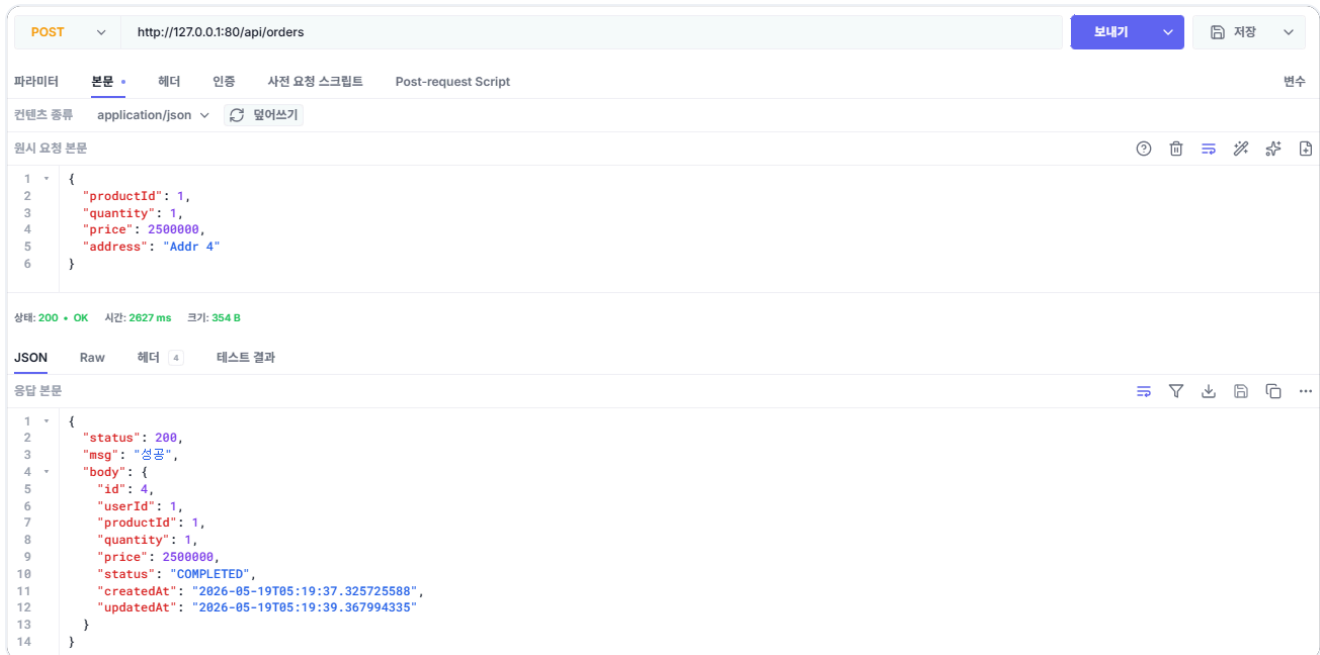


그림 3-11. 주문 결과 확인

테스트가 끝났으면 이번 챕터에서 실행한 리소스를 정리합니다.

터미널 리소스 정리

```
kubectl delete namespace metacoding
```

오픈이 : "Kubernetes에서도 주문이 잘 되네요! 안은 도메인이 자기 일을 알고, 밖은 한 입구, 위에서는 K8s가 살려주잖아요."

선배 : "K8s가 Pod 죽으면 살려주죠. 그런데 어떤 Pod이 죽었다가 다시 살아나는 그 사이에는 어떻게 될 것 같아요?"

살아나는 동안에는...?

오픈이가 답을 못 했습니다. K8s가 죽은 Pod을 알아서 다시 띄워준다니 괜찮을 것 같기도 하고, 그런데 다시 뜨는 그 사이에 들어온 요청이 어떻게 될지는 아직 떠올려 본 적이 없었습니다.

코드 구조도 좋아졌고, 운영 배포도 됐고, `Order`가 자기 일을 답하게 됐습니다. 다만 선배의 질문이 머릿속에 남았습니다.

이것만은 기억하자

→ **DDD**로 비즈니스 규칙을 도메인 객체에 캡슐화했습니다.

`Order.validateCancelable()` · `Order.cancel()` 로 도메인이 자기 일을 압니다.

→ **클린 아키텍처**(UseCase 인터페이스)로 컨트롤러가 `OrderService` 코드가 아닌 약속에 의존하게 했습니다. 환경별·테스트별로 `OrderService` 코드를 갈아끼울 수 있습니다.

→ **Nginx API Gateway**로 네 서비스 진입점을 하나로 모았습니다. URL 경로에 따라 적절한 서비스로 라우팅됩니다.

→ **Kubernetes 배포**에서 ConfigMap·Secret으로 환경변수를 주입하고, Deployment가 Pod를 자동으로 복구합니다.

다음 챕터에서는 동기 호출의 한계를 정면으로 마주칩니다. Kafka로 서비스 간 전화를 끊고, 메시지를 사이에 두며, 중앙에 지휘자(orchestrator)를 둡니다. 한 서비스가 잠시 멈춰도 전체 시스템은 계속 동작합니다.

챗터 4. 비동기 MSA - Kafka로 서비스를 분리하다

이 챗터의 전체 소스코드는 <https://github.com/metacoding-12-msa/ex03> 에서 확인할 수 있습니다.

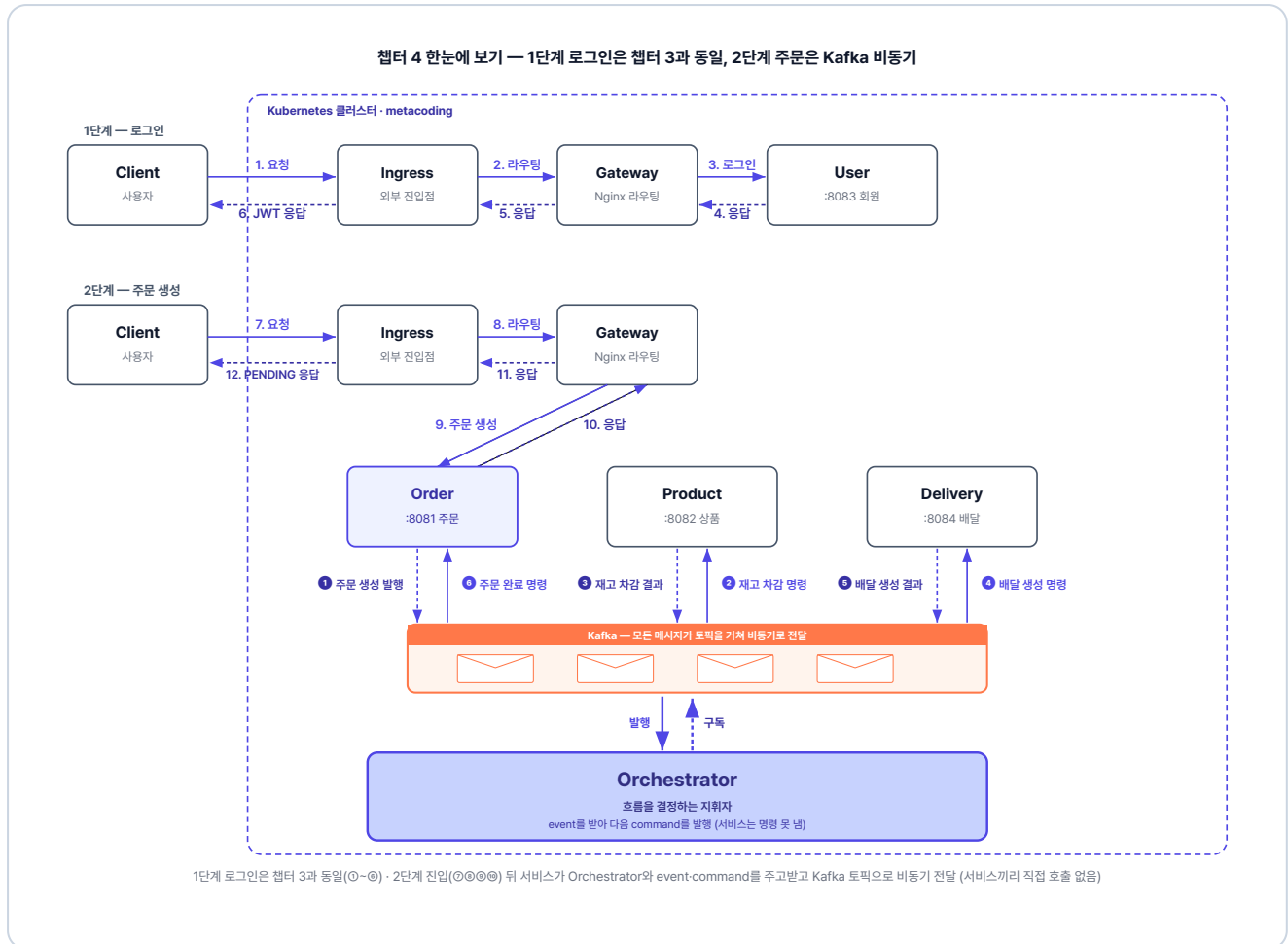


그림 4-1. 챗터 4 한눈에 보기 - 서비스·Orchestrator·Kafka 3층 구조

이번 챗터가 끝나면

- 동기 REST 호출의 한계를 이해하고 비동기 메시지 방식의 장점을 설명할 수 있습니다.
- Kafka의 토픽, 프로듀서, 컨슈머, 컨슈머 그룹 개념을 이해할 수 있습니다.
- Orchestration Saga 패턴을 Kafka로 구현할 수 있습니다.
- order-service의 동기 호출을 Kafka 이벤트 발행으로 교체할 수 있습니다.
- orchestrator가 워크플로우 상태를 추적하고 실패 시 자동 롤백하는 방법을 이해할 수 있습니다.

준비하기. 실습 시작 전 한 번만 설정

1. 소스 코드 클론

터미널 레포 클론

```
git clone https://github.com/metacoding-12-msa/ex03.git
cd ex03
```

2. 파일 구조

ex03 디렉토리

```
ex03/
├─ order/           # 포트 8081
├─ product/        # 포트 8082
├─ user/           # 포트 8083
├─ delivery/       # 포트 8084
├─ orchestrator/   # Kafka 워크플로우 조울 (이번 챕터 신규)
├─ gateway/        # Nginx API Gateway
├─ db/             # MySQL
└─ k8s/            # Kubernetes 매니페스트 (kafka 포함)
```

서비스마다 패키지 구조가 조금씩 다르므로, 코드를 작성할 파일 경로는 각 실습 코드블록 바로 위에서 안내합니다.

3. Kafka

이 챕터는 Kafka를 별도로 설치하지 않습니다. Kubernetes 매니페스트(`k8s/kafka/`)로 Minikube 안에 띄우므로, 챕터 3에서 준비한 Docker Desktop과 Minikube만 있으면 됩니다.

4. 실습 순서

1. order-service의 REST 호출을 Kafka 이벤트 발행으로 교체
2. product-service · delivery-service에 Kafka Consumer/Producer 추가
3. 새 서비스 **orchestrator**에서 워크플로우 조울 로직 작성
4. K8s에 Kafka + orchestrator 배포
5. 정상 주문 / 품질 롤백 시나리오 검증

이번 챕터는 책에서 가장 복잡한 내용을 다룹니다. 여러 서비스가 동시에 Kafka 메시지를 주고받으며 상태가 바뀝니다. 전체 토픽맵(4.2.2)과 흐름 표(4.3.1~4.3.2)를 먼저 훑어보고 시작하면 길을 잃지 않습니다.

4.1 동기 호출의 한계 - 한 서비스가 멈추면 전체가 멈춘다

배포 다음 날 아침, 모니터에 알림이 울렸습니다. 에러 그래프가 한 칸 솟았다가 곧 가라앉았습니다. 상품 서버가 잠깐 죽었고, 쿠버네티스가 다시 띄웠습니다.

그런데 그 잠깐 사이에 들어온 주문이 전부 실패해 있었습니다. 주문 서버는 재고를 줄이려고 상품 서버를 직접 호출했는데, 상품 서버가 죽어 있어 호출이 실패했습니다. 주문 처리는 그 응답을 기다리던 중이라, 호출이 실패하면서 주문 처리도 함께 실패했습니다.

서로 직접 부르니까, 하나가 멈추면 기다리던 쪽도 같이 멈추는구나.

선배가 모니터를 들여다봤습니다.

선배 : "동기로 직접 호출하니까 그래요. 호출한 쪽은 상대가 응답할 때까지 기다리는데, 상대가 잠깐 죽으면 호출한 쪽도 같이 멈춰요. 이럴 땐 호출을 붙잡고 기다리지 말고 비동기로 메시지를 주고받아야 해요. 그 비동기 통신을 해 주는 게 **Kafka**예요."

오픈이 : "메시지요? 상대가 없으면 어떻게 되는데요?"

선배 : "주문하는 쪽은 메시지를 남기고 진동벨을 받고 돌아가요. 재고 담당자는 자기 차례가 되면 쌓인 메시지를 꺼내 처리해요. 담당자가 잠깐 자리를 비워도 메시지는 그 자리에 남아 있어요. 돌아와서 그때 처리하면 되죠."

선배 : "결국 이게 MSA가 노린 거예요. 서비스를 따로 배포하고, 하나가 죽어도 나머지는 멀쩡해야죠. 그런데 서로 직접 호출하면서 응답을 기다리는 한, 하나가 멈추면 다 같이 멈춰요. 카페로 치면 카운터 앞에서 기다리는 것과 진동벨을 받고 자리에 앉는 것의 차이예요. 그게 바로 동기와 비동기의 차이예요."

4.1.1 카운터 대기 vs 진동벨

카운터 대기 방식 (동기적)

카운터 앞에 서서 "아메리카노 하나요"라고 주문합니다. 바리스타가 커피를 만드는 동안 카운터 앞에서 기다립니다. 내 커피가 나와야 다음 손님이 주문할 수 있습니다. 바리스타가 원두를 갈다가 커피 머신이 고장 나면 뒤에 줄 선 손님 전부가 멈춥니다.

진동벨 방식 (비동기적)

"아메리카노 하나요"라고 주문하면 진동벨을 받고 자리에 앉습니다. 바리스타는 주문을 순서대로 처리하고, 커피가 완성되면 벨이 울립니다. 손님은 기다리는 동안 다른 일을 할 수 있고, 뒤에 줄 선 손님도 바로 주문할 수 있습니다. 커피 머신이 잠깐 멈춰도 주문 목록은 사라지지 않습니다.

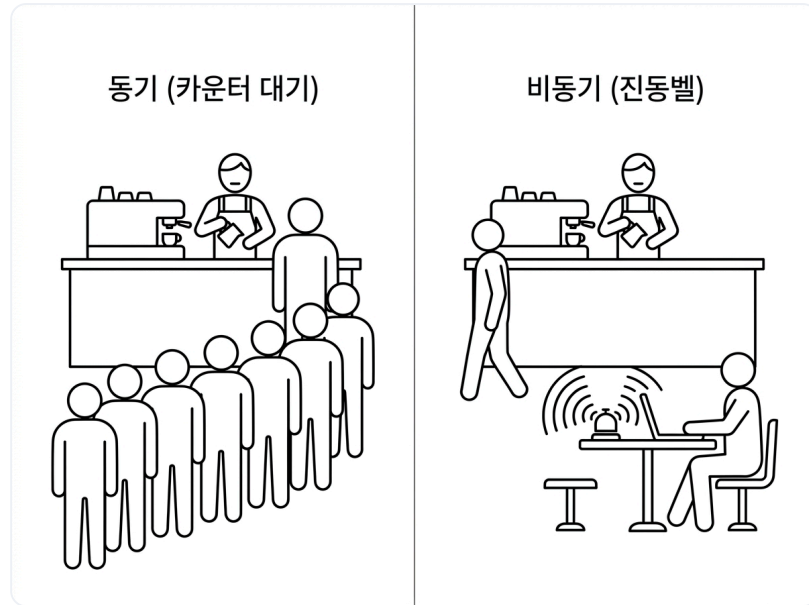


그림 4-2. 동기 vs 비동기 통신

아침에 겪은 그 잠깐의 멈춤이 바로 카운터 대기의 풍경이었습니다. 커피 머신이 멈추자 카운터 앞에 선 손님까지 함께 멈췄고, 그 사이 들어온 주문이 전부 실패했습니다. 진동벨 방식이라면 손님은 주문만 남기고 자리로 돌아가 있으니, 커피 머신이 잠깐 멈춰도 주문은 그대로 남아 있다가 커피 머신이 복구된 후에 처리됩니다. 동기서로를 직접 호출하며 응답을 기다리면 한 서비스의 멈춤이 곧 전체의 멈춤이 되지만, 비동기로 메시지를 주고받으면 그렇지 않습니다.

4.2 Kafka - 메시지를 전달하는 우체국

비동기 통신을 실제로 담당하는 것이 **Kafka**입니다.

Apache Kafka는 서비스끼리 직접 호출하지 않고, 가운데에서 메시지를 받아 보관했다가 전달해주는 메시지 시스템입니다. 받는 쪽이 잠시 멈춰도 메시지가 그대로 남아 있어 비동기 통신이 가능합니다.

Kafka는 우체국입니다. 발신자가 우편을 부치면, 우체국은 **일반 편지, 등기, 특송, 택배처럼 종류별로 나눠** 따로 보관합니다. 그리고 집배원은 자기가 처리할 종류의 칸에서만 우편을 꺼내 갑니다. 우체국을 사이에 두고 우편을 주고받기 때문에, 비동기적으로 각자의 시간에 일을 처리할 수 있습니다.



그림 4-3. Kafka 우체국 - 부치고, 보관·분류되고, 집배원이 가져간다

그림 4-3의 Kafka 요소의 역할을 정리하면 다음과 같습니다.

구성요소	우체국 비유	하는 일
Kafka	우체국	메시지를 받아 보관하고 전달
토픽(Topic)	종류별 우편함 (일반/등기/특송 등)	메시지를 목적별로 나눠 담음
프로듀서(Producer)	발신자	토픽에 메시지를 발행
컨슈머(Consumer)	집배원	토픽을 구독해 메시지를 처리

프로듀서가 토픽에 메시지를 보내는 일을 **발행**, **컨슈머**가 자기가 다룰 토픽을 정해 두고 거기에 들어온 메시지를 받아 처리하는 일을 **구독**이라고 합니다.

그런데 같은 일을 하는 컨슈머가 여러 대 떠 있을 때는 메시지를 어떻게 나눠 받아야 할까요?

4.2.1 컨슈머 그룹

상품 서비스를 2대로 늘려 운영한다고 가정해 보겠습니다. 두 서버가 같은 토픽을 구독하면 "재고 1개 차감" 메시지를 둘 다 받아 처리해, 재고가 2개 줄어듭니다. 이 중복 처리를 막아 주는 것이 **컨슈머 그룹**입니다. 같은 일을 하는 서버를 한 그룹으로 묶으면, 그 메시지는 그룹 안의 한 서버에만 전달됩니다.

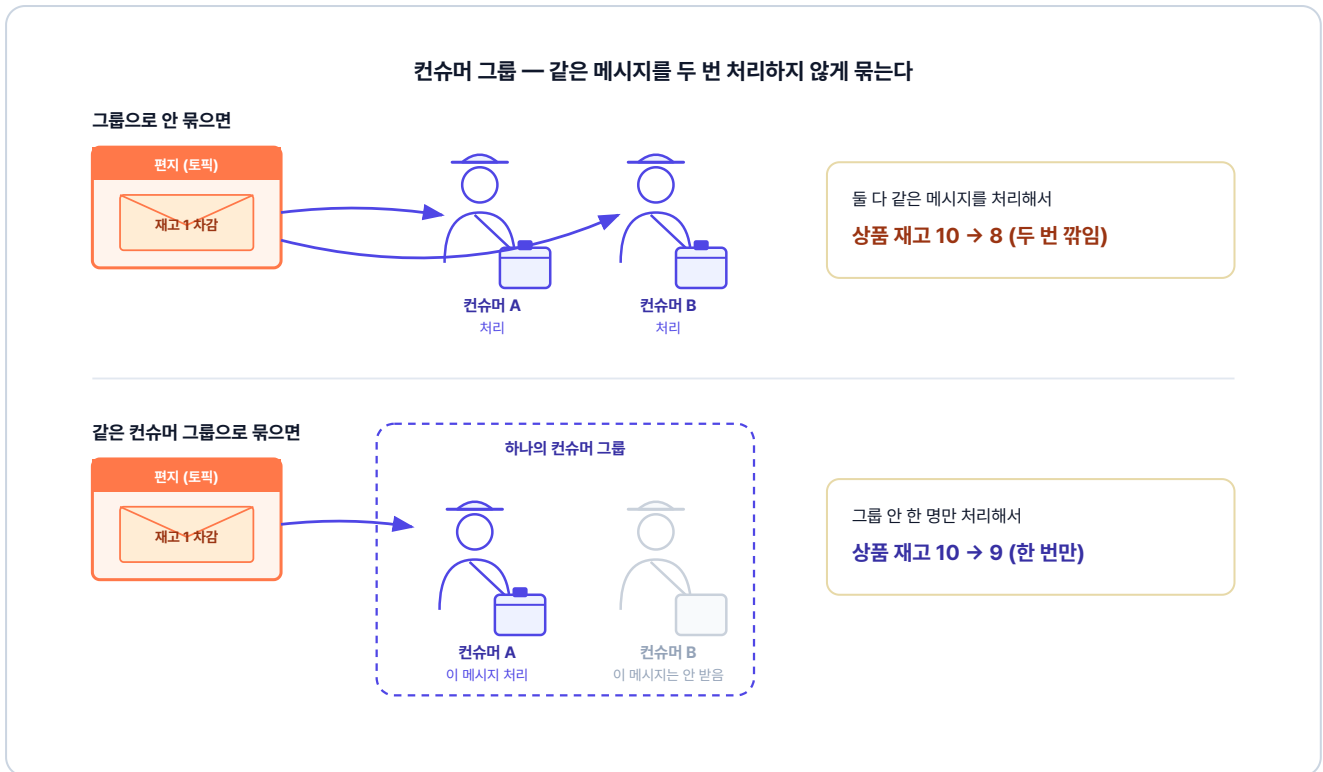


그림 4-4. 컨슈머 그룹 - 안 묶으면 재고가 두 번 깎이고, 같은 그룹으로 묶으면 한 번만 깎인다

4.2.2 이번 챕터에서 사용하는 토픽 맵

이 챕터에서는 핵심 흐름만 직접 다루므로, 각 토픽의 발행·구독 코드 전체는 깃헙 레포에서 확인합니다.

총 8개의 토픽을 사용합니다. `command` 가 붙은 토픽은 orchestrator가 각 서비스에 내리는 명령입니다.

`command` 가 없는 토픽은 각 서비스가 처리 결과를 orchestrator에 보고하는 이벤트입니다.

정상 흐름 (6단계)

단계	토픽	발행 → 구독	목적
1	<code>order-created</code>	order → orchestrator	새 주문 발생
2	<code>decrease-product-command</code>	orchestrator → product	재고 감소 명령
3	<code>product-decreased</code>	product → orchestrator	재고 감소 결과
4	<code>create-delivery-command</code>	orchestrator → delivery	배달 생성 명령
5	<code>delivery-created</code>	delivery → orchestrator	배달 생성 결과
6	<code>complete-order-command</code>	orchestrator → order	주문 완료 명령

롤백 흐름 (2개)

토픽	발행 → 구독	목적
<code>cancel-order-command</code>	orchestrator → order	주문 취소
<code>increase-product-command</code>	orchestrator → product	재고 복구

오픈이 : "메시지 방식으로 바꾸니까, 전체 흐름을 누가 관리해야 할지 모르겠어요."

선배 : "전체 흐름을 관리하는 **지휘자**를 만들면 편해요. 전체 악보를 보고 순서를 정하는 것처럼, 각 서비스는 자기 일만 하고 결과만 보고하게 해요."

Kafka 더 알아보기

- **컨슈머가 읽어도 메시지는 남는다**: 우체국에서 우편을 꺼내 읽어도 그 우편은 함에 그대로 보관됩니다(기본 7일). 그래서 서로 다른 컨슈머가 같은 메시지를 각자 자기 속도로 찾아 읽을 수 있고, 장애가 나도 다시 읽어 처리할 수 있습니다.
- **RabbitMQ 같은 큐와의 차이**: 전통적인 메시지 큐는 컨슈머가 메시지를 읽으면 큐에서 바로 사라집니다. Kafka는 읽어도 남아 있다는 점이 가장 큰 차이입니다.

4.3 Orchestration Saga - 지휘자가 흐름을 조율하다

각 서비스가 자기 단계만 처리해 결과를 메시지로 발행하고, 별도의 관리자가 다음 단계와 보상을 책임지는 분산 트랜잭션 패턴을 **Saga 패턴**이라고 합니다.

Saga 패턴의 흐름 관리를 한 서비스가 중앙에서 지휘하는 방식이 **Orchestration Saga**, 그 지휘자 서비스가 **orchestrator**입니다. 흐름을 한 곳에 모으면 상태가 한 곳에 있어 추적과 롤백이 단순해지고, 각 서비스는 자기 일에만 집중할 수 있습니다.

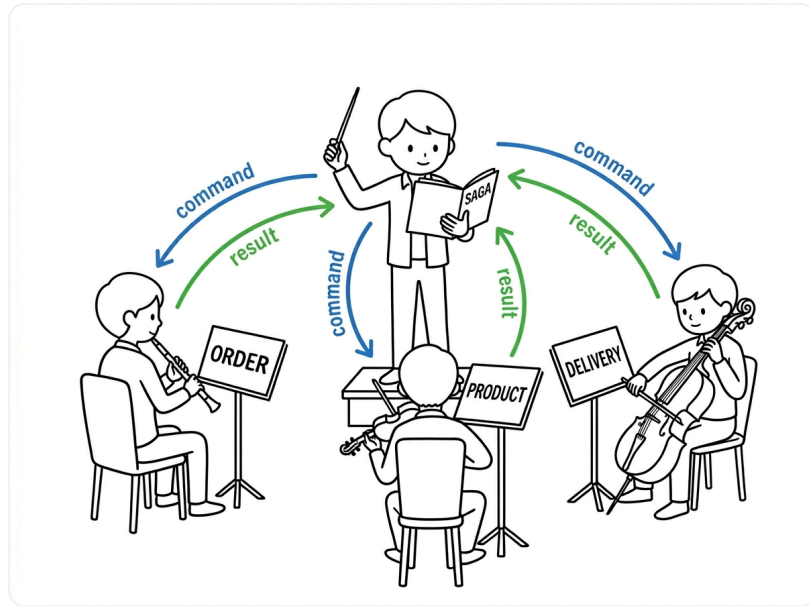


그림 4-5. Orchestration Saga 구조

이번 챕터의 예제에서는 주문 요청이 들어오면 orchestrator가 재고 차감, 배달 생성, 주문 완료 단계를 순서대로 지휘합니다.

Saga 패턴을 구현하는 방식에는 Choreography도 있습니다. 중앙 지휘자 없이 각 서비스가 이벤트를 발행하고 구독하며 다음 단계를 스스로 이어 갑니다. 단계가 적을 때는 가볍지만, 단계가 늘거나 보상이 복잡해질수록 전체 흐름이 여러 서비스에 흩어져 추적이 어렵습니다. 이 책은 상태를 한 곳에서 추적할 수 있는 Orchestration을 선택합니다.

4.3.1 주문 요청 성공 흐름

orchestrator를 사이에 두고 비동기로 동작하면, 주문 요청이 들어왔을 때 주문 서비스는 우선 **PENDING** 상태를 응답합니다. 이후 orchestrator가 아래 단계를 차례로 진행해 모두 성공하면 주문 상태가 **COMPLETED**로 바뀝니다.

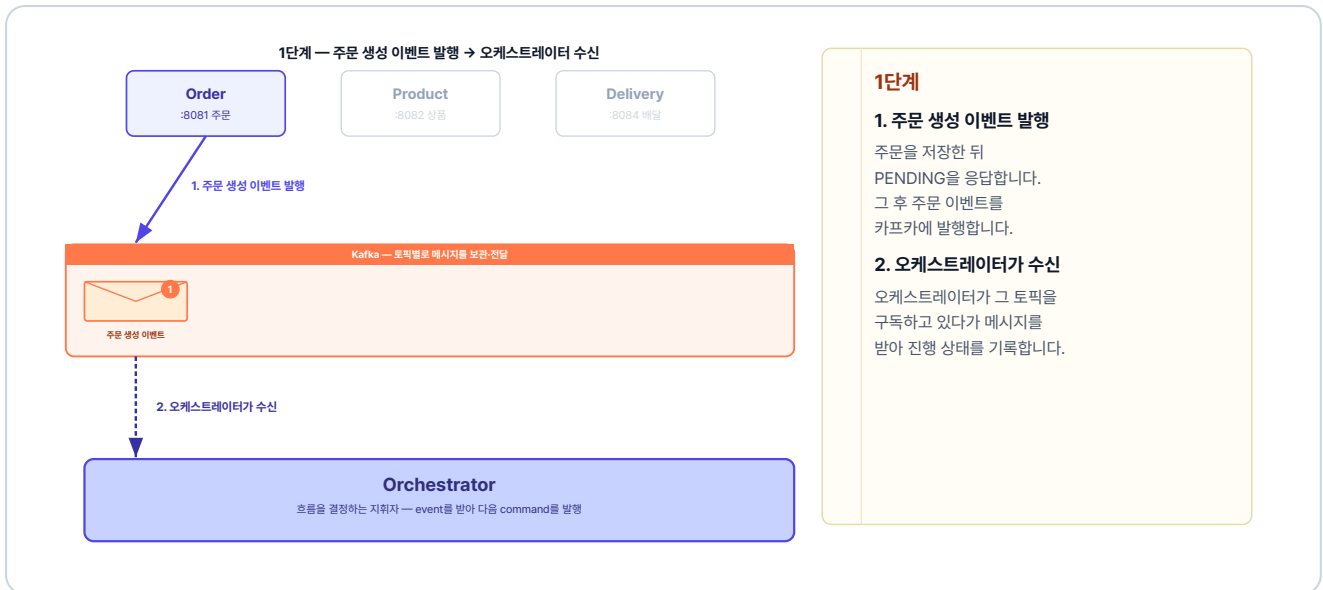


그림 4-6. 1단계 — 주문 생성 이벤트 발행 → 오케스트레이터 수신

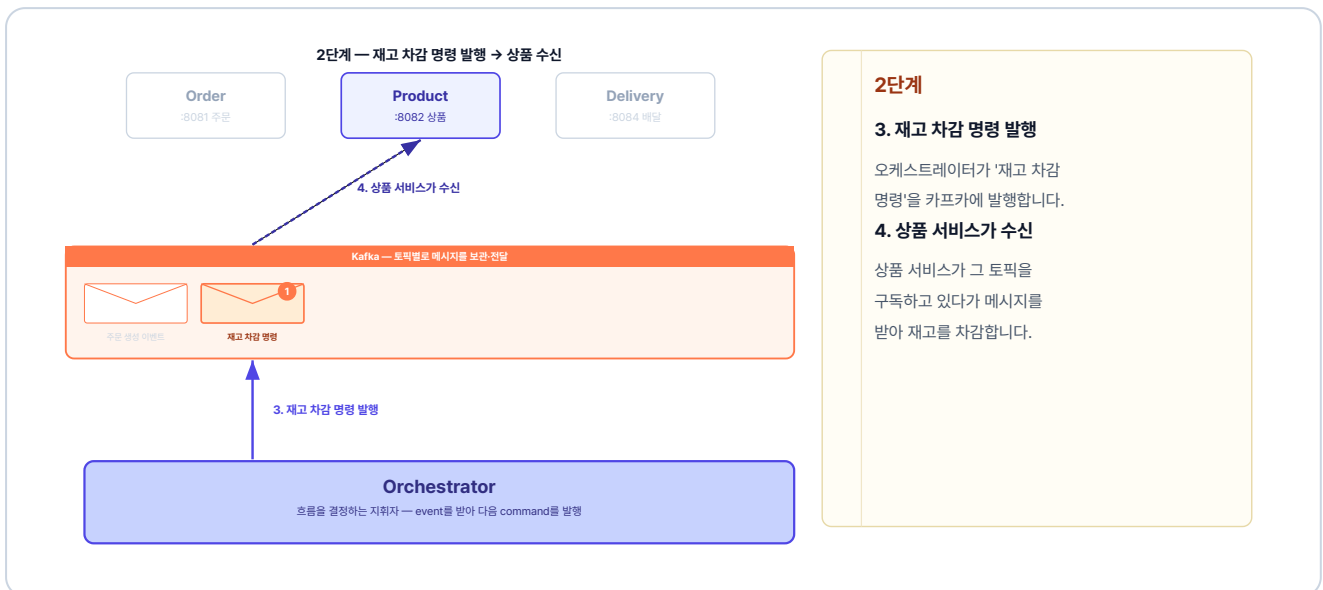


그림 4-7. 2단계 — 재고 차감 명령 발행 → 상품 수신

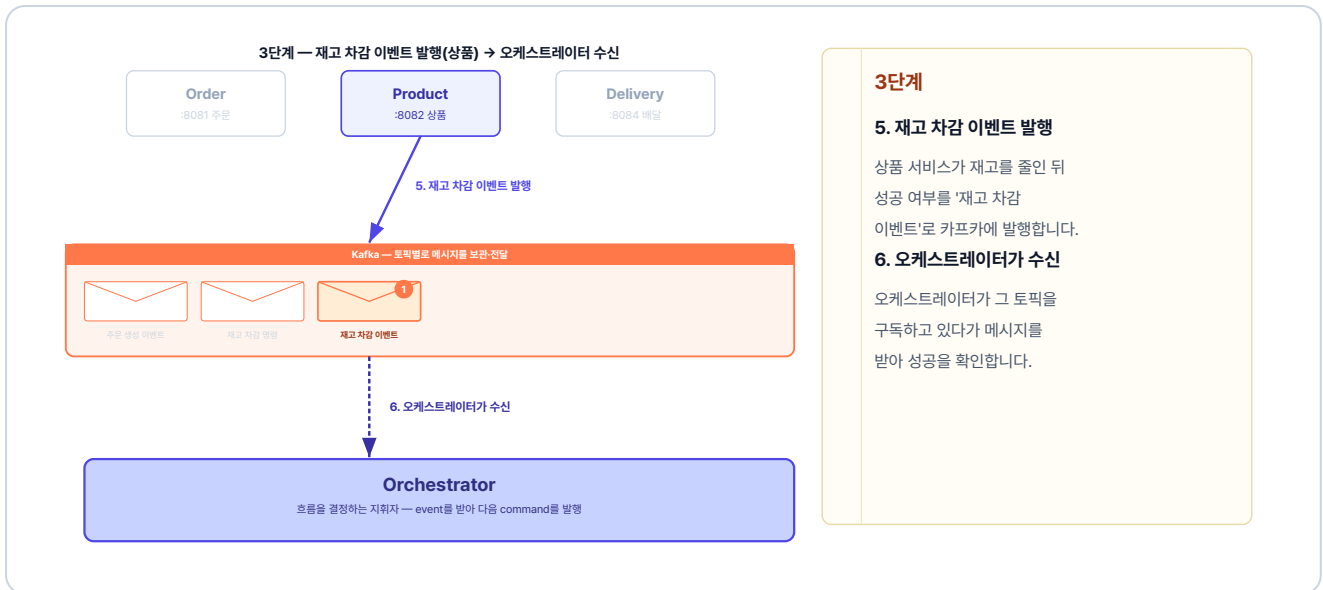


그림 4-8. 3단계 — 재고 차감 이벤트 발행(상품) → 오케스트레이터 수신

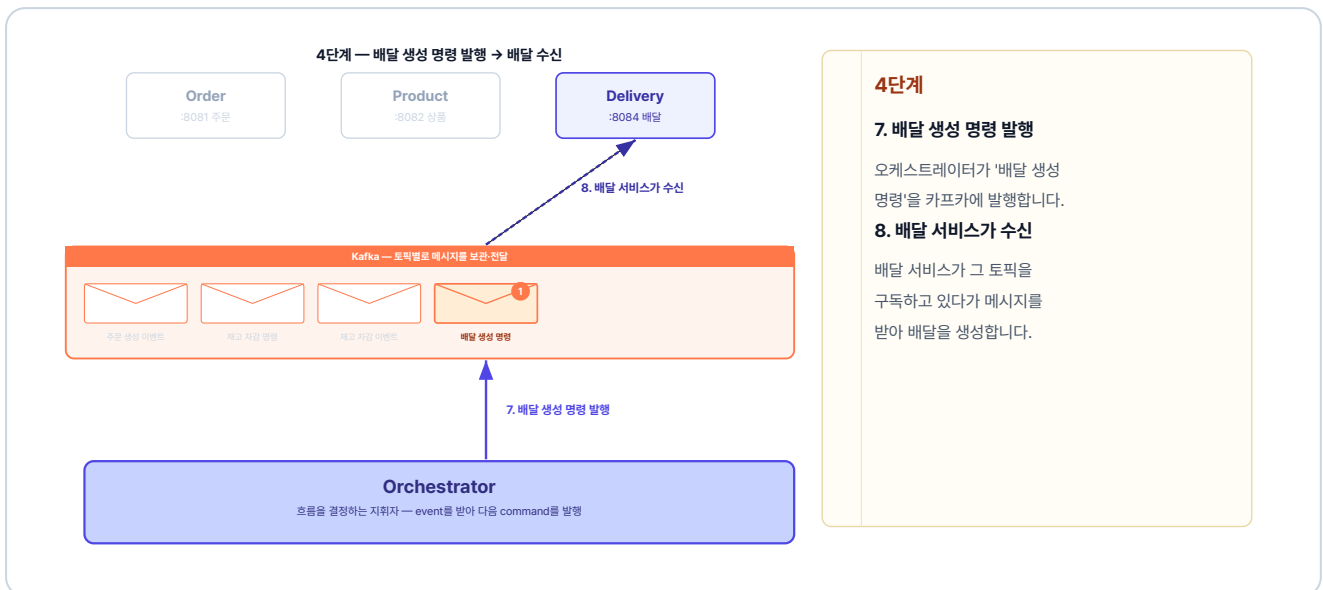


그림 4-9. 4단계 — 배달 생성 명령 발행 → 배달 수신

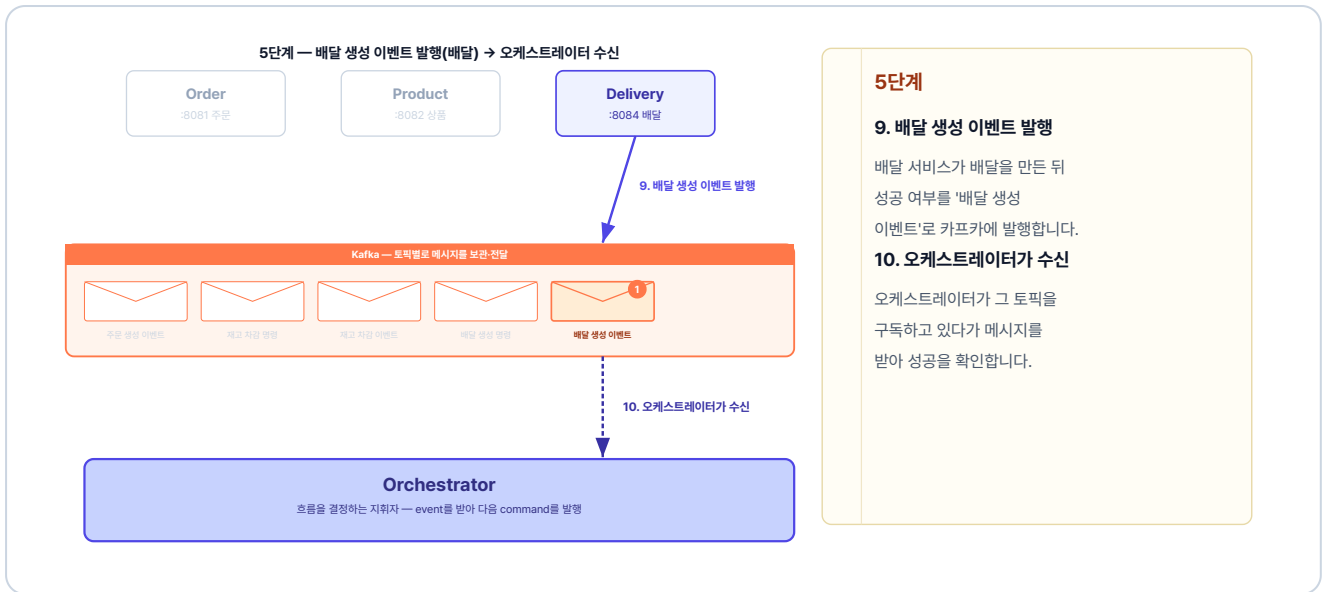


그림 4-10. 5단계 — 배달 생성 이벤트 발행(배달) → 오케스트레이터 수신

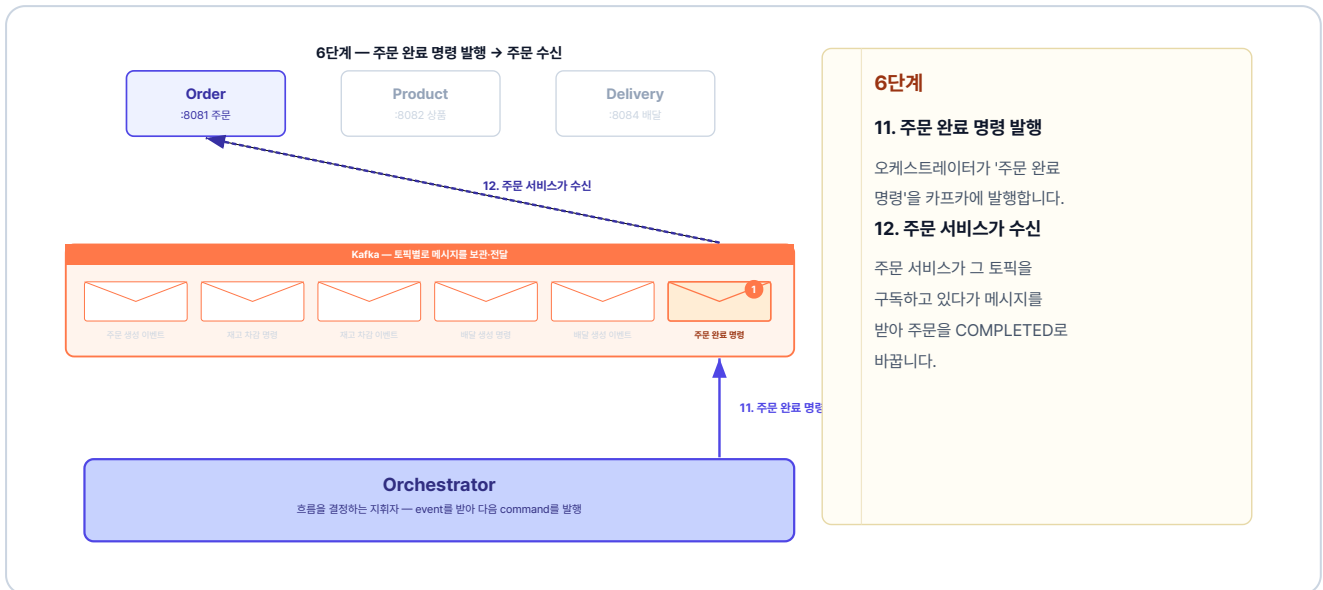


그림 4-11. 6단계 — 주문 완료 명령 발행 → 주문 수신

4.3.2 주문 요청 실패 흐름 (롤백)

상품 서비스가 재고 차감에 실패하면 재고 차감 실패 이벤트를 발행합니다. 오케스트레이터는 이미 처리된 단계만 역순으로 되돌립니다.



그림 4-12. 롤백 · 실패 알림 — 재고 차감에 실패한 상품이 그 결과를 오케스트레이터에 알립니다

되돌릴 재고가 없으니 남은 일은 주문을 무르는 것입니다. 오케스트레이터는 주문 취소 명령을 발행하고, 주문 상태가 CANCELLED로 변경됩니다.

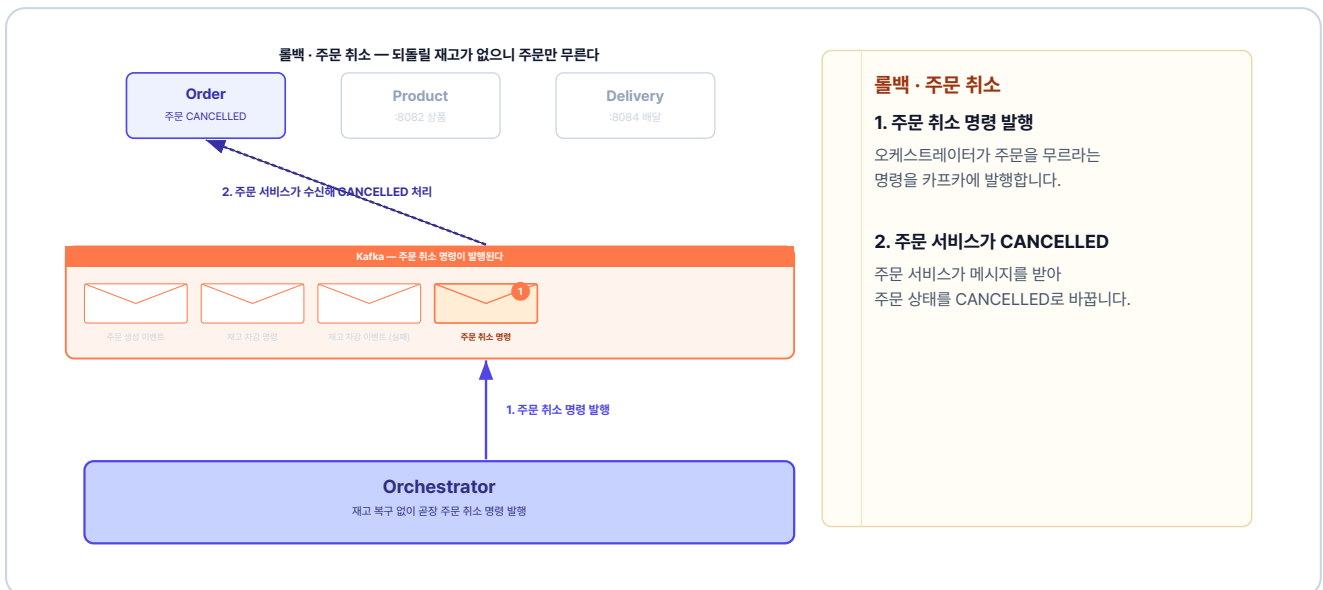


그림 4-13. 롤백 · 주문 취소 — 되돌릴 재고가 없으니, 주문 취소 한 단계로 롤백이 끝납니다

선배 : "흐름은 잡혔으니, 이제 직접 만들어 봐요."

4.4 Kafka로 주고받기 - 발행과 구독

4.4부터는 이 흐름을 코드로 옮깁니다. Kafka 코드는 메시지를 토픽에 넣는 발행과, 토픽을 듣고 있다가 받는 구독으로 이뤄집니다.

4.4.1 발행 - 토픽에 메시지를 넣는다

앞에서 본 발신자가 우편을 부치듯, 프로듀서도 메시지를 토픽에 부칠 도구가 필요합니다. Spring에서 그 도구로 제공하는 것이 `KafkaTemplate` 입니다. `"order-created"` 처럼 이름을 토픽 자리에 넘기면 그 토픽으로 메시지가 들어갑니다. 주문 서비스가 주문을 저장한 뒤 '주문 생성 이벤트'를 발행하는 코드는 다음과 같습니다.

`adapter/producer/OrderEventProducer.java` 를 열고 아래 메서드를 작성합니다.

실습 1 `adapter/producer/OrderEventProducer.java`. 주문 생성 이벤트 발행

```
@Component
@RequiredArgsConstructor
public class OrderEventProducer {
    private final KafkaTemplate<String, Object> kafkaTemplate;

    public void publishOrderCreated(OrderCreatedEvent event) {
        // "order-created" 토픽에 이벤트를 넣는다
        kafkaTemplate.send("order-created", event);
    }
}
```

`kafkaTemplate.send` 에 토픽 이름과 보낼 객체를 넘기면 해당 토픽으로 메시지가 들어갑니다.

주문 서비스는 주문을 저장하고 '주문 생성 이벤트'를 발행하면 됩니다. 그다음 재고 차감·배달 생성·실패 복구는 orchestrator가 맡습니다.

4.4.2 orchestrator - 흐름을 조율하는 코드

이벤트를 받아 다음 명령을 정하는 일은 orchestrator만 합니다. 1단계(주문 생성 이벤트 수신)를 코드로 보면 다음과 같습니다.

`@KafkaListener`의 `topics`는 구독할 토픽 이름입니다. 토픽은 메시지가 종류별로 쌓이는 우편함이고, "order-created"처럼 그 우편함 이름을 적습니다. `groupId`는 앞에서 본 컨슈머 그룹의 이름입니다. 같은 `groupId`를 적은 컨슈머끼리 한 그룹이 됩니다.

`handler/OrderOrchestrator.java`를 열고 아래 메서드를 작성합니다.

실습 2 handler/OrderOrchestrator.java. 주문 생성 이벤트를 받아 재고 차감 명령 발행

```
@KafkaListener(topics = "order-created", groupId = "orchestrator")
public void orderCreated(OrderCreatedEvent event) {
    // 1. 주문 진행 상태를 메모리에 기록
    states.put(event.orderId(), new WorkflowState(
        event.orderId(), event.address(),
        event.productId(), event.quantity(), event.price()));

    // 2. 다음 단계: 재고 차감 명령 발행
    kafkaTemplate.send("decrease-product-command",
        new DecreaseProductCommand(event.orderId(),
            event.productId(), event.quantity(), event.price()));
}
```

이 메서드는 주문 생성 이벤트를 받아 진행 상태를 기록하고, 다음 명령을 발행합니다. 그림 4-6·4-7의 흐름이 이 한 메서드에 그대로 담겨 있습니다.

`WorkflowState`는 주문 한 건의 진행 정보를 메모리에 들고 있는 객체입니다. 결과 이벤트가 돌아올 때 orchestrator는 이 기록을 보고 다음 명령을 정합니다.

4.4.3 구독 - 토픽의 메시지를 받는다

앞에서 orchestrator가 발행한 '재고 차감 명령'을 이번에는 상품 서비스가 받습니다. 받는 쪽은 메서드에 `@KafkaListener`를 붙이고 토픽 이름과 그룹 이름을 적습니다. 그 토픽에 메시지가 들어오면 이 메서드가 자동으로 실행됩니다. 상품 서비스는 이 명령을 받아 재고를 줄이고, 그 결과를 '재고 차감 이벤트'로 발행합니다.

`adapter/consumer/ProductCommandConsumer.java`를 열고 아래 메서드를 작성합니다.

실습 3 adapter/consumer/ProductCommandConsumer.java. 재고 차감 명령 구독

```
@KafkaListener(topics = "decrease-product-command", groupId = "product-service")
public void decreaseProductCommand(DecreaseProductCommand command) {
    boolean success = false;
    // 1. 재고 차감 (성공하면 success = true)
    try {
        productService.decreaseQuantity(command.productId(), command.quantity(), command.price());
        success = true;
    } catch (Exception e) {
        // 재고 부족 등 실패는 success = false로 그대로 보고
    }

    // 2. 처리 결과를 '재고 차감 이벤트'로 발행
    productEventProducer.publishProductDecreased(
        new ProductDecreasedEvent(command.orderId(), command.productId(), command.quantity(), success));
}
```

상품 서비스는 명령을 받아 재고를 줄이고, 성공·실패를 **success** 값에 담아 결과 이벤트로 돌려줍니다.

각 서비스 코드는 모두 같은 발행·구독 패턴이고, 토픽 이름만 다릅니다. 그래서 코드를 일일이 보지 않아도, 어떤 서비스가 무슨 토픽을 구독해 무엇을 하고 무엇을 발행하는지만 알면 전체가 보입니다.

서비스	구독(받음)	처리	발행(보냄)
order	주문 요청	주문 저장(PENDING)	주문 생성 이벤트
order	주문 완료·취소 명령	주문 상태 변경	—
product	재고 차감·복구 명령	재고 증감	재고 차감 이벤트
delivery	배달 생성 명령	배달 생성	배달 생성 이벤트
orchestrator	주문 생성·재고 차감·배달 생성 이벤트	다음 단계 결정	재고 차감·배달 생성·주문 완료 명령 (실패 시 복구·취소 명령)

각 서비스의 전체 코드는 GitHub에서 확인하세요.

이제 Kubernetes에 Kafka와 orchestrator를 추가하고 전체 시스템을 배포합니다.

4.5 Kubernetes - Kafka와 orchestrator 배포

챕터 3에서 6개 서비스를 Kubernetes에 올렸습니다. Kafka를 도입하면서 더해지는 건 둘입니다. Kafka 서버 한 대와 orchestrator 한 개. 그리고 기존 order-product-delivery는 Kafka가 어디 있는지 알아야 하므로 ConfigMap에 주소 한 줄을 더합니다. 챕터 3 배포에서 바뀌는 부분은 이 세 가지가 전부입니다.

K8s 리소스	역할
<code>kafka-deploy.yaml</code>	KRaft 모드로 Kafka 서버 한 대를 띄웁니다.
<code>kafka-service.yaml</code>	Kafka 서버를 <code>kafka-service:9092</code> 로 노출합니다.
<code>orchestrator-deploy.yaml</code>	워크플로우를 조율하는 orchestrator Pod를 띄웁니다. REST가 없어 Service는 두지 않습니다.
<code>orchestrator-configmap.yaml</code>	orchestrator에 환경변수를 주입합니다.

KRaft 모드는 Kafka가 클러스터 관리 정보를 스스로 처리하는 실행 방식입니다. 덕분에 별도 컴포넌트 없이 Kafka 컨테이너 하나로 동작합니다.

각 서비스는 Kafka 서버에 `kafka-service:9092` 주소로 접근합니다. 클라이언트 쪽은 각 서비스의 ConfigMap에 `SPRING_KAFKA_BOOTSTRAP_SERVERS`로 접속 주소를 적고, Kafka 서버 쪽은 `kafka-deploy.yaml`에 `KAFKA_ADVERTISED_LISTENERS`로 자기 주소를 알립니다. 둘 다 `kafka-service:9092`로 맞춥니다.

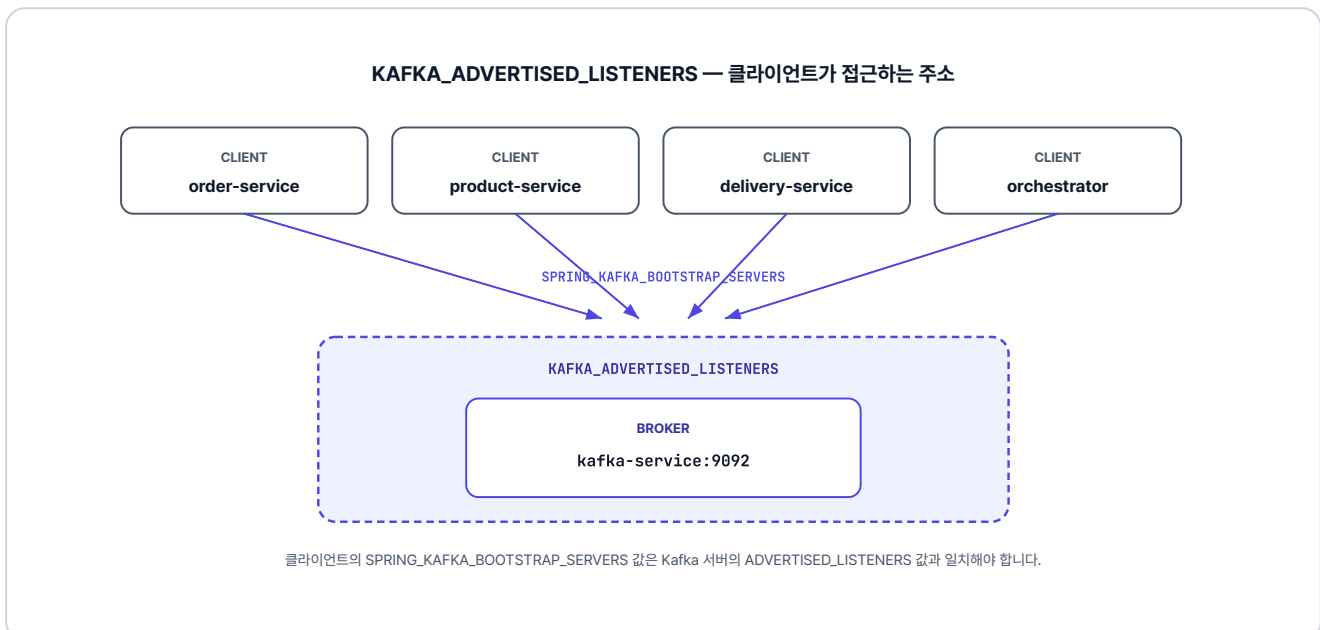


그림 4-14. 클라이언트는 kafka-service:9092로 Kafka에 접근합니다

`kafka-deploy.yml`의 핵심 환경변수는 다음과 같습니다. 전체 YAML은 GitHub에서 확인하세요.

환경변수	역할
<code>CLUSTER_ID</code>	Kafka 서버들을 하나의 클러스터로 묶는 식별자입니다.
<code>KAFKA_PROCESS_ROLES</code>	한 컨테이너가 브로커와 컨트롤러 역할을 겸하도록 지정합니다.
<code>KAFKA_ADVERTISED_LISTENERS</code>	클라이언트가 접근할 주소를 알립니다. ConfigMap의 <code>SPRING_KAFKA_BOOTSTRAP_SERVERS</code> 와 같아야 합니다.
<code>KAFKA_CONTROLLER_QUORUM_VOTERS</code>	컨트롤러 역할을 맡는 Kafka 서버 목록을 지정합니다.

4.6 실행 및 결과 확인

4.6.1 이미지 빌드

Minikube 내부에 이미지를 빌드합니다. 챕터 3 대비 orchestrator 서비스가 새로 추가됩니다.

터미널 이미지 빌드

```
minikube image build -t metacoding/db:2 ./db
minikube image build -t metacoding/gateway:2 ./gateway
minikube image build -t metacoding/order:2 ./order
minikube image build -t metacoding/product:2 ./product
minikube image build -t metacoding/user:2 ./user
minikube image build -t metacoding/delivery:2 ./delivery
minikube image build -t metacoding/orchestrator:2 ./orchestrator
```

4.6.2 배포 순서

Kafka가 준비되기 전에 서비스가 시작되면 연결 오류가 발생합니다. Kafka를 먼저 배포하고 ready 상태를 확인한 다음 나머지를 배포합니다.

터미널 배포 순서 (Kafka 우선)

```
# 1. 네임스페이스 생성
kubectl create namespace metacoding

# 2. Kafka 먼저 배포
kubectl apply -f k8s/kafka

# 3. Kafka가 준비될 때까지 대기
kubectl wait --for=condition=ready pod -l app=kafka -n metacoding --timeout=120s

# 4. 나머지 서비스 배포
kubectl apply -f k8s/db
kubectl apply -f k8s/order
kubectl apply -f k8s/product
kubectl apply -f k8s/user
kubectl apply -f k8s/delivery
kubectl apply -f k8s/gateway
kubectl apply -f k8s/orchestrator

# 5. Ingress 활성화 (최초 1회)
minikube addons enable ingress
```



kubectl apply · Kafka + 서비스 배포

```
namespace/metacoding created

[1] Kafka 우선 배포
deployment.apps/kafka-deploy created
service/kafka-service created
pod/kafka-xxx condition met (28s)

[2] 나머지 서비스 배포
db · order · product · user · delivery · gateway · orchestrator ...

-----
8개 Deployment + 7개 Service 배포 완료 █
```

그림 4-15. Kafka 및 서비스 배포 실행

모든 Pod가 Running 상태인지 확인합니다.

터미널 Pod 상태 확인

```
kubectl get pods -n metacoding
```

```
kubectl get pods -n metacoding
```

NAME	READY	STATUS	AGE
kafka-deploy-7d4c8b9f5-2xk9p	1/1	Running	2m
db-deploy-6f9b7c4d8-m4t2q	1/1	Running	90s
gateway-deploy-5c8d6f7b9-h7w3r	1/1	Running	88s
order-deploy-8b7f6c9d4-q2k8m	1/1	Running	85s
product-deploy-7c9d8b6f5-x4r2t	1/1	Running	83s
user-deploy-6d8c7b9f4-p3m9k	1/1	Running	80s
delivery-deploy-9f7c8b6d5-t6w2x	1/1	Running	78s
orchestrator-deploy-8c6f9b7d4-k9m4q	1/1	Running	75s

8개 Pod Running (Kafka + orchestrator 추가) ■

그림 4-16. Pod 상태 확인 (kubectl get pods)

4.6.3 서비스 접근

Ingress를 통해 외부에서 접속하려면 `minikube tunnel` 을 실행합니다.

```
터미널 외부 접근 터널

minikube tunnel
```

터널이 실행되면 `http://127.0.0.1:80` 로 gateway-service에 접속할 수 있습니다.

4.6.4 비동기 흐름 테스트

AirPods (productId=3)를 주문합니다.

```
POST http://127.0.0.1:80/api/orders

{
  "productId": 3,
  "quantity": 2,
  "price": 300000,
  "address": "Addr 4"
}
```



그림 4-17. 주문 생성 응답 (PENDING 상태)

챗터 3과 다르게 즉시 **PENDING** 상태로 반환됩니다. Kafka 이벤트가 처리되면 상태가 **COMPLETED** 로 변경됩니다. 잠시 후 주문 상태를 다시 조회합니다.

GET <http://127.0.0.1:80/api/orders/4>



그림 4-18. 주문 완료 확인 (COMPLETED 상태)

오픈이 : "PENDING이었다가 COMPLETED로 바뀌었어요! 그런데 롤백도 자동으로 되나요?"

선배 : "품질 상품으로 주문해 봐요. orchestrator 로그를 보면 보일 거예요."

4.6.5 롤백 확인 - 품질 상품 주문

동기 방식에서는 품질 상품이 첫 단계에서 막혀 되돌릴 것이 없었습니다. 반면 비동기 방식에서는 주문이 PENDING으로 먼저 저장됩니다. 그래서 재고 감소가 실패하면 orchestrator가 그 주문을 취소하는 롤백을 시작합니다. iPhone 15(productId=2, 재고 0)로 확인합니다.

POST <http://127.0.0.1:80/api/orders>

```
{
  "productId": 2,
  "quantity": 1,
  "price": 1300000,
  "address": "Addr 5"
}
```

상태: 200 • OK 시간: 30 ms 크기: 351 B

JSON Raw 헤더 4 테스트 결과

응답 본문

```
1  {
2    "status": 200,
3    "msg": "성공",
4    "body": {
5      "id": 5,
6      "userId": 1,
7      "productId": 2,
8      "quantity": 1,
9      "price": 1300000,
10     "status": "PENDING",
11     "createdAt": "2026-05-19T07:03:48.603301791",
12     "updatedAt": "2026-05-19T07:03:48.60336786"
13   }
14 }
```

그림 4-19. 품질 상품 주문 요청

잠시 후 상태를 확인하면 **CANCELLED** 가 됩니다.

GET <http://127.0.0.1:80/api/orders/5>



그림 4-20. 주문 취소 확인 (CANCELLED 상태)

테스트가 끝났으면 이번 챕터에서 실행한 리소스를 정리합니다.

터미널 리소스 정리

```
kubectl delete namespace metacoding
```

상품·배달을 직접 호출하고 실패까지 떠안던 주문 서비스는 이제 주문을 저장하고 이벤트 하나만 발행합니다. 정상 주문은 곧바로 PENDING으로 응답한 뒤 비동기로 COMPLETED가 되고, 품질 주문은 orchestrator가 알아서 CANCELLED로 되돌립니다. 직접 호출이 사라지면서 서비스 사이의 강한 결합이 풀렸고, try/catch 보상 블록도 함께 없어졌습니다.

며칠 뒤, 같은 동료가 책상 앞에 다시 섰습니다. 표정에서 불만이 묻어났습니다.

동료 : "어제 직접 주문해 봤는데, 좀 이상한 게 있었어요."

이것만은 기억하자

- 동기 REST 호출을 **Kafka 비동기 이벤트**로 교체하여 서비스 간 결합을 끊었습니다.
- **orchestrator** 서비스가 Kafka 토픽을 통해 전체 워크플로우를 조율합니다.
- `ConcurrentHashMap`으로 주문별 진행 상태를 추적하고, 실패 시 이미 처리된 단계만 자동 롤백합니다.
- order-service는 주문 생성 즉시 `PENDING` 상태로 반환하고, 처리 완료 후 `COMPLETED`로 갱신됩니다.

다음 챕터에서는 이를 해결합니다. 배달 기사가 완료 API를 호출하여 실제 배달 완료를 처리하고, WebSocket으로 클라이언트에게 주문 완료를 실시간 Push로 알립니다.

챕터 5. 실시간 알림 - 주문 완료를 즉시 전달하다

이 챕터의 전체 소스코드는 <https://github.com/metacoding-12-msa/ex04> 에서 확인할 수 있습니다.

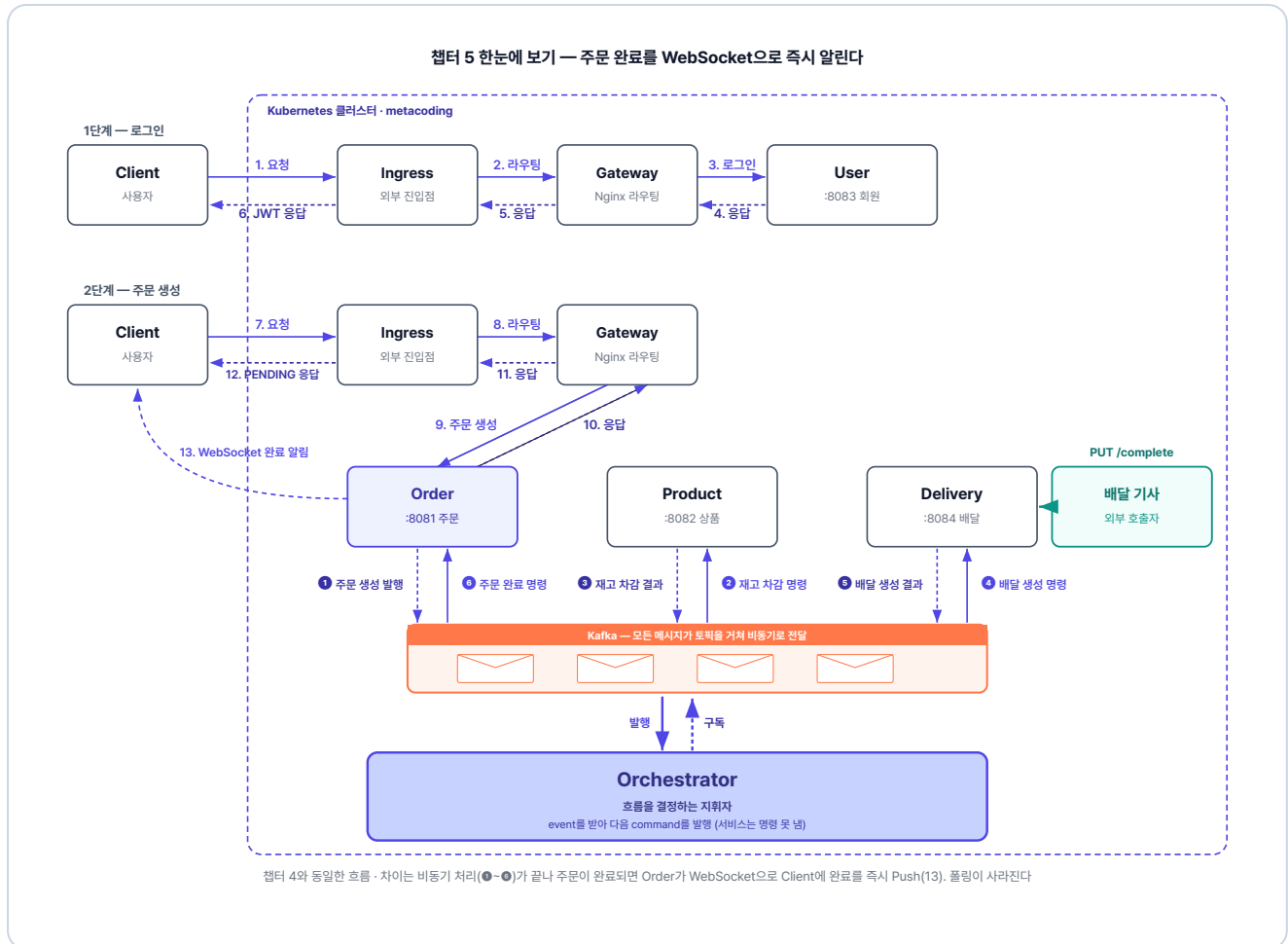


그림 5-1. 챕터 5 한눈에 보기 - 주문 완료를 WebSocket으로 즉시 알린다

이번 챕터가 끝나면

- 폴링의 문제점을 이해하고 WebSocket의 장점을 설명할 수 있습니다.
- 배달 서비스에 배달 완료 API를 추가하고 배달 완료 이벤트를 발행할 수 있습니다.
- 오케스트레이터가 배달 완료 이벤트를 받아 주문 완료 명령을 발행하도록 수정할 수 있습니다.
- 주문 서비스에 WebSocket을 설정하고 주문 완료 시 사용자에게 실시간 알림을 보낼 수 있습니다.
- 전체 시스템을 통합 테스트할 수 있습니다.

준비하기. 실습 시작 전 한 번만 설정

1. 소스 코드 클론

터미널 레포 클론

```
git clone https://github.com/metacoding-12-msa/ex04.git
cd ex04
```

2. 파일 구조

ex04 디렉토리

```
ex04/
├─ order/           # 포트 8081 (WebSocket Push 추가)
├─ product/         # 포트 8082
├─ user/            # 포트 8083
├─ delivery/        # 포트 8084 (배달 완료 API 추가)
├─ orchestrator/    # Kafka 워크플로우 조율
├─ frontend/        # Nginx + SockJS 클라이언트 (이번 챕터 신규)
├─ gateway/         # Nginx API Gateway
├─ db/              # MySQL
└─ k8s/             # Kubernetes 매니페스트 (kafka·frontend 포함)
```

서비스마다 패키지 구조가 조금씩 다르므로, 코드를 작성할 파일 경로는 각 실습 코드블록 바로 위에서 안내합니다.

3. 실습 환경

챕터 4까지 사용한 Docker Desktop, Minikube가 그대로 필요합니다. 별도 추가 도구는 없습니다. 프론트엔드도 Nginx 컨테이너로 Minikube 안에서 함께 띄우므로 따로 서버를 띄울 필요는 없습니다.

4. 실습 순서

1. 배달 서비스에 배달 완료 API + `delivery-completed` 이벤트 추가
2. 오케스트레이터에 `delivery-completed` 처리 + `delivery-created` 성공 시 대기로 변경
3. 주문 서비스에 STOMP WebSocket 설정 + 주문 완료 시 Push
4. SockJS 기반 index.html 프론트엔드와 Nginx 프록시 구성
5. K8s에 frontend 추가 배포 → 통합 시나리오 검증

5.1 처리 중에서 멈춘 화면

베타 테스터로 등록한 동료가 자리로 왔습니다. 표정이 떨떠름했습니다.

동료 : "어제 책 한 권 주문했는데, 화면이 계속 '처리 중'이더라고요. 끝났는지 알 수가 없어서 한참 뒤에 주문 내역을 다시 열어 봤어요. 그랬더니 '배달 완료'래요. 근데 책은 오늘까지 안 왔어요."

오픈이는 배달 코드를 따라가 봤습니다. 배달은 만들어지자마자 곧바로 완료로 바뀌었습니다. 정작 물건이 진짜 전달되는 순간은 코드 어디에도 없었습니다. 그렇게 찍힌 완료조차 화면에 바로 뜨지 않아, 동료는 끝났는지 직접 확인할 수밖에 없었습니다.

이 문제를 들고 선배에게 갔습니다.

오픈이 : "배달이 만들어지자마자 완료로 찍혀요. 지금 이대로 사용자에게 '완료'라고 알리면 거짓말입니다. 어디부터 손대야 하죠?"

선배 : "알림은 진짜 끝났을 때 보내야 의미가 있어요. 진짜 끝나는 순간은 곧 만들 거예요. 그 전에, 사용자가 끝났는지 왜 직접 확인해야 했는지부터 봐요."

5.2 WebSocket - 폴링의 한계를 넘다

5.2.1 폴링 vs 푸시

서버에 생긴 변화를 알아내는 방법은 크게 두 가지입니다.

택배가 왔는지 현관문을 5분마다 열어 확인합니다. 도착했는지 아닌지는 직접 열어 봐야만 알 수 있습니다. 이렇게 클라이언트가 서버에 "완료됐나요?"라고 일정한 간격으로 반복해서 묻는 방식이 **폴링(Polling)**입니다. 서버는 물어볼 때마다 그 시점의 상태를 응답할 뿐이라, 아직 변화가 없으면 클라이언트는 같은 질문을 다음 간격에 다시 보냅니다.

반대로 택배가 도착하면 초인종이 한 번 울립니다. 안에 있는 사람은 문을 여닫으며 확인할 필요 없이, 울리는 순간 도착을 압니다. 이렇게 클라이언트가 묻지 않아도 서버가 변화가 생긴 순간 먼저 알리는 방식이 **푸시(Push)**입니다. 클라이언트는 반복해서 묻지 않고, 변화가 생기면 그 즉시 화면까지 전달됩니다.

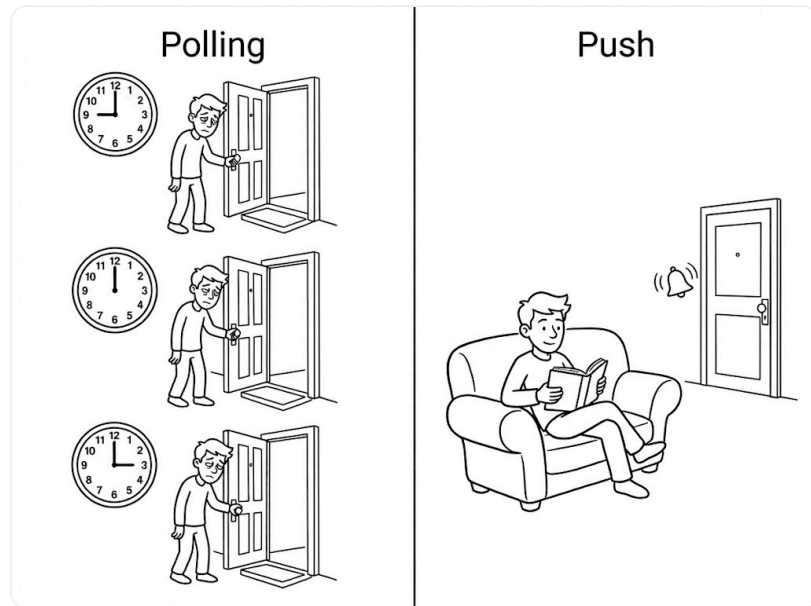


그림 5-2. 폴링 vs 푸시

동료가 주문 내역을 몇 번이고 다시 열어 봐야 했던 건 화면에 초인종이 없었기 때문입니다. 사용자에게 끝난 사실을 곧바로 알려려면, 서버가 먼저 보내는 푸시가 필요합니다.

5.2.2 WebSocket

푸시를 구현하는 기술이 **WebSocket**입니다.

HTTP 요청-응답은 편지와 같습니다. 한 통 보내고 답장이 오면 한 번의 왕복이 끝나고, 또 궁금하면 다시 한 통을 보내야 합니다. 그래서 보내는 쪽이 묻지 않으면 상대는 아무것도 전할 수 없습니다.

반대로 WebSocket은 전화 통화에 가깝습니다. 한 번 연결되면 끊지 않고 그대로 이어 두어, 묻지 않아도 어느 쪽이든 먼저 말할 수 있습니다. 서버는 변화가 생긴 순간 "주문 완료"라고 먼저 전합니다.



그림 5-3. 편지와 전화로 본 HTTP 요청·응답과 WebSocket

WebSocket이란? 클라이언트와 서버가 한 번 연결을 맺으면 끊지 않고 유지하는 통신 방식입니다. 연결이 살아 있는 동안에는 서버가 요청을 기다릴 필요 없이 클라이언트에 데이터를 보낼 수 있습니다. 상태가 바뀌면 그 즉시 알림이 전달됩니다.

WebSocket을 붙이면 서버는 주문이 완료되는 순간 사용자 화면으로 곧바로 알림을 보낼 수 있습니다. 이제 주문 완료를 위해 배달 완료 기능을 추가해 보겠습니다.

5.3 배달 완료 - 생성과 완료를 분리한다

이제부터는 접수하면 배달이 **PENDING**으로 남고, 배달 기사가 전달한 뒤에야 **COMPLETED**가 됩니다. 배달이 만들어진 뒤 완료되기까지를 순서대로 보겠습니다.



그림 5-4. 1단계 - 배달 생성 이벤트 발행 → 오케스트레이터 수신 후 대기

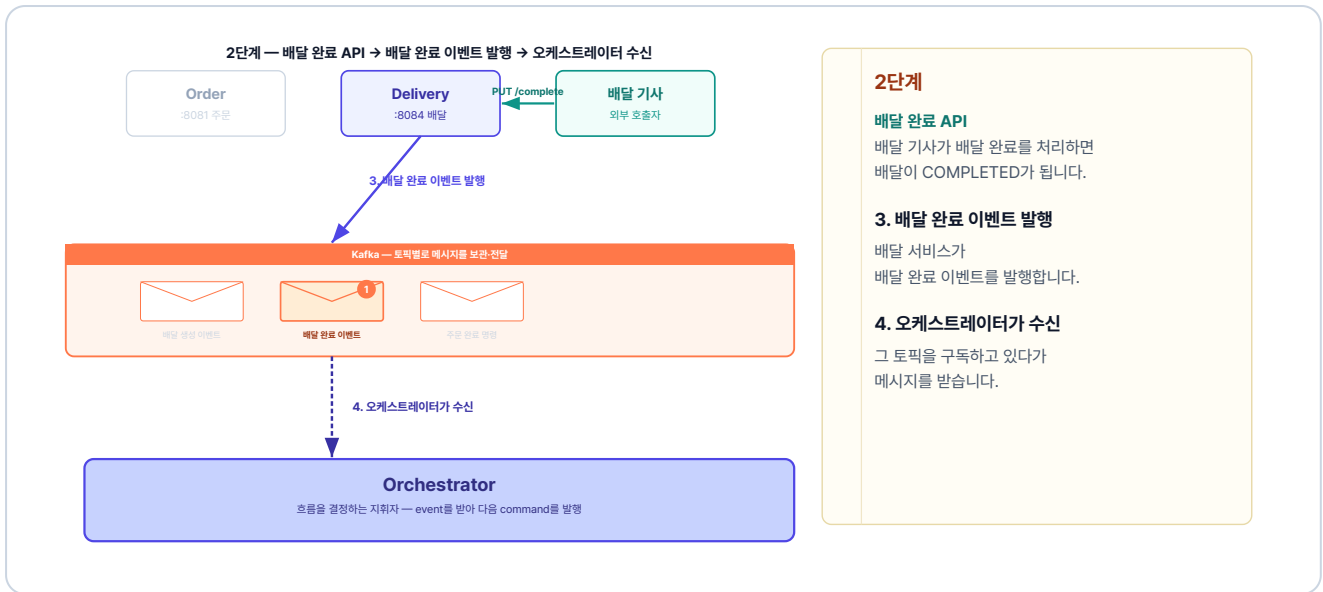


그림 5-5. 2단계 - 배달 완료 API → 배달 완료 이벤트 발행 → 오케스트레이터 수신

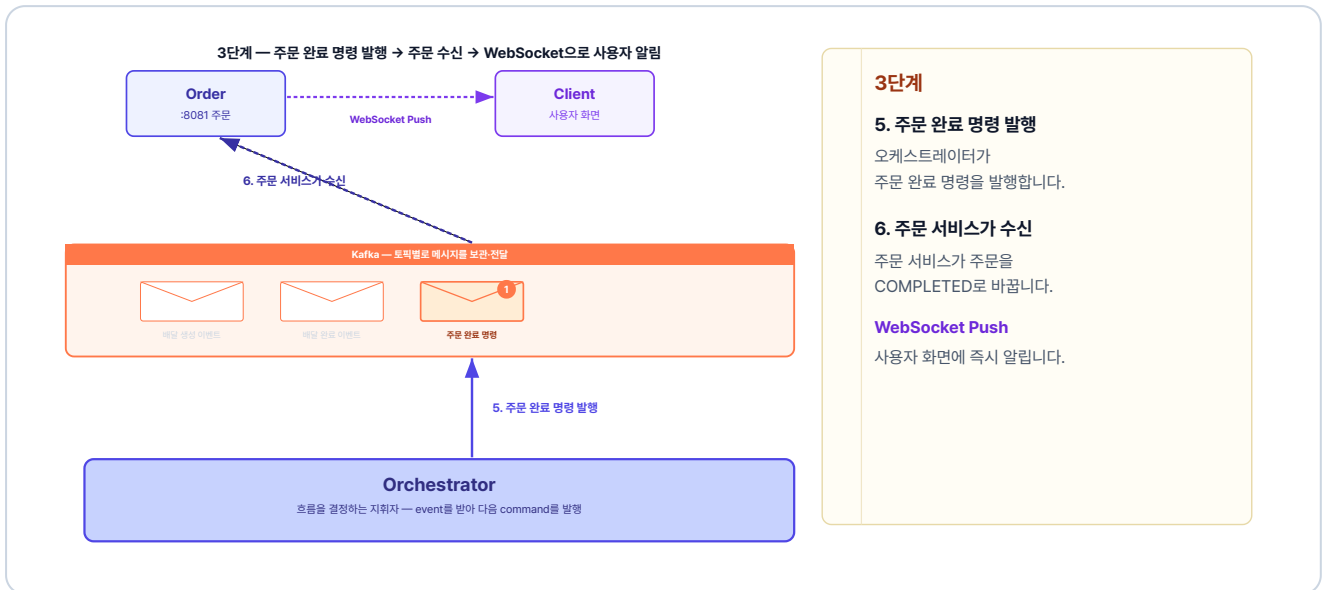


그림 5-6. 3단계 - 주문 완료 명령 발행 → 주문 수신 → WebSocket 알림

이 세 단계가 차례로 이어지면 배달이 진짜 끝나는 순간 사용자에게 완료 알림이 전달됩니다. 첫 단계가 일어나는 배달 서비스부터 구현합니다.

5.4 배달 서비스 - 배달 완료 API

배달 서비스에서 손볼 곳은 두 군데입니다. 생성에서 완료를 분리하고, 배달 기사가 호출할 완료 처리를 더합니다. 두 작업에 앞서 컨트롤러·포트·이벤트부터 보겠습니다.

5.4.1 컨트롤러·포트·이벤트 발행

요청을 받는 컨트롤러, 컨트롤러가 부르는 포트, 완료 이벤트를 발행하는 코드는 패턴이 단순해 표로 정리합니다. 전체 코드는 깃헙 레포에서 확인합니다.

파일	역할
<code>web/DeliveryController.java</code>	배달 완료 API 요청을 받아 처리로 넘김
<code>usecase/CompleteDeliveryUseCase.java</code>	배달 완료 기능을 정의한 인터페이스
<code>adapter/producer/DeliveryEventProducer.java</code>	배달 완료를 카프카로 알림 (앞 챕터와 같은 방식)

5.4.2 생성과 완료 분리

완료 요청을 받을 컨트롤러와 포트는 갖췄습니다. 그런데 배달은 아직 만들어지자마자 곧바로 완료로 바뀝니다. 앞에서 본 1단계가 여기입니다. 이 완료 호출을 지워 생성과 완료를 분리하면 배달은 PENDING으로 남고, 완료는 배달 기사가 직접 호출할 때로 미뤄집니다.

`usecase/DeliveryService.java` 의 `createDelivery` 를 아래처럼 고칩니다.

실습 1 usecase/DeliveryService.java. 생성 시 완료 호출 제거

```
@Transactional
public DeliveryResponse createDelivery(int orderId, String address) {
    Delivery createdDelivery = deliveryRepository.save(Delivery.create(orderId, address));
    Delivery.validateAddress(address);
    // 삭제: createdDelivery.complete(); ← 생성 시 완료 호출 제거
    return DeliveryResponse.from(createdDelivery);
}
```

5.4.3 서비스 - 완료 처리

실습 1에서 완료 호출을 지운 `DeliveryService` 에 이번엔 배달 기사가 호출할 완료 메서드를 더합니다.

실습 2 usecase/DeliveryService.java. completeDelivery 추가

```
@Override
@Transactional
public DeliveryResponse completeDelivery(int deliveryId) {
    Delivery findDelivery = deliveryRepository.findById(deliveryId)
        .orElseThrow(() -> new Exception404("배달 정보를 조회할 수 없습니다."));
    findDelivery.complete();
    deliveryEventProducer.publishDeliveryCompleted(new DeliveryCompletedEvent(findDelivery.getId()));
    return DeliveryResponse.from(findDelivery);
}
```

배달 기사가 완료를 호출하면 배달이 완료되고, 완료 이벤트가 발행됩니다. 이제 오케스트레이터가 완료 이벤트를 받도록 수정합니다.

5.5 오케스트레이터 - 배달 완료 이벤트 처리 추가

오케스트레이터는 배달 생성 결과를 받아도 더 진행하지 않고, 배달 완료를 받아 주문을 끝냅니다.

5.5.1 deliveryCreated 수정 - 성공 시 발행 중단

배달 생성이 성공하면 더 이상 주문 완료 명령을 보내지 않습니다.

`handler/OrderOrchestrator.java`의 `deliveryCreated` 를 아래처럼 수정합니다.

실습 3 handler/OrderOrchestrator.java. deliveryCreated - 성공 시 발행 중단

```
@KafkaListener(topics = "delivery-created", groupId = "orchestrator")
public void deliveryCreated(DeliveryCreatedEvent event) {
    int orderId = event.orderId();
    WorkflowState state = states.get(orderId);
    if (state == null) return;

    if (!event.success()) {
        // 실패: 재고 복구 + 주문 취소 (앞 챕터와 동일)
        ...
        return;
    }

    // 성공
    // ↓ 아래 5줄을 삭제합니다 (전에는 배달 생성이 성공하면 여기서 주문 완료 명령을 보냈습니다)
    kafkaTemplate.send(
        "complete-order-command",
        String.valueOf(orderId),
        new CompleteOrderCommand(orderId)
    );

    states.remove(orderId);
}
```

실패 처리는 앞과 같습니다. 성공 분기만 달라집니다.

5.5.2 deliveryCompleted 추가 - 배달 완료 시 주문 완료

handler/OrderOrchestrator.java 에 deliveryCompleted 리스너를 추가합니다.

실습 4 handler/OrderOrchestrator.java. deliveryCompleted - 주문 완료 명령 발행

```
@KafkaListener(topics = "delivery-completed", groupId = "orchestrator")
public void deliveryCompleted(DeliveryCompletedEvent event) {
    // 배달기사가 완료 API를 호출한 시점 → 주문 완료 명령 발행
    kafkaTemplate.send(
        "complete-order-command",
        String.valueOf(event.orderId()),
        new CompleteOrderCommand(event.orderId())
    );
}
```

배달 기사가 완료를 호출하면, 주문 완료와 화면 알림까지 이어집니다.

여기까지 다 이어졌는데, 정작 사용자한테는 어떻게 알리지?

5.6 주문 서비스 - STOMP로 실시간 Push 구현

남은 곳은 주문 서비스입니다. 주문 완료 명령을 받으면 주문을 끝내고, 사용자 화면으로 곧바로 알립니다.

5.6.1 WebSocket 설정

WebSocketConfig는 WebSocket을 켜고, 클라이언트가 `/api/ws/orders`로 실시간 연결할 엔드포인트를 등록합니다.

WebSocket 위에 STOMP 프로토콜을 사용합니다. STOMP를 얹으면 "이 채널을 구독한다", "이 채널에 메시지를 보낸다" 같은 발행-구독 구조를 쓸 수 있습니다. 클라이언트가 `/topic/orders/{userId}` 채널을 구독하면 해당 사용자에게만 알림이 전달됩니다.

`core/config/WebSocketConfig.java`를 열고 아래 클래스를 작성합니다.

실습 5 core/config/WebSocketConfig.java. STOMP WebSocket 설정

```
@Configuration
@EnableWebSocketMessageBroker // STOMP WebSocket 브로커 활성화
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void configureMessageBroker(MessageBrokerRegistry config) {
        // /topic 접두사로 메시지 라우팅 (Kafka 토픽과는 다른 개념)
        config.enableSimpleBroker("/topic");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        // WebSocket 연결 엔드포인트 - withSockJS()는 미지원 브라우저용 폴백
        registry.addEndpoint("/api/ws/orders").setAllowedOriginPatterns("*").withSockJS();
    }
}
```

5.6.2 SimpMessagingTemplate - 주문 완료 시 Push 발송

`completeOrder` 메서드에서 `/topic/orders/{userId}` 채널에 메시지를 보내면 `userId`에 해당하는 사용자만 알림을 수신합니다.

SimpMessagingTemplate이란? Spring이 제공하는 메시지 전송 도구입니다.

`convertAndSend(destination, payload)` 메서드로 지정한 채널을 구독한 모든 클라이언트에게 메시지를 Push합니다.

`usecase/OrderService.java`의 `completeOrder` 메서드를 아래처럼 수정합니다.

실습 6 usecase/OrderService.java. completeOrder + WebSocket Push

```
private final SimpMessagingTemplate messagingTemplate; // 추가: 생성자 주입

@Transactional
public void completeOrder(int orderId) {
    Order findOrder = orderRepository.findById(orderId)
        .orElseThrow(() -> new Exception404("주문을 찾을 수 없습니다."));
    findOrder.complete();
    messagingTemplate.convertAndSend("/topic/orders/" + findOrder.getUserId(), (Object) Map.of("orderId", orderId)); // 추가: WebSocket Push
}
```

서버 측 WebSocket 구현이 끝났습니다. 동작에 필요한 보조 설정과 클라이언트만 남았습니다.

클라이언트가 알림을 받으려면, 서버가 정한 주소를 클라이언트도 똑같이 써야 합니다.



그림 5-7. WebSocket 주소 일치 - 같은 색은 글자까지 똑같아야 동작합니다

5.6.3 보조 설정과 클라이언트

핵심 코드 외에 필요한 설정과 클라이언트는 표로 정리합니다. 전체 코드는 깃헙 레포에서 확인합니다.

파일	역할
<code>order-service/build.gradle</code>	WebSocket-STOMP 의존성 추가
<code>JwtAuthenticationFilter.java</code>	WebSocket 연결 경로를 로그인 검사에서 제외
<code>frontend/nginx.conf, gateway/nginx.conf</code>	WebSocket 연결을 끊지 않도록 <code>/api/ws/</code> 요청에 업그레이드 헤더 추가
<code>frontend/index.html</code>	서버 알림을 받아 화면에 주문 완료를 표시

업그레이드 헤더란?

보통 HTTP 요청은 한 번 주고받으면 연결이 끝납니다. WebSocket은 연결을 끊지 않고 계속 열어 두는 방식이라, 처음 연결할 때 "일반 HTTP가 아니라 계속 유지하는 WebSocket으로 바꾸자"고 알리는 신호, 곧 업그레이드 헤더가 필요합니다.

문제는 브라우저와 서버 사이에 프록시(frontend-gateway)가 있다는 점입니다. 프록시가 업그레이드 헤더를 전달하지 않으면 일반 요청처럼 처리돼 연결이 끊깁니다. 그래서 두 프록시의 `/api/ws/` 설정 모두에 업그레이드 헤더를 전달하도록 적어 둡니다.

5.7 전체 시스템 통합 테스트

모든 구현이 완료됐습니다. 이제 전체 시스템을 실행하고 처음부터 끝까지 한 번에 흐름을 확인합니다.

5.7.1 K8s 매니페스트

앞 챕터까지의 구성에서 frontend 서비스가 새로 추가됩니다. `k8s/frontend/` 폴더에 Deployment, Service, Ingress가 정의되어 있습니다.

파일	역할
<code>frontend-deploy.yml</code>	Nginx 기반 프론트엔드 Pod
<code>frontend-service.yml</code>	클러스터 내부 접근용 Service
<code>frontend-ingress.yml</code>	외부 요청을 frontend-service로 라우팅

이번 챕터부터는 Ingress가 gateway-service가 아닌 **frontend-service**를 가리킵니다. 프론트엔드의 Nginx가 정적 파일을 직접 제공하고, `/api/` 요청만 gateway-service로 전달합니다.

5.7.2 이미지 빌드

Minikube 내부에 이미지를 빌드합니다.

터미널 이미지 빌드

```
minikube image build -t metacoding/db:3 ./db
minikube image build -t metacoding/gateway:3 ./gateway
minikube image build -t metacoding/order:3 ./order
minikube image build -t metacoding/product:3 ./product
minikube image build -t metacoding/user:3 ./user
minikube image build -t metacoding/delivery:3 ./delivery
minikube image build -t metacoding/orchestrator:3 ./orchestrator
minikube image build -t metacoding/frontend:3 ./frontend
```

5.7.3 배포

Kafka를 먼저 배포하고 ready 상태를 확인한 다음 나머지를 배포합니다.

터미널 배포 순서 (Kafka 우선)

```
# 1. 네임스페이스 생성
kubectl create namespace metacoding

# 2. Kafka 먼저 배포
kubectl apply -f k8s/kafka

# 3. Kafka가 준비될 때까지 대기
kubectl wait --for=condition=ready pod -l app=kafka -n metacoding --timeout=120s

# 4. 나머지 서비스 배포
kubectl apply -f k8s/db
kubectl apply -f k8s/gateway
kubectl apply -f k8s/order
kubectl apply -f k8s/product
kubectl apply -f k8s/user
kubectl apply -f k8s/delivery
kubectl apply -f k8s/orchestrator
kubectl apply -f k8s/frontend

# 5. Ingress 활성화 (최초 1회)
minikube addons enable ingress
```

모든 Pod가 Running 상태가 될 때까지 대기합니다.

터미널 Pod 상태 확인

```
kubectl get pods -n metacoding
```

```
kubectl get pods -n metacoding
```

NAME	READY	STATUS	AGE
kafka-deploy-7d4c8b9f5-2xk9p	1/1	Running	2m
db-deploy-6f9b7c4d8-m4t2q	1/1	Running	90s
gateway-deploy-5c8d6f7b9-h7w3r	1/1	Running	88s
order-deploy-8b7f6c9d4-q2k8m	1/1	Running	85s
product-deploy-7c9d8b6f5-x4r2t	1/1	Running	83s
user-deploy-6d8c7b9f4-p3m9k	1/1	Running	80s
delivery-deploy-9f7c8b6d5-t6w2x	1/1	Running	78s
orchestrator-deploy-8c6f9b7d4-k9m4q	1/1	Running	75s
frontend-deploy-7b9c6d8f5-w3k2m	1/1	Running	72s

9개 Pod Running (frontend 추가) █

그림 5-8. Pod 상태 확인

5.7.4 서비스 접근

Ingress를 통해 외부에서 접속하려면 `minikube tunnel` 을 실행합니다.

```
터미널 외부 접근 터널
minikube tunnel
```

터널이 실행되면 `http://127.0.0.1:80` 로 프론트엔드에 접속할 수 있습니다.

5.7.5 통합 테스트 시나리오

Step 1: WebSocket 연결 및 주문 생성 (클라이언트 역할)

브라우저를 통해 index.html에 접속합니다.

```
브라우저 http://127.0.0.1:80/index.html
```

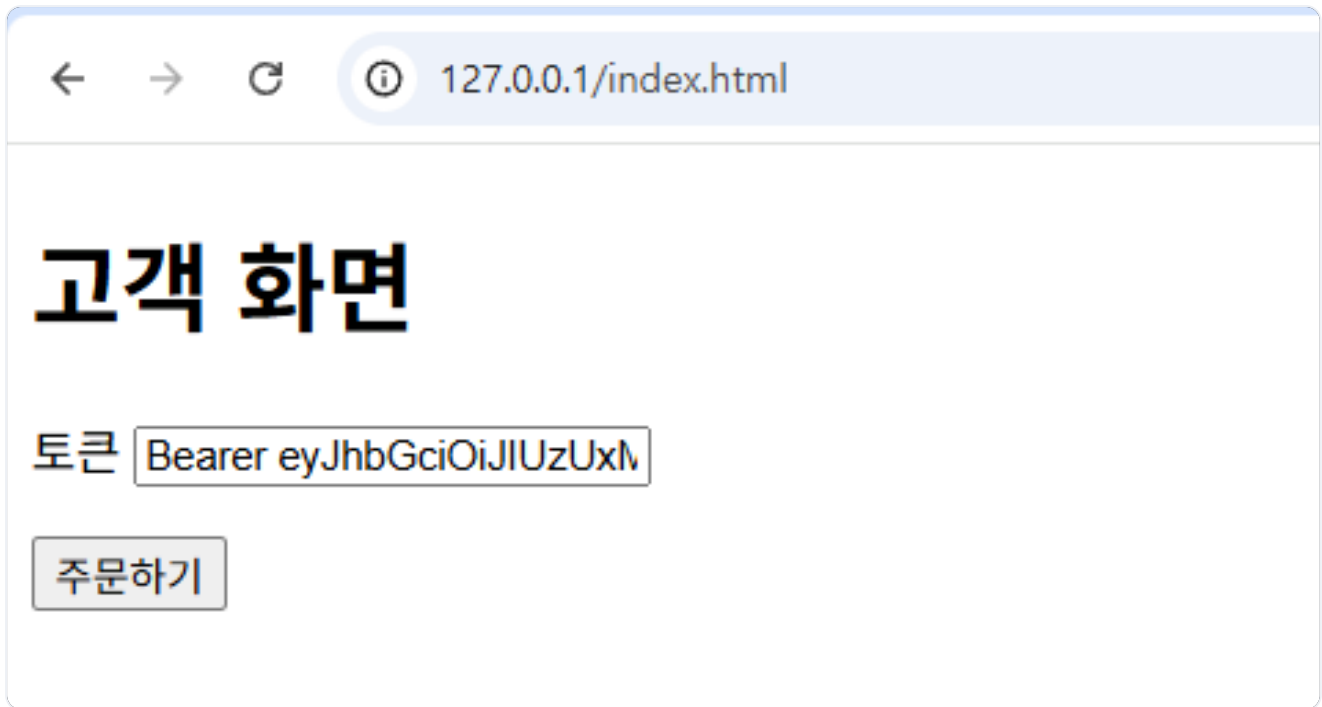



그림 5-9. 브라우저에서 index.html 접속 화면

WebSocket 연결을 위해 토큰을 입력합니다. 로그인 API(`POST /login`)로 발급받은 JWT 토큰을 입력합니다. `Bearer` 접두사는 붙어 있어도 그대로 인식됩니다.

토큰을 입력하고 주문하기 버튼을 클릭합니다. index.html이 내부적으로 WebSocket에 연결하고 `/topic/orders/{userId}` 채널을 구독한 뒤 다음과 같은 주문 요청을 보냅니다.

`POST /api/orders`

```
{
  "productId": 1,
  "quantity": 1,
  "price": 2500000,
  "address": "Addr 4"
}
```

이 요청은 주문하기 버튼을 누르면 index.html이 자동으로 보냅니다. 따로 직접 호출하지 않습니다.

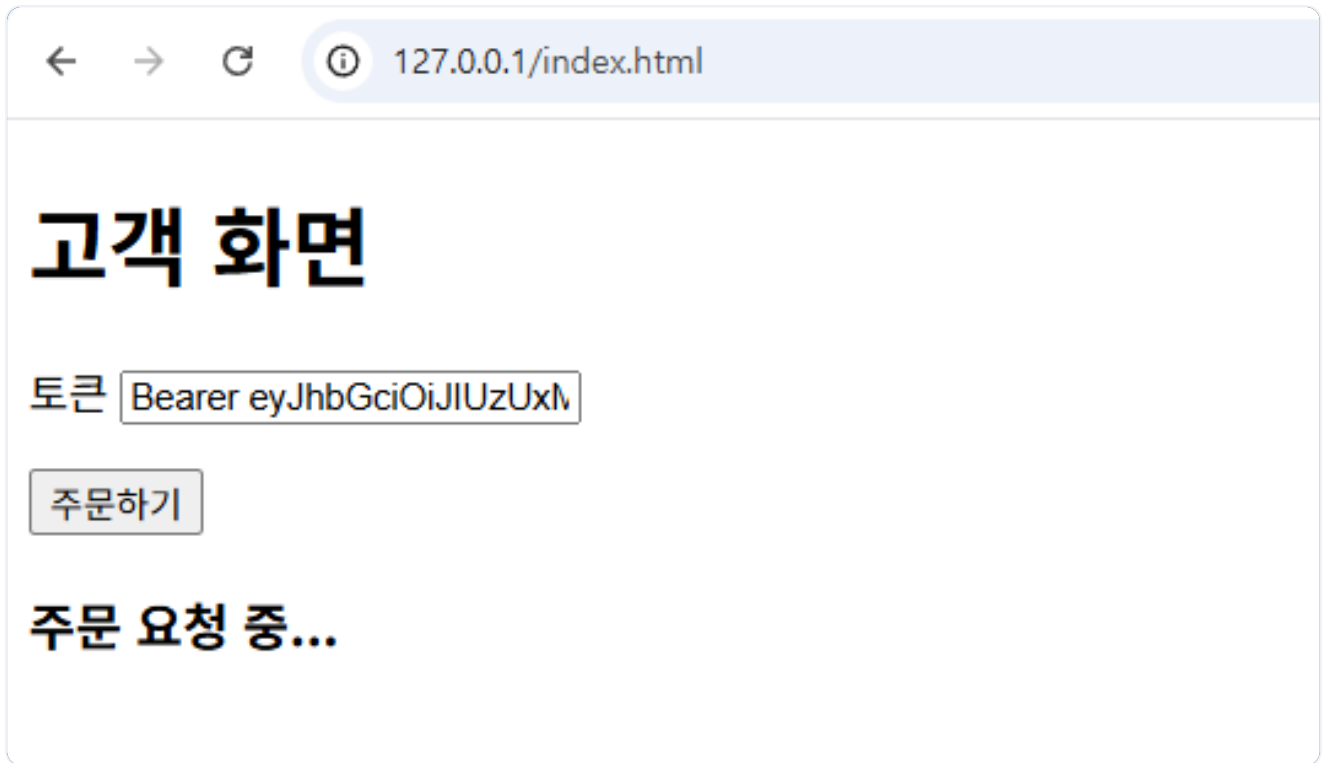


그림 5-10. 토큰 입력 후 주문하기 버튼 클릭

브라우저 **F12** - **Console** 에서 WebSocket이 연결됨을 확인할 수 있습니다.

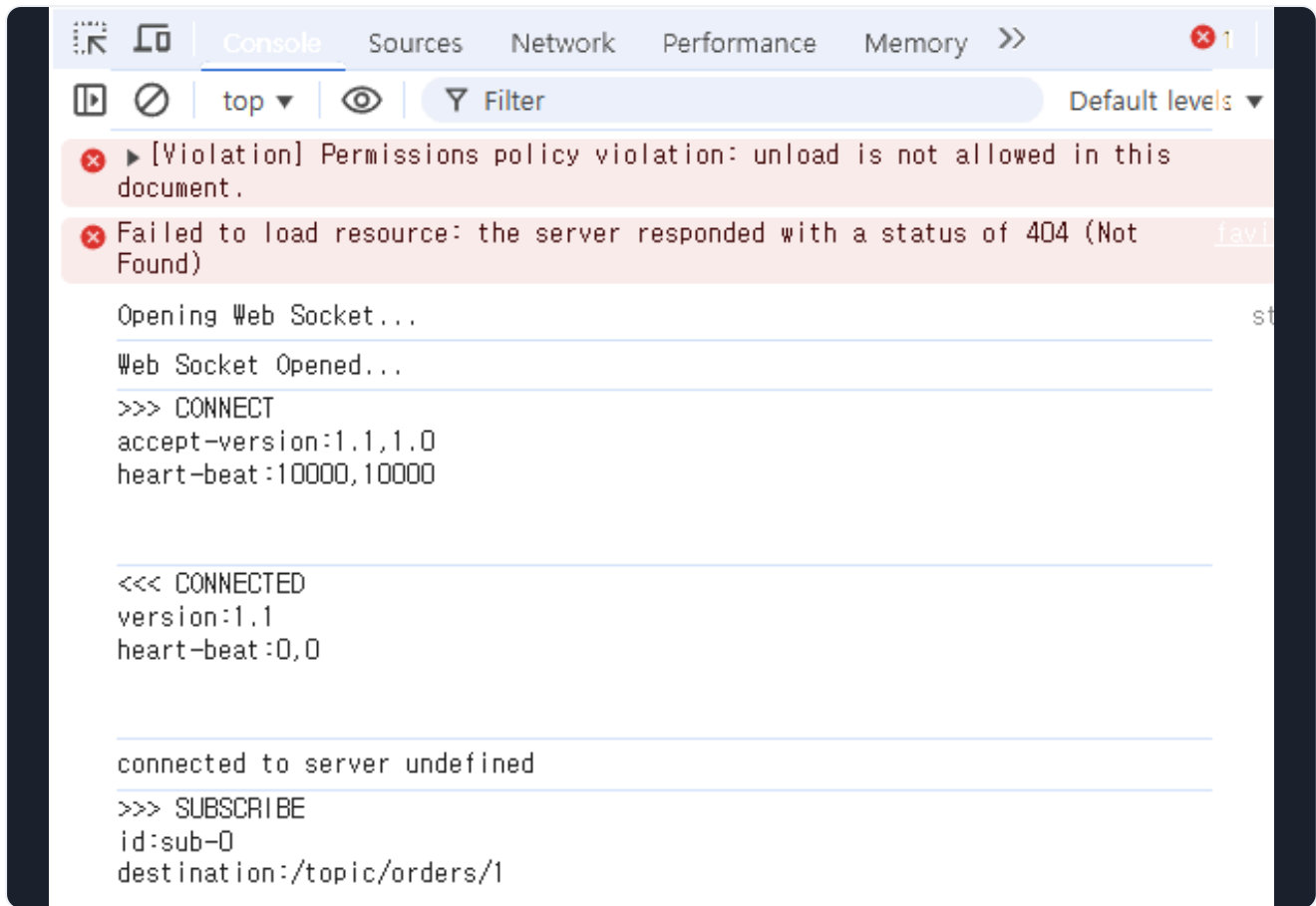


그림 5-11. 브라우저 Console에서 WebSocket 연결 확인

Hoppscotch로 생성된 주문을 확인하면 **PENDING** 상태로 머물러 있습니다.

GET <http://127.0.0.1:80/api/orders/4>



그림 5-12. 주문 조회 결과 - PENDING 상태

Step 2: 배달 완료 (배달 기사 역할)

먼저 생성된 배달을 확인해보겠습니다.

GET <http://127.0.0.1:80/api/deliveries/4>



그림 5-13. 배달 조회 결과 - PENDING 상태

배달 ID가 4인 배달이 **PENDING** 상태로 생성되었습니다.

배달 기사가 물건을 전달한 뒤 배달 완료 처리를 합니다.

PUT <http://127.0.0.1:80/api/deliveries/4/complete>

상태: 200 • OK 시간: 32 ms 크기: 321 B

JSON Raw 헤더 4 테스트 결과

응답 본문

```

1  {
2    "status": 200,
3    "msg": "성공",
4    "body": {
5      "id": 4,
6      "orderId": 4,
7      "address": "Addr 4",
8      "status": "COMPLETED",
9      "createdAt": "2026-05-19T07:46:38",
10     "updatedAt": "2026-05-19T07:49:08.447194796"
11   }
12 }

```

그림 5-14. 배달 완료 API 호출 결과 - COMPLETED 상태

배달 완료 버튼을 눌렀다. 이제 주문도 바뀌었을까?

Step 3: 주문 완료 및 웹소켓 응답 확인

배달 완료 처리 후, 주문 완료 명령으로 주문이 **COMPLETED** 상태가 됩니다.

GET <http://127.0.0.1:80/api/orders/4>

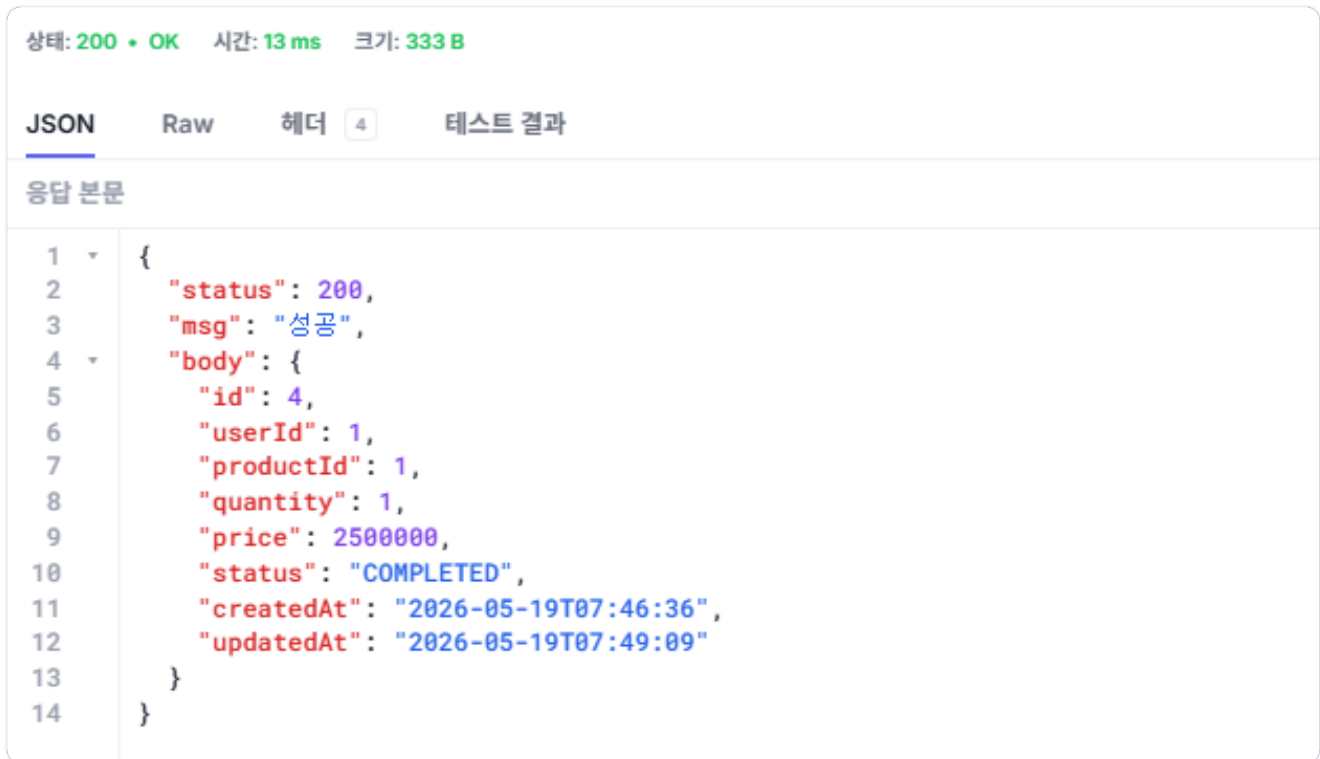


그림 5-15. 주문 조회 결과 - COMPLETED 상태

주문이 완료되면 WebSocket이 클라이언트에게 주문 완료 메시지를 전송합니다. WebSocket 응답을 수신하면 클라이언트 화면이 주문 완료 상태로 변경됩니다.



그림 5-16. WebSocket 알림 수신 - 클라이언트 화면에 주문 완료 표시

완성된 시스템을 동료에게 보여줬습니다. 같은 화면, 같은 주문, 같은 배달이었지만 이번에는 다음 날 책이 도착할 때까지 동료가 주문 내역을 다시 열어 보지 않았습니다.

동료 : "전엔 '완료'라고 떠도 못 믿어서 주문 내역을 자꾸 다시 열어 봤는데, 이번엔 뜨자마자 진짜 끝난 거였어요."

진짜 끝났을 때만 알린다. 그 한 줄 차이였다.

이것만은 기억하자

- **배달 완료 라이프사이클**: 배달 생성 시 `PENDING`, 배달 기사 API 호출 시 `COMPLETED` 로 실제 배달 완료 시점을 정확히 반영합니다.
- **delivery-completed 토픽 추가**: 배달 서비스가 완료 이벤트를 발행하고 오케스트레이터가 받아 주문 완료 명령을 발행합니다. 이로써 실제 사용하는 Kafka 토픽이 9개가 됩니다.
- **오케스트레이터 변경**: 배달 생성(`delivery-created`)이 성공하면 대기하고, 배달 완료(`delivery-completed`)를 받으면 주문 완료를 처리합니다. 실패 시에는 재고를 복구합니다.
- **WebSocket Push**: 주문 서비스가 `SimpMessagingTemplate` 으로 사용자별 채널에 실시간 알림을 보냅니다.

오픈이는 문득 그날 새벽을 떠올렸습니다. 휴대폰 두 대가 동시에 울리고, 대시보드가 빨강게 차 있고, 주문 API 옆에서 로그인 API가 같이 죽어 있던 새벽 세 시.

주문이랑 로그인이 무슨 상관이야?

그때는 답을 몰랐습니다. 지금은 압니다. 회원도, 상품도, 주문도, 배달도 각자의 서비스로 분리됐고, 하나가 흔들려도 나머지는 멈추지 않습니다. Kafka에 쌓인 메시지는 사라지지 않고, 오케스트레이터가 실패한 주문을 되돌리며, 완료된 주문은 사용자 화면에 바로 표시됩니다.

선배가 지나가다 모니터를 봤습니다.

선배 : "이제 새벽에 안 깨워도 되겠네요."

오픈이는 대답 대신 웃었습니다. 모놀리식 한 덩어리에서 시작해, 코드를 한 줄씩 바꿔 여기까지 왔습니다.